



République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université des Sciences et de la Technologie Houari Boumédiène

Faculté d'Informatique
Département d'Intelligence Artificielle et Sciences des Données

Mémoire de Master

Spécialité : Systèmes informatiques intelligents

Prédiction et optimisation des processus dans la raffinerie d'Alger

Encadré par :

- Mr. TAKDJOUT Amine
- Mme. HEDJAZI-DELLAL Badiâa

Réalisé par :

- AIRECHE Maria
- AIT AMARA Mohamed

Présenté devant le jury composé de :

- Président du jury : Mme. BOUGHACI Dalila
- Membre du jury : Mme. BELAGGOUN Nassima

Projet No : SII_05
2024/2025

Remerciements

En premier lieu, nous remercions Dieu, le Tout-Puissant, qui nous a accordé la force et la volonté de réaliser ce travail.

Il nous est agréable de remercier toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce mémoire.

Nous tenons à exprimer notre profonde reconnaissance à Madame HEDJAZI-DELLAL Badiâa, notre encadreuse académique, pour sa disponibilité, son accompagnement méthodique et ses conseils précieux tout au long du projet.

Nous tenons également à remercier chaleureusement Monsieur TAKDJOUT Amine, notre encadreur industriel chez Sonatrach, pour la richesse de ses retours, sa rigueur professionnelle et la confiance qu'il nous a accordée dans un contexte industriel exigeant.

Nos remerciements vont aussi à Madame LOULOU Farah, ingénieure de procédés chez Sonatrach, pour son expertise métier, ses explications techniques et son implication dans l'interprétation des résultats.

Enfin, nos pensées les plus sincères vont à nos parents, qui n'ont cessé de nous encourager, de nous soutenir moralement et de croire en nous, même dans les moments les plus difficiles.

Dédicaces

À nos parents, pour leur soutien constant tout au long de notre parcours.

À nos frères et sœurs, pour leur accompagnement indéfectible.

À nos amis, pour leur aide et leurs encouragements.

À tous ceux qui s'intéressent à ce travail académique.

Nous dédions ce mémoire à l'ensemble de ces personnes.

Résumé

Ce travail présente la conception d'un pipeline intelligent combinant des techniques de prévision et de détection d'anomalies pour la surveillance d'un procédé industriel complexe (RFCC) à la raffinerie d'Alger. L'approche repose sur un modèle LSTM avec mécanisme d'attention multi-tête pour anticiper les valeurs futures des capteurs, couplé à un autoencodeur BiLSTM pour détecter les écarts anormaux. Des heuristiques spécifiques au domaine, basées sur des seuils critiques validés par les ingénieurs procédé, sont également intégrées afin d'améliorer la fiabilité du système de détection.

L'évaluation a été réalisée à l'aide de métriques standards (MSE, MAE, précision, rappel, F1-score) et complétée par des outils visuels tels que les courbes d'erreur de reconstruction et les déviations lissées. Le modèle LSTM-Attention a obtenu un MSE de 0,0042 et un MAE de 0,0468, tandis que le pipeline de détection d'anomalies a atteint une précision de 73%, un rappel de 89% et un F1-score de 80%. Les résultats montrent que le pipeline permet de détecter à la fois des anomalies connues et d'autres non étiquetées auparavant, parfois même avant qu'elles ne surviennent. À terme, ce système pourrait servir d'outil d'étiquetage semi-autonome, contribuant à un diagnostic plus intelligent dans le cadre de l'Industrie 4.0.

Mots-clés : détection d'anomalies, séries temporelles, autoencodeur LSTM, BiLSTM, attention multi-têtes, RFCC, Industrie 4.0, maintenance prédictive.

Abstract

This work presents the design of an intelligent pipeline combining forecasting and anomaly detection techniques for monitoring a complex industrial process (RFCC) in the Algiers refinery. The approach relies on an LSTM model with multi-head attention for anticipating future sensor values, coupled with a BiLSTM Autoencoder to detect abnormal deviations. We also integrate domain-specific heuristics based on critical thresholds validated by process engineers to improve the reliability of the detection system.

The evaluation is conducted using standard metrics (MSE, MAE, precision, recall, F1-score) and enriched with visual tools such as reconstruction error plots and smoothed data deviation curves. The LSTM-Attention model achieved an MSE of 0.0042 and a MAE of 0.0468, while the anomaly detection pipeline reached a precision of 73%, recall of 89%, and F1-score of 80%. Results demonstrate that the pipeline can detect both known anomalies and previously unlabeled ones, sometimes before they occur. Looking ahead, this system could serve as a semi-autonomous labeling tool, contributing to smarter diagnostics in the context of Industry 4.0.

Keywords : anomaly detection, time series, LSTM autoencoder, BiLSTM, multi-head attention, RFCC, Industry 4.0, predictive maintenance.

Table des matières

Remerciements	
Dédicaces	
Résumé	
Abstract	
Introduction générale.....	1
Partie I : État de l’art	3
Chapitre 1 : Organisme d’accueil et problématique.....	4
Section 1.1 : Introduction.....	4
Section 1.2 : La Raffinerie d’Alger	4
Sous-section 1.2.1 : Sonatrach - Activité RPC	4
Section 1.3 : L’unité RFCC (Residue Fluid Catalytic Cracking)	5
Sous-section 1.3.1 : Anomalies dans la régénération du catalyseur.....	5
Section 1.4 : Problématique	7
Section 1.5 : Conclusion	7
Chapitre 2 : Détection d’anomalies	9
Section 2.1 : Introduction.....	9
Section 2.2 : Méthodes statistiques classiques	9
Sous-section 2.2.1 : Le seuillage.....	9
Sous-section 2.2.2 : Analyse en composantes principales (PCA).....	10
Section 2.3 : Approches basées sur l’apprentissage automatique.....	11
Sous-section 2.3.1 : Support Vector Machines (SVM)	11
Sous-section 2.3.2 : One-Class Support Vector Machines.....	11
Sous-section 2.3.3 : Density Based Spatial Clustering of Applications with Noise (DBS-CAN)	12
Sous-section 2.3.4 : Isolation Forest	13
Section 2.4 : Approches basées sur l’apprentissage profond.....	13
Sous-section 2.4.1 : AutoEncodeurs	14
Sous-section 2.4.2 : Long Short-Term Memory (LSTM)	15
Section 2.5 : Conclusion	17

Chapitre 3 : Prédiction de séries temporelles	18
Section 3.1 : Introduction	18
Section 3.2 : Méthodes de prédiction	18
Sous-section 3.2.1 : Forêts Aléatoires (Random Forests)	18
Sous-section 3.2.2 : RNN	19
Sous-section 3.2.3 : LSTM	20
Sous-section 3.2.4 : Gated Recurrent Units (GRU)	21
Sous-section 3.2.5 : AutoRegressive Integrated Moving Average (ARIMA)	21
Sous-section 3.2.6 : Transformateurs	22
Section 3.3 : Travaux existants	24
Sous-section 3.3.1 : Travaux de Chenwei Tang et al.	24
Sous-section 3.3.2 : Travaux de Ge He et al.	25
Sous-section 3.3.3 : Travaux de Wende Tian et al.	25
Section 3.4 : Conclusion	26
Partie II : Conception et réalisation	27
Chapitre 4 : Conception	28
Section 4.1 : Introduction	28
Section 4.2 : Données	28
Sous-section 4.2.1 : Prétraitement des données	29
Section 4.3 : Conception de notre solution	30
Sous-section 4.3.1 : Prédiction	31
Sous-section 4.3.2 : Détection	57
Section 4.4 : Conclusion	60
Chapitre 5 : Réalisation	61
Section 5.1 : Introduction	61
Section 5.2 : Réalisation de notre approche	61
Sous-section 5.2.1 : Outils utilisés	61
Section 5.3 : Environnement matériel	62
Section 5.4 : Implémentation de notre approche	62
Sous-section 5.4.1 : Technologies utilisées	62
Sous-section 5.4.2 : Architecture logicielle	63
Sous-section 5.4.3 : Fonctionnalités de l'application	64
Sous-section 5.4.4 : Déploiement	66
Section 5.5 : Comparaisons des modèles de prédiction	67
Section 5.6 : Validation du modèle de détection	75
Section 5.7 : Pipeline d'évaluation	77
Sous-section 5.7.1 : Chargement des modèles et des données	79
Sous-section 5.7.2 : Prédiction avec le modèle LSTM Attention multi-tête	80
Sous-section 5.7.3 : Détection d'anomalies avec l'Autoencodeur BiLSTM	80
Sous-section 5.7.4 : Analyse neutre par la déviation absolue moyenne	83

Section 5.8 : Limites des approches causales et analyse des causes racines	84
Section 5.9 : Conclusion	85
Conclusion générale	86
Références	88
Annexes.....	93
Chapitre A : Glossaire technique	94
Section A.1 : Raffinage	94
Section A.2 : Hydrocarbures	94
Section A.3 : Distillation atmosphérique	94
Section A.4 : Distillation sous vide	94
Section A.5 : Hydrodésulfuration	95
Section A.6 : Catalyseur	95
Section A.7 : Combustion.....	95
Section A.8 : Ingénieur procédés	95
Section A.9 : Apprentissage automatique (machine learning)	95
Section A.10 : Réseau de neurones	96
Section A.11 : Surapprentissage (Overfitting)	96
Section A.12 : Sous-apprentissage (Underfitting)	96
Section A.13 : Transduction.....	96
Section A.14 : Horodatage	97
Section A.15 : Framework	97
Section A.16 : Frontend	97
Section A.17 : Backend.....	97
Section A.18 : Kafka	98
Chapitre B : Tableaux de données	99
Section B.1 : Affinage des modèles de prédiction	99
Sous-section B.1.1 : Modèle LSTM seq2seq	100
Sous-section B.1.2 : Modèle LSTM Attention multi-tête	100
Sous-section B.1.3 : Modèle Transformers	100
Sous-section B.1.4 : Modèle PatchTST personnalisé	102
Sous-section B.1.5 : Modèle PatchTST.....	103
Sous-section B.1.6 : Modèle Informers	103
Section B.2 : Affinage des modèles de détection	106
Sous-section B.2.1 : Modèle Autoencodeur BiLSTM	106

Table des figures

1.1	Vue schématique de l'unité RFCC	5
1.2	Interface de supervision de l'unité RFCC	6
1.3	Illustration d'une anomalie dans une série temporelle.	7
2.1	Méthode des Seuils de Température Critiques.	9
2.2	Illustration de réduction de dimension utilisant l'analyse multivariée (PCA)	10
2.3	Illustration du principe de fonctionnement d'un SVM dans un espace bidimensionnel.	11
2.4	Illustration du One-Class SVM avec l'approche hypersphère.	12
2.5	Illustration d'un modèle Autoencodeur	15
2.6	Architecture interne d'une cellule LSTM	16
3.1	illustration du fonctionnement global d'un modèle Random Forest.	19
3.2	Structure interne d'un RNN	20
3.3	Structure interne d'un GRU	21
3.4	Architecture du transformateur	23
4.1	Architecture globale de notre système	31
4.2	Illustration de la projection linéaire	32
4.3	Illustration de l'encodage positionnel	33
4.4	Projection linéaire de la matrice Q	34
4.5	Projection linéaire des matrices Q, K, V	35
4.6	Découpage de représentation Q en sous-espaces	36
4.7	Fonctionnement de l'Attention multi-têtes dans un encodeur transformateur	37
4.8	Architecture Transformateur à encoder seul	38
4.9	Architecture de notre Informeur	42
4.10	Vue global du modèle PatchTST	44
4.11	Illustration du concept indépendance des canaux	44
4.12	Illustration du mécanisme de <i>Patching</i> suivi de l'encodage	45
4.13	Architecture générale LSTM seq2seq	48
4.14	Architecture de l'encodeur de notre modèle LSTMseq2seq	49
4.15	Architecture décodeur à une cellule LSTM avec la méthode <i>Teacher Forcing</i>	50
4.16	Architecture décodeur à une cellule LSTM sans la méthode <i>Teacher Forcing</i>	51
4.17	Mécanisme du décodeur de notre modèle LSTM seq2seq	52
5.1	Le pipeline de l'application.	63
5.2	La séquence historique introduite dans l'interface de l'application.	65
5.3	La séquence prédite résultante dans l'interface de l'application.	65
5.4	Les statistiques de la séquence prédite dans l'interface de l'application.	66

5.5	Courbe de perte – LSTM seq2seq	67
5.6	Courbe de perte – LSTM Attention multi-tête	68
5.7	Courbe de perte – Informeurs	68
5.8	Courbe de perte – Transformateurs	69
5.9	Courbe de perte – PatchTST (standard)	69
5.10	Courbe de perte – PatchTST (adapté)	70
5.11	Prévision réelle vs prédite modèle LSTM seq2seq	73
5.12	Prévision réelle vs prédite modèle PatchTST adapté	73
5.13	Prévision réelle vs prédite modèle PatchTST standard	73
5.14	Prévision réelle vs prédite du modèle Transformateur	74
5.15	Prévision réelle vs prédite du modèle Informeur	74
5.16	Prévision réelle vs prédite du modèle LSTM Attention Multi-tête	74
5.17	Courbe d'apprentissage du modèle Autoencodeur BiLSTM pour la détection d'anomalies.	76
5.18	Architecture finale de notre solution	79
5.19	Prévision vs valeurs réelles sur l'attribut 20	80
5.20	Courbe ROC du modèle de détection ($AUC = 0.92$)	81
5.21	Erreur de reconstruction vs anomalies réelles	82
5.22	Matrice de confusion (flagging par séquence)	83
5.23	Évolution de la déviation absolue moyenne lissée (fenêtre = 32)	84
B.1	Pipeline d'affinage des modèles de prédiction	99
B.2	Pipeline d'affinage du modèle de détection Autoencodeur BiLSTM	106

Liste des tableaux

4.1	Étapes du mécanisme de Attention Multi-tête ProbSparse suivies des formes du tenseur	40
4.2	Résumé des étapes du modèle PatchTST et des transformations successives des dimensions des tenseurs	46
5.1	Environnement matériel utilisé	62
5.2	Résultats du tuning des hyperparamètres des modèles de prédiction	71
5.3	Performances du modèle Autoencodeur BiLSTM après ajustement des hyperparamètres	75
B.1	Résultats d’affinage des hyperparamètres du modèle LSTM seq2seq	100
B.2	Résultats d’affinage du modèle LSTM Attention multi-tête	100
B.3	Résultats d’affinage des hyperparamètres pour le modèle Transformers	101
B.4	Résultats d’affinage des hyperparamètres pour le modèle PatchTST personnalisé . .	103
B.5	Résultats d’affinage des hyperparamètres du modèle PatchTST	103
B.6	Résultats d’affinage des hyperparamètres du modèle Informer	106
B.7	Résultats d’affinage des hyperparamètres du modèle Autoencodeur BiLSTM	107

Liste des algorithmes

1	Entrainement_modèle_Transformateur	39
2	Entrainement_modèle_Informer	43
3	Entrainement_modèle_PatchTST_multi-couches	47
4	Entrainement_modèle_LSTM_Seq2Seq	53
5	Entrainement_modèle_PatchTST_personnalisé	55
6	Entrainement_modèle_LSTM_Attention_Multi-tête	57
7	Entrainement_BiLSTM_Autoencoder_pour_détection_d'anomalies	59

Introduction générale

L'unité RFCC

l'unité RFCC (Residue Fluid Catalytic Cracking) faisant partie des nouvelles installations dans la raffinerie d'Alger, constitue une partie essentielle de cette dernière.

Elle est dédiée au craquage catalytique des résidus lourds issus de la distillation afin de les convertir en produits plus légers et commercialisables tels que l'essence et le diesel.

Son fonctionnement repose sur des réactions chimiques sous des conditions spécifiques de température, de pression...etc. L'efficacité de cette unité influe sur la rentabilité de la raffinerie ; cependant, elle est souvent confrontée à des anomalies qui peuvent nuire à son rendement.

Ce document se concentre sur la prédiction et la détection de ces anomalies, dans le but d'améliorer les performances de l'unité RFCC et de réduire les risques de dysfonctionnement.

Objectif

L'objectif de ce travail est de concevoir un système intelligent capable de prédire et détecter les anomalies relatives aux réactions chimiques au sein de l'unité RFCC.

Une telle solution présente un intérêt stratégique pour Sonatrach, en permettant une surveillance proactive et intelligente de ses unités critiques, ce qui contribue à améliorer la rentabilité, la fiabilité opérationnelle et la sécurité des installations industrielles.

En utilisant des techniques avancées d'apprentissage profond, ce système permettra de :

- Prédire les valeurs futures des variables critiques de l'unité RFCC sur un horizon temporel (ex. 1 heure ou plus), en se basant sur les séries temporelles historiques.
- Détecter dans notre prédiction les anomalies en comparant nos résultats avec les comportements attendus, afin d'identifier les écarts inhabituels ou suspects.
- Déployer les résultats du système à travers une application web interactive, facilitant l'interprétation, la visualisation et la prise de décision par les opérateurs et ingénieurs.

Notre contribution se situe à la fois sur le plan théorique et applicatif, en concevant une solution légère et généralisable, adaptée aux environnements industriels disposant de peu de données labellisées et de ressources matérielles limitées. Cette approche permet non seulement d'améliorer la détection précoce des anomalies, mais aussi de prévenir des pannes coûteuses, générant ainsi des économies substantielles pour l'entreprise.

La mise en œuvre de ce système vise à optimiser le fonctionnement de l'unité RFCC tout en réduisant les coûts liés aux arrêts imprévus et à la maintenance.

Organisation du mémoire

Ce mémoire est divisé en deux parties principales. La première partie, dédiée à l'état de l'art, comprend quatre chapitres : d'abord, nous décrirons le contexte industriel et l'unité RFCC ; ensuite, nous aborderons les techniques de détection couramment employées dans le secteur de la détection d'anomalies industrielles. Nous procéderons de la même manière pour les techniques de prédiction. Finalement, nous évoquerons quelques travaux similaires existants. La deuxième partie, axée sur la conception et l'implémentation, comprend deux chapitres : le premier détaille la conception de notre solution. Puis nous passons au dernier chapitre de ce mémoire, qui illustre la mise en œuvre de la solution et présente l'évaluation finale du pipeline.

Partie I : État de l'art

Chapitre 1 : Organisme d'accueil et problématique

1.1 Introduction

Ce chapitre a pour objectif de contextualiser notre projet dans son environnement industriel réel. Nous présentons tout d'abord la raffinerie d'Alger et plus spécifiquement l'unité RFCC (Residue Fluid Catalytic Cracking), cœur du procédé étudié. Nous détaillons ensuite son fonctionnement, les anomalies thermiques critiques qui y surviennent et les limites des méthodes de surveillance actuelles utilisées par les ingénieurs procédés. Enfin, nous exposons la problématique que notre solution se propose de résoudre, à savoir la prédiction et la détection précoce de ces anomalies à partir des séries temporelles.

1.2 La Raffinerie d'Alger

L'Algérie dispose de cinq raffineries principales notamment à Skikda, Alger, Arzew, Adrar et Hassi-Messaoud [1], toutes placées sous la gestion de Sonatrach, la compagnie pétrolière nationale. Parmi elles, la raffinerie d'Alger, située à Sidi Arcine, se distingue par sa portée historique et stratégique. Mise en service en 1964, elle avait été initialement conçue pour traiter environ 3,6 millions de tonnes de brut par an [2]. Après plusieurs décennies d'exploitation, un ambitieux projet de modernisation a été lancé afin de rénover ses installations et accroître sa performance. Si le contrat initial avait été attribué à la société Technip en 2010, c'est finalement l'entreprise chinoise CPECC qui a mené les travaux de réhabilitation à partir de 2016. Moins de trois ans plus tard, la raffinerie était réinaugurée, le 21 février 2019, avec une capacité de raffinage augmentée de 35% et une production conforme aux normes Euro 5, lui permettant de satisfaire durablement les besoins en carburants de la région Centre, tout en réduisant le recours aux importations [2].

Cette transformation s'inscrit dans le cadre d'un vaste programme national de remise à niveau des raffineries du Nord, avec pour objectifs l'augmentation de la capacité de traitement et l'amélioration de la qualité des produits finis, notamment l'essence et le diesel. Les efforts ont également porté sur la sécurité industrielle et le respect des normes environnementales, prolongeant la durée de vie des installations d'au moins quinze ans [1].

Au cœur de cette infrastructure modernisée figure également l'unité RFCC (Residue Fluid Catalytic Cracking), l'un des éléments les plus complexes et essentiels de la raffinerie.

1.2.1 Sonatrach - Activité RPC

Sonatrach est le principal acteur du secteur énergétique en Algérie, couvrant l'exploration, la production, le transport, la transformation et la commercialisation des hydrocarbures. Son activité RPC (Raffinage & Pétrochimie) est spécifiquement chargée de la gestion des activités de raffinage, y compris la raffinerie d'Alger.

1.3 L'unité RFCC (Residue Fluid Catalytic Cracking)

L'unité RFCC est dédiée au craquage catalytique fluide des résidus lourds pour les convertir en produits plus légers comme l'essence et le diesel.

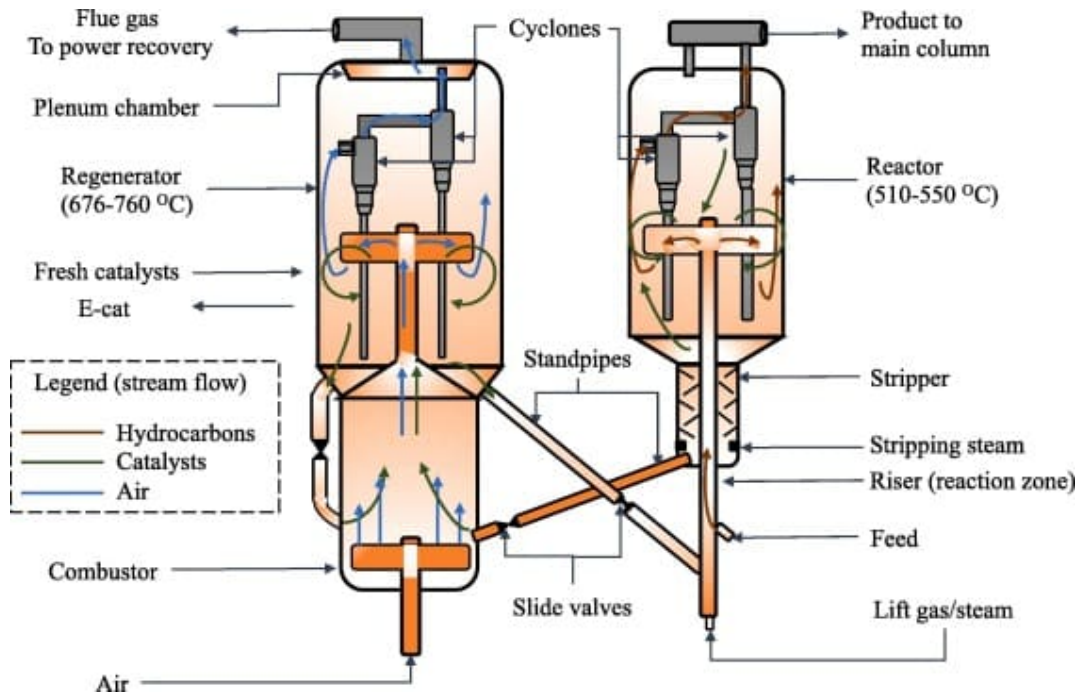


FIGURE 1.1 – Vue schématique de l'unité RFCC

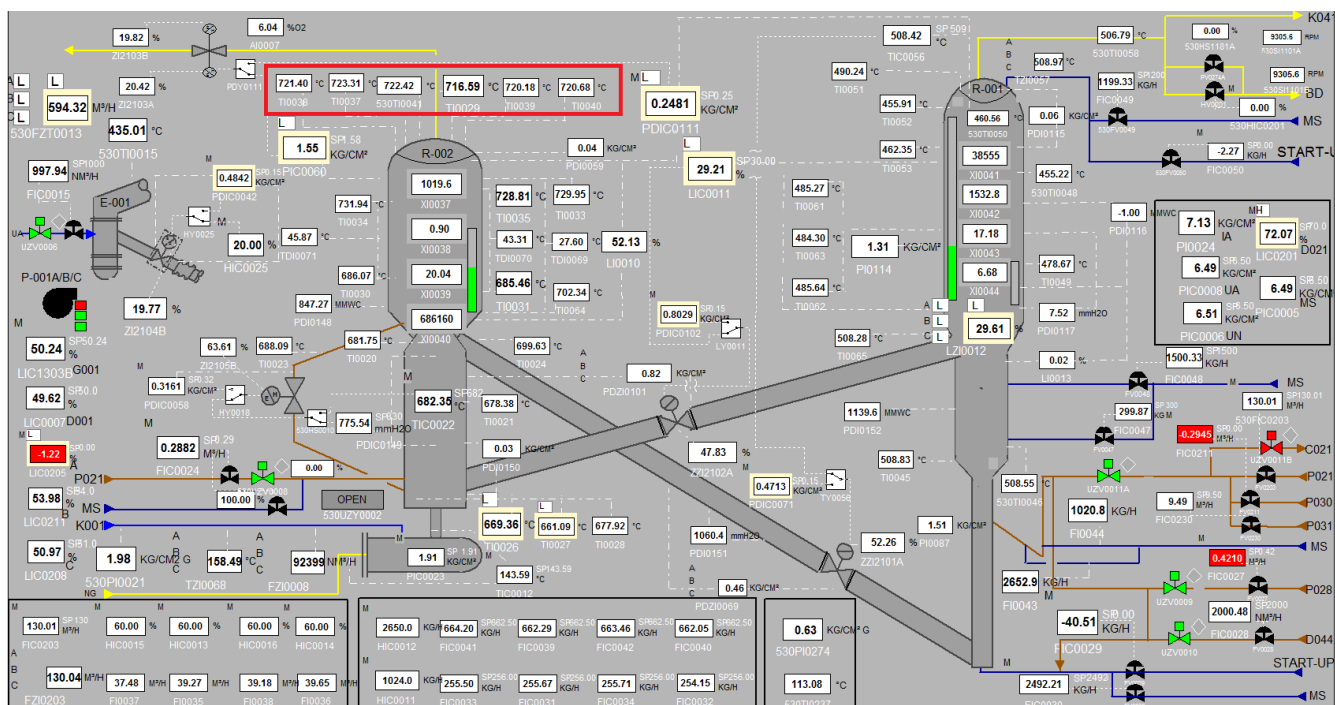
Le processus de craquage catalytique se déroule à haute température (508 °C environ) et pression modérée, en présence d'un catalyseur en poudre fluidisé, circulant en continu entre le réacteur et le régénérateur pour maintenir une activité catalytique optimale, favorisant la rupture des longues chaînes carbonées. Cependant, lors de ce processus, le catalyseur utilisé devient progressivement enrobé d'une matière non volatile carbonée, communément appelée "coke", celui-ci une fois déposé sur le catalyseur, agit pour désactiver son activité de craquage catalytique [4].

La restauration du catalyseur usé, cumulé au fond du réacteur dans une section nommée **stripping**, est nécessaire, il est donc conduit vers le régénérateur et exposé à un flux d'air chaud, variant entre 648 °C et 746 °C [4], permettant la combustion du coke et la récupération de l'activité du catalyseur. Cette régénération doit être optimisée pour éviter une consommation excessive d'énergie ou des dommages au catalyseur, ce qui aurait un impact négatif sur l'économie globale du procédé.

1.3.1 Anomalies dans la régénération du catalyseur

Le régénérateur fonctionne normalement dans des conditions permettant la combustion complète du coke déposé sur le catalyseur usé. Toutefois, la température de combustion peut varier en cas de combustion partielle ou de postcombustion, si les conditions de fonctionnement induisent un niveau de production de chaleur inférieur ou supérieur aux valeurs nominales [4].

Afin de détecter ces anomalies, les ingénieurs utilisent les six capteurs de chaleur : TI0036, TI0037, TI0038, TI0039, TI0040, TI0041, situés à la tête du régénérateur, encadrés dans la Figure 1.2.



[4]

FIGURE 1.2 – Interface de supervision de l'unité RFCC

Les principales anomalies sont :

Afterburning (Postcombustion)

Lorsque la température indiquée par les capteurs dépasse 730 °C, un phénomène appelé **afterburning** peut se produire. Ce problème résulte :

- D'une combustion excessive du coke due à une alimentation trop importante en air.
- D'une mauvaise répartition de l'air dans le régénérateur.

Bien que ce phénomène soit majoritairement observé dans le régénérateur, il peut également se produire dans la partie réacteur, notamment si des particules de catalyseur mal régénérées, encore porteuses de coke ou partiellement brûlées, sont réintroduites dans le riser. Parfois, des résidus d'oxygène parviennent à s'infiltrer dans cette zone, venant tout perturber, en déclenchant des réactions de combustion en plus dans un système pensé pour des réactions qui absorbent de la chaleur (endothermiques), ces intrus déclenchent à l'inverse des combustions parasites. Résultat ? Une hausse de température non contrôlée, ce qui endommage le catalyseur, dérègle le craquage normal des molécules et altère la qualité finale des produits.

Régénération incomplète

Lorsque la température du régénérateur est inférieure à 690 °C, la combustion du coke n'est pas complète. Cela laisse le catalyseur partiellement encrassé, ce qui réduit :

- Son efficacité dans le réacteur.
- Le rendement global du craquage, avec une accumulation progressive de résidus carbonés.

Ce défaut de régénération peut aussi induire des anomalies dans le réacteur : un catalyseur insuffisamment régénéré y limite l'efficacité des réactions de craquage, diminue la sélectivité des

produits et favorise une accumulation accélérée de coke, ce qui accélère la dégradation des performances du système.

Ces deux anomalies (l'afterburning et la régénération incomplète) posent un vrai problème pour l'unité RFCC, tant pour le réacteur que pour le régénérateur. Bien qu'elles se manifestent différemment, elles ont un point commun : elles dérèglent la température du système et perturbent son efficacité. Plutôt que de les traiter séparément, nous avons choisi de les regrouper en une seule catégorie. Cette simplification nous permettra de mieux prédire les risques de surchauffe et de détecter les signaux anormaux.

Du point de vue de l'IA, cette anomalie thermique unifiée apparaît comme des valeurs aberrantes (outliers) dans les données des capteurs. Imaginez un ECG industriel : quand la température ne convient pas aux plages normales, c'est le signe que quelque chose ne tourne pas rond. Ces pics inattendus nous alertent sur un déséquilibre thermique qui nécessite une intervention rapide.

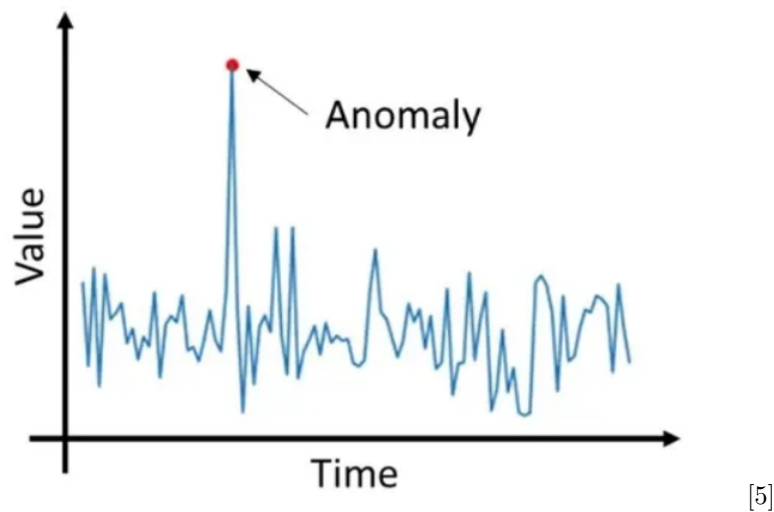


FIGURE 1.3 – Illustration d'une anomalie dans une série temporelle.

1.4 Problématique

Dans le cadre de la surveillance de l'unité RFCC (Résidu Fluid Catalytic Cracking), les ingénieurs procédés de la Sonatrach utilisent actuellement les indicateurs énumérés précédemment pour détecter d'éventuels comportements anormaux ou des signes de dégradation. Si ces derniers ont l'avantage de pouvoir signaler des situations critiques, ils ne retranscrivent qu'une partie et le plus souvent avec retard l'état global de l'unité. Les défaillances naissantes, qui s'aggravent dans le temps avant de motiver ces signaux d'alarme, échappent ainsi à l'analyse, restreignant les possibilités à l'anticipation et générant des risques plus élevés de défaillances graves, voire d'arrêts de production. Ce recours à une unique variable représente donc un véritable obstacle en termes de détection précoce, de diagnostic et de maintenance prédictive.

1.5 Conclusion

Ce chapitre nous a permis de nous placer dans le contexte de l'industrie dans lequel se situe notre projet en présentant l'unité RFCC de la raffinerie d'Alger, son rôle dans le process de craquage catalytique et les différents types d'anomalies thermiques pouvant y apparaître et ainsi

engendrer des désordres. Nous y avons relevé les limites stratégiques des méthodologies de diagnostics existantes et avons introduit la problématique à laquelle notre proposition se positionne.

Le chapitre suivant en constitue la transition, il nous permettra d'aborder un état de l'art des méthodes de détection d'anomalies et des méthodes de prédiction de séries temporelles.

Chapitre 2 : Détection d'anomalies

2.1 Introduction

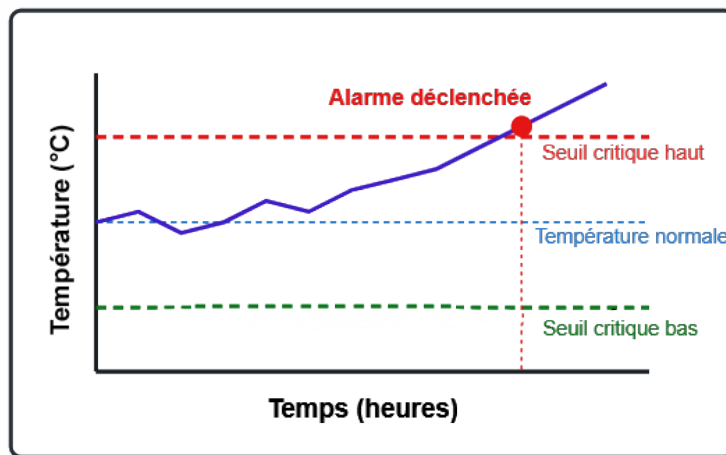
Dans ce chapitre, nous proposons ensemble de techniques les plus répandues pour la détection d'anomalies au sein de systèmes industriels. Nous entamons notre exposé par les méthodes statistiques classiques, encore très utilisées pour leur facilité d'implémentation, avant de décrire les techniques issues de l'apprentissage automatique et de ses deux grandes catégories, à savoir les approches supervisées et non supervisées. Enfin, nous abordons les méthodes populaires de ces dernières années, fondées sur l'apprentissage profond et capables d'appréhender des relations complexes au sein de séries observées dans le temps.

2.2 Méthodes statistiques classiques

Dans notre domaine, les premières approches reposaient sur des méthodes simples à implémenter ; cependant, elles manquent de flexibilité face à des comportements complexes et non linéaires, comme ceux observés dans les procédés de craquage. Parmi ces méthodes, nous citons les suivantes :

2.2.1 Le seuillage

Cette approche consiste à définir les seuils critiques. Actuellement, la raffinerie utilise un système similaire, appelé «consigne», pour surveiller les variations de température en tête du régénérateur.



[5]

FIGURE 2.1 – Méthode des Seuils de Température Critiques.

Dans la Figure 2.1, nous constatons une détection tardive d'un problème progressif. Aussi, avoir les seuils des valeurs normales de tous les capteurs (attributs) est parfois très difficile, surtout quand la dimension du data-set est importante et que les valeurs réelles sont différentes des valeurs design (de conception).

2.2.2 Analyse en composantes principales (PCA)

Le PCA est une technique de réduction de la dimensionnalité, très répandue dans l'industrie pour la détection d'anomalies. Elle consiste à projeter les données dans un nouvel espace dont les axes appelés «composantes principales» ont la propriété de maximiser la variance des projections. En ce sens, nous pouvons retrouver l'information contenue dans nos données à travers un nombre restreint de directions pertinentes [6].

La détection d'anomalies avec cette technique se base sur l'identification des observations qui ne suivent plus la structure de corrélation observée sur les données normales. Notamment, nous pouvons nous intéresser aux projections des instances sur les composantes de faibles variances : alors que les données normales auront tendance à avoir de faibles projections sur ces composantes, les anomalies qui viennent perturber la structure du système se projettent au contraire fortement dessus [6].

Une des façons de procéder est d'utiliser le PCA robuste pour mieux résister à la présence de bruit ou de valeurs aberrantes. Ceci consiste en la recherche des composantes principales à partir de données supposées normales puis à calculer la distance d'une nouvelle observation à ce sous-espace. Si cette distance est statistiquement significative (au-delà d'un seuil calculé via une loi χ^2), l'observation est considérée comme anormale [6].

Des approches similaires ont été appliquées avec succès dans divers domaines comme la surveillance de composants embarqués (dans l'aérospatial), la cybersécurité (détection d'intrusions), ou encore l'analyse de graphes évolutifs dans des systèmes complexes [6].

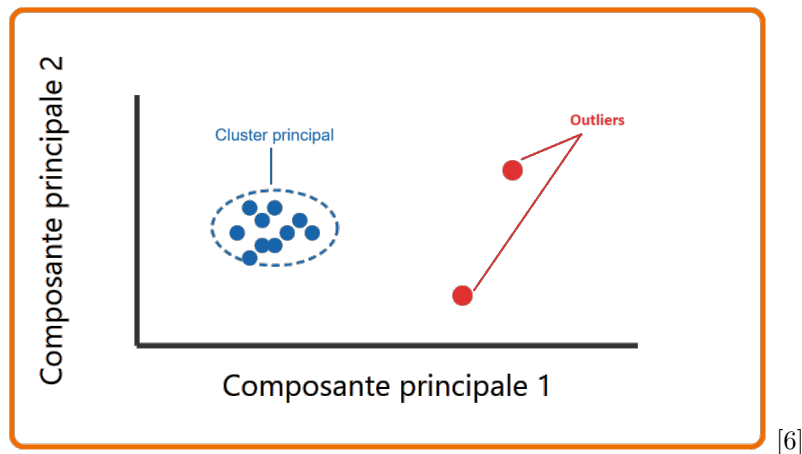


FIGURE 2.2 – Illustration de réduction de dimension utilisant l'analyse multivariée (PCA)

Hormis le fait que ce type d'analyse présente plusieurs avantages, tels que son applicabilité à des données non étiquetées, sa capacité à modéliser des corrélations multivariées, ou encore son efficacité en tant qu'étape de prétraitement ou en combinaison avec d'autres modèles, il présente néanmoins certaines limites, notamment une forte sensibilité au bruit, une moindre performance dans la détection d'anomalies non linéaires, ainsi que sa dépendance à l'hypothèse selon laquelle la majorité des données sont normales.

2.3 Approches basées sur l'apprentissage automatique

Avec l'essor du machine learning, des techniques plus sophistiquées ont été développées :

2.3.1 Support Vector Machines (SVM)

Le Support Vector Machine (SVM) est une méthode d'apprentissage supervisé principalement utilisée pour résoudre des problèmes de classification binaire. Il consiste à trouver un hyperplan optimal qui sépare les différentes classes dans un espace des données tout en maximisant la marge entre les classes [7].

Lorsque les données sont linéairement séparables, un SVM à marge dure est utilisé. Pour des données non séparables ou bruitées, un SVM à marge souple laisse une certaine tolérance aux erreurs [7].

Dans un cas où la séparation n'est pas linéaire, le SVM peut faire appel aux méthodes noyau (Kernel Methods), qui projettent les données dans un espace de caractéristiques de dimension plus élevée afin de rendre la séparation possible [7].

Ainsi, le SVM classique peut être vu comme un prérequis avant de se lancer dans des cas plus complexes de détection d'anomalies, tels que le One-Class SVM.

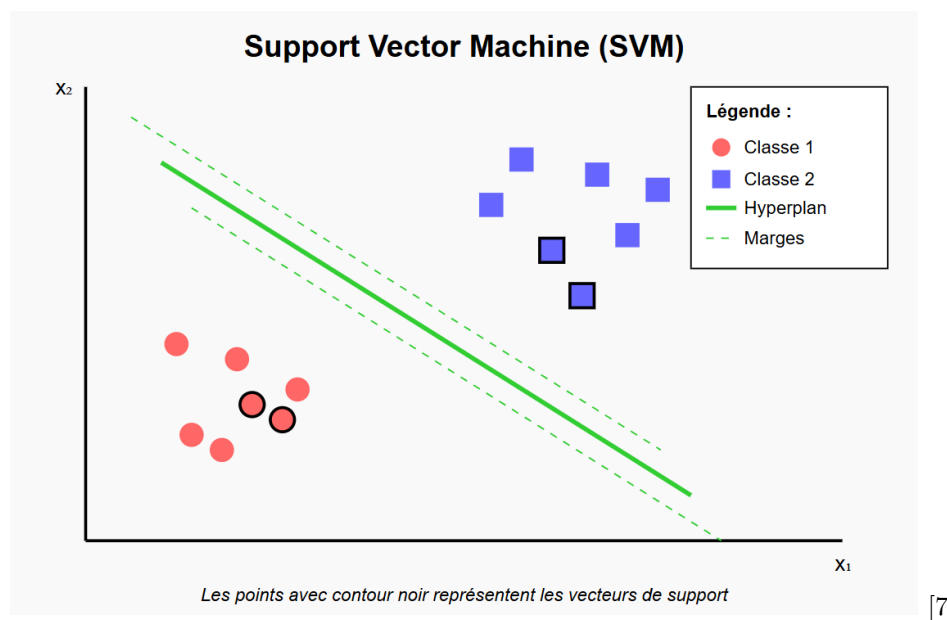


FIGURE 2.3 – Illustration du principe de fonctionnement d'un SVM dans un espace bidimensionnel.

2.3.2 One-Class Support Vector Machines

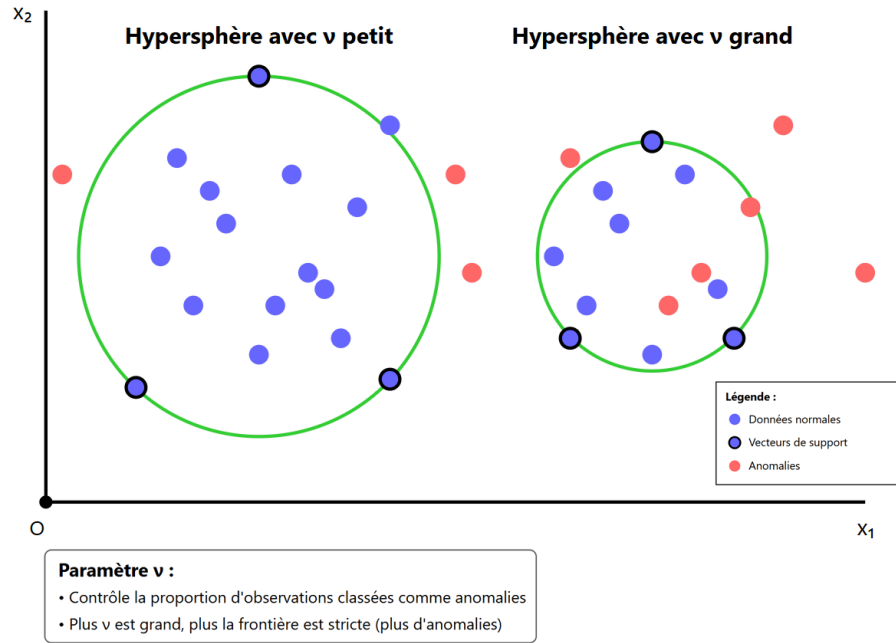
Le One-Class SVM est une extension non supervisée du SVM standard, spécifiquement conçue pour la détection d'anomalies ou l'apprentissage sur des ensembles contenant uniquement des données normales [7].

Contrairement au SVM binaire qui sépare deux classes, le One-Class SVM cherche à séparer la majorité des données normales du reste de l'espace, en construisant un hyperplan qui maximise la marge par rapport à l'origine.

Deux grandes approches existent pour définir la frontière :

- **Hypersphère** : entoure la majorité des données normales (approche Tax et Duin).
- **Hyperplan par rapport à l'origine** : sépare les données normales de l'origine (approche Schölkopf et Müller) [7].

Le paramètre ν est important car il détermine la fraction des observations considérées comme des anomalies. Par conséquent, il fournit un levier pour s'adapter à de différentes situations industrielles, notamment lorsque les anomalies sont rares ou difficiles à étiqueter [7].



[7]

FIGURE 2.4 – Illustration du One-Class SVM avec l'approche hypersphère.

Le One-Class SVM présente plusieurs avantages ; en particulier, il est efficace si l'on cherche à détecter des anomalies sans que des exemples anormaux fassent partie des données d'entraînement. Aussi, grâce à l'utilisation de noyaux (kernel), il est capable de détecter des seuils de décision plutôt complexes. Cependant, cette technique possède des inconvénients. Par exemple, la qualité de la détection est sensible aux choix des hyper-paramètres, comme le ν et γ . En outre, le coût algorithmique du One-Class SVM est élevé ($O(n^2)$ à $O(n^3)$), ce qui rend son application compliquée lorsque l'on dispose de beaucoup de données et de variables. D'autre part et contrairement à des techniques plus interprétables comme les arbres de décision, le One-Class SVM ne fournit pas d'explication sur le motif pour lequel une observation est considérée comme anormale.

Enfin, son premier avantage peut devenir un désavantage dans le cas où les données d'entraînement ne sont pas propres (sans anomalies).

Malgré ses défauts, le One-Class SVM demeure quand même une méthode essentielle pour l'identification de comportements anormaux dans des processus industriels compliqués [7].

2.3.3 Density Based Spatial Clustering of Applications with Noise (DBSCAN)

DBSCAN est une méthode de clustering basée sur la densité qui est particulièrement utile pour la détection d'anomalies où ces dernières forment de petits clusters. Contrairement aux méthodes classiques de clustering, DBSCAN ne nécessite pas de spécifier le nombre de clusters à l'avance, ce

qui en fait une approche flexible. De plus, cette méthode est très sensible aux anomalies dans le sens où elle les détecte facilement comme des points isolés. [8].

Cependant, DBSCAN présente également certains inconvénients. Sa complexité algorithmique peut la rendre inefficace pour des séries temporelles de grande taille surtout quand le nombre d'attributs est important. De plus, les hyperparamètres de DBSCAN, tels que la distance maximale entre deux points pour qu'ils soient considérés comme voisins (ϵ) et le nombre minimum de points pour former un cluster (MinPts), peuvent être difficiles à ajuster de manière optimale, nécessitant souvent un choix expérimental ou un ajustement délicat [8].

2.3.4 Isolation Forest

L'Isolation Forest est une approche d'apprentissage non supervisé qui vise à détecter toutes sortes d'anomalies. Comparé aux autres méthodes qui tentent de créer un modèle normal sur les données, Isolation Forêt a une approche opposée : elle cherche directement à contourner la majorité des données considérées comme normales pour atteindre celles hors normes [9].

L'idée complémentaire consiste au développement et à la mise en place d'un ensemble d'arbres de décisions aléatoires qu'on appelle les arbres d'isolement. Chaque arbre partitionne les données dans un processus appelé découpage qui se divise en plusieurs étapes avec des attributs choisis au hasard et des seuils coupants également tirés au sort. Les anomalies sont moyennement communes étant donné qu'elles sont rares mais obéissent logiquement aux schémas du groupe, elles nécessitent pour être reconnues moins de divisions pour être séparées. Une observation est anormale si l'observation parvient rapidement à un certain seuil minimal considéré comme anomalie et peut donc, sans problème, rester séparée en peu d'étapes [9].

Isolation Forest est particulièrement adaptée aux données de haute dimension et aux ensembles de grande taille, car sa complexité algorithmique est linéaire par rapport au nombre d'échantillons, ce qui en fait une méthode très efficace pour des applications industrielles nécessitant une détection rapide [9].

Il s'agit d'une méthode entièrement non supervisée qui n'a pas besoin d'exemples d'anomalies pour l'entraînement. Elle fonctionne bien sur des ensembles de données très larges et de haute dimension. De plus, l'algorithme a un temps d'exécution particulièrement rapide de $O(n \log n)$, ce qui le rend adapté aux contextes industriels nécessitant une détection efficace à grande échelle.

D'autre part, cette technique présente d'autres inconvénients. Elle peut avoir du mal à isoler les anomalies qui sont trop proches des points normaux, ce qui impacte la qualité de détection globale. De plus, la performance diminue considérablement lorsqu'il y a trop d'anomalies fréquentes dans les données. Enfin, les forêts d'isolement ont tendance à être sensibles à certains hyperparamètres, comme le nombre d'arbres créés et la profondeur maximale autorisée, créant ainsi un besoin d'un réglage minutieux afin d'atteindre des résultats optimaux.

2.4 Approches basées sur l'apprentissage profond

Avec l'évolution rapide du deep learning, de nouvelles méthodes puissantes ont émergé pour traiter des problèmes complexes de détection d'anomalies, en exploitant la capacité des réseaux neuronaux profonds à extraire automatiquement des caractéristiques représentatives des données.

2.4.1 AutoEncodeurs

Un Autoencodeur (AE) est un type particulier de réseau de neurones feedforward composé de neurones artificiels interconnectés. Dans un réseau de neurones feedforward, les connexions entre les neurones ne forment aucun cycle ; le flux d'information est unidirectionnel, partant d'une couche d'entrée, traversant une ou plusieurs couches cachées, jusqu'à une couche de sortie.

Dans chaque couche cachée, un neurone reçoit plusieurs entrées, calcule la somme pondérée de ces entrées additionnée d'un biais, puis applique une fonction d'activation au résultat pour produire une sortie. Cette sortie sert ensuite d'entrée pour la couche suivante. La sortie finale y du réseau peut être exprimée comme une fonction de l'entrée x , en fonction des poids, des biais et de la ou des fonctions d'activation successives.

La structure d'un Autoencodeur est symétrique par rapport à son point central, appelé le goulot d'étranglement (bottleneck). Contrairement aux réseaux standards, la couche de sortie d'un AE possède exactement le même nombre de neurones que la couche d'entrée. L'Autoencodeur est entraîné de manière non supervisée : chaque vecteur d'entrée est utilisé à la fois comme entrée et comme étiquette (sortie désirée). L'objectif est de minimiser l'erreur de reconstruction, c'est-à-dire la différence entre l'entrée initiale et sa reconstruction à la sortie.

Le bottleneck représente une codification comprimée (ou représentation latente) des données d'entrée. Cette codification peut être exploitée comme une extraction de caractéristiques ou une méthode de réduction de dimension.

Formellement, un autoencodeur se compose de deux parties :

- L'encodeur α , qui projette l'entrée $x \in X$ vers une représentation codée $y \in Y$.
- Le décodeur β , qui reconstruit la donnée d'entrée à partir du code y en générant $x' \in X'$.

Dans un modèle simple à une seule couche cachée, ces opérations sont définies par :

$$y = \theta(Wx + b)$$

$$x' = \theta'(W'y + b')$$

où W et W' sont des matrices de poids, b et b' des vecteurs de biais et θ , θ' des fonctions d'activation.

L'entraînement de l'AE vise à minimiser une fonction de perte de reconstruction :

$$\text{Loss}(x, x') = \|x - x'\|^2$$

Dans des architectures plus complexes comportant plusieurs couches cachées, ces formules sont généralisées en introduisant des activations intermédiaires supplémentaires, que nous omettons ici pour des raisons de simplification.

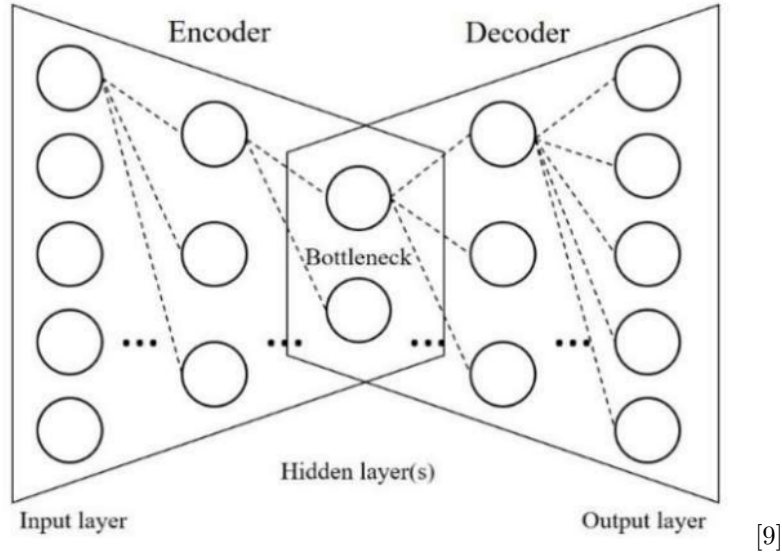


FIGURE 2.5 – Illustration d'un modèle Autoencoder .

2.4.2 Long Short-Term Memory (LSTM)

Les réseaux de neurones LSTM (Long Short-Term Memory) sont une extension des réseaux neuronaux récurrents (RNN), spécifiquement conçus pour gérer les dépendances à long terme dans les données séquentielles. Contrairement aux RNNs classiques, les LSTMs intègrent des mécanismes de contrôle appelés "portes" (entrée, oubli, sortie), qui permettent de réguler le flux d'information au sein du réseau, évitant ainsi les problèmes d'explosion ou de disparition du gradient lors de l'apprentissage [10].

Leur fonctionnement est comme suit [10] :

- **Équation de la porte d'entrée (z) :**

$$\mathbf{z}_{i,j} = \mathbf{g}(\mathbf{W}^{(z)}\mathbf{x}_{i,j} + \mathbf{R}^{(z)}\mathbf{h}_{i,j-1} + \mathbf{b}^{(z)})$$

Cette formule calcule la porte d'entrée qui détermine quelles informations seront ajoutées à l'état de la cellule. La fonction $\mathbf{g}(\cdot)$ est la tangente hyperbolique ($\tanh$).

- **Équation de la porte d'oubli (s) :**

$$\mathbf{s}_{i,j} = \sigma(\mathbf{W}^{(s)}\mathbf{x}_{i,j} + \mathbf{R}^{(s)}\mathbf{h}_{i,j-1} + \mathbf{b}^{(s)})$$

Cette formule calcule la porte d'oubli qui détermine quelles informations seront conservées de l'état cellulaire précédent. La fonction $\sigma(\cdot)$ est la fonction sigmoïde.

- **Équation de la porte de contrôle (f) :**

$$\mathbf{f}_{i,j} = \sigma(\mathbf{W}^{(f)}\mathbf{x}_{i,j} + \mathbf{R}^{(f)}\mathbf{h}_{i,j-1} + \mathbf{b}^{(f)})$$

Cette formule calcule la porte de contrôle qui régule la quantité d'information à conserver dans l'état de la cellule. Elle produit une valeur entre 0 et 1.

- Équation de mise à jour de l'état de la cellule (c) :

$$\mathbf{c}_{i,j} = \mathbf{s}_{i,j} \odot \mathbf{z}_{i,j} + \mathbf{f}_{i,j} \odot \mathbf{c}_{i,j-1}$$

Cette formule met à jour l'état de la cellule en combinant les nouvelles informations avec les informations précédentes. L'opérateur $\odot$ représente la multiplication élément par élément.

- Équation de la porte de sortie (o) :

$$\mathbf{o}_{i,j} = \sigma(\mathbf{W}^{(o)}\mathbf{x}_{i,j} + \mathbf{R}^{(o)}\mathbf{h}_{i,j-1} + \mathbf{b}^{(o)})$$

Cette formule calcule la porte de sortie qui détermine quelles parties de l'état de la cellule seront transmises à la sortie.

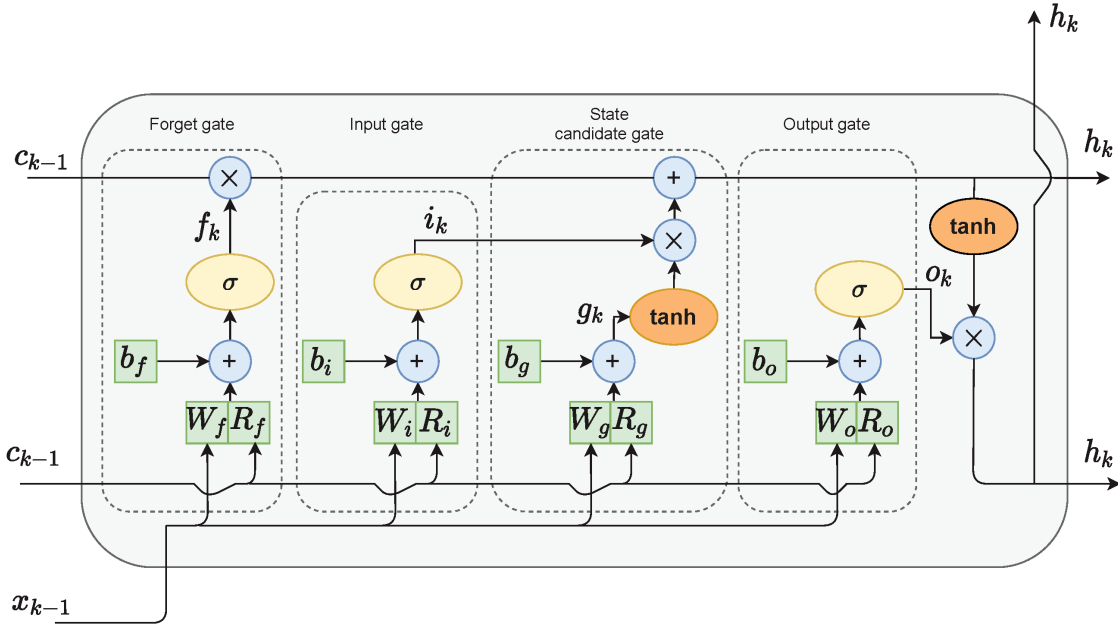
- Équation de la sortie cachée (h) :

$$\mathbf{h}_{i,j} = \mathbf{o}_{i,j} \odot \mathbf{g}(\mathbf{c}_{i,j})$$

Cette formule calcule la sortie cachée qui est transmise à la prochaine unité LSTM.

où $c_{i,j} \in \mathbb{R}^m$ est le vecteur d'état, $x_{i,j} \in \mathbb{R}^p$ est le vecteur d'entrée et $h_{i,j} \in \mathbb{R}^m$ est le vecteur de sortie pour la j -ème unité LSTM. Les variables $s_{i,j}$, $f_{i,j}$ et $o_{i,j}$ représentent respectivement les portes d'entrée, d'oubli et de sortie. La fonction $g(\cdot)$ est définie comme la tangente hyperbolique ($\tanh$) et s'applique aux vecteurs d'entrée point par point. De même, $\sigma(\cdot)$ est définie comme la fonction sigmoïde. L'opérateur $\odot$ représente la multiplication élément par élément de deux vecteurs de même taille.

Les paramètres $W^{(\cdot)}$, $R^{(\cdot)}$ et $b^{(\cdot)}$ sont les paramètres de l'architecture LSTM, dont la taille est déterminée selon la dimensionnalité des vecteurs d'entrée et de sortie. L'état cellulaire $c_{i,j-1}$ représente l'état de la cellule du réseau provenant du bloc LSTM précédent, fournissant ainsi un flux d'information entre les blocs LSTM consécutifs.



[11]

FIGURE 2.6 – Architecture interne d'une cellule LSTM

Les LSTMs sont particulièrement bien adaptés à la détection d'anomalies dans les séries temporelles, grâce à leur capacité à modéliser la dynamique temporelle des données. Dans le cadre de l'apprentissage non supervisé, une approche courante consiste à entraîner le LSTM sur des séquences considérées comme normales, puis à détecter les anomalies en mesurant l'erreur de reconstruction. Une erreur élevée indique qu'un comportement inattendu s'est produit, suggérant une anomalie [10].

Une méthode innovante a été proposée : au lieu de simplement utiliser l'erreur de reconstruction, les auteurs combinent les sorties du LSTM avec des méthodes de classification comme le One-Class SVM ou le SVDD et optimisent conjointement les paramètres des deux modèles. Cela permet d'améliorer significativement la performance de détection, même avec des séquences de longueur variable [10].

En conclusion, les LSTMs présentent des avantages significatifs pour l'analyse des données séquentielles. Cela est dû à leur capacité de prise en charge des dépendances temporelles complexes, de gestion des séquences de longueur variable grâce aux méthodes de pooling, leur capacité d'adaptation à divers cadres d'apprentissage tels que le non supervisé, semi-supervisé et supervisé et leur faculté de détection précise comparée à leur concurrent classique OC-SVM ou le SVDD lorsqu'ils sont utilisés seuls [10]. Cependant, ces bénéfices s'accompagnent d'inconvénients notables : une complexité d'entraînement élevée due à l'optimisation conjointe des LSTMs et des classifieurs, nécessitant des ressources computationnelles importantes et pouvant présenter des instabilités ; un besoin de réglages fins des hyperparamètres critiques tels que la taille des cellules, le taux d'apprentissage et les paramètres de régularisation ; une moindre interprétabilité comparée aux approches basées sur des règles ou des arbres de décision ; et enfin, un risque d'overfitting significatif si les mécanismes de régularisation et les contraintes appropriées ne sont pas correctement implémentés et calibrés [10].

2.5 Conclusion

Dans ce chapitre, nous avons abordé diverses méthodes de détection des anomalies sur les systèmes industriels, à partir des approches statistiques classiques jusqu'aux modèles récents basés sur l'apprentissage automatique et l'apprentissage profond. Ces techniques jouent un rôle crucial pour identifier les mouvements anormaux sur un processus industriel critique, tel que celui de l'unité RFCC.

Cependant, l'efficacité de la détection s'accompagne de la qualité des prédictions fournies. Par ce fait, le chapitre suivant traite les méthodes pour la prédiction de séries temporelles, où nous allons exposer les modèles sous-jacents pour avoir des prédictions d'anticipation des variables industrielles et ensuite une détection robuste.

Chapitre 3 : Prédiction de séries temporelles

3.1 Introduction

La prédiction constitue un enjeu fondamental dans les systèmes de surveillance industrielle, notamment pour l'anticipation de comportements inhabituels. Ce chapitre présente différentes approches utilisées dans le domaine de la prédiction appliquée aux séries temporelles. Ces méthodes, issues de diverses familles de modèles, sont largement explorées dans la littérature pour leur potentiel à modéliser l'évolution des variables dans des contextes complexes comme les procédés industriels.

3.2 Méthodes de prédiction

Parmi les méthodes prédictives les plus couramment utilisées dans les systèmes industriels, certaines reposent sur des algorithmes d'apprentissage automatique bien établis. On peut citer plusieurs approches reconnues pour leur efficacité, telles que les forêts aléatoires, les Autoencodeurs ou encore les réseaux neuronaux récurrents.

3.2.1 Forêts Aléatoires (Random Forests)

Le Random Forest (RF) est une technique d'apprentissage par ensemble utilisée pour des tâches de classification, de régression et d'autres applications d'analyse prédictive. Ce modèle repose sur l'utilisation d'un grand nombre d'arbres de décision et sur le mécanisme de bagging (bootstrap aggregating) afin de limiter le risque de surapprentissage (overfitting) généralement associé aux arbres de décision complexes [12].

Un arbre de décision est une structure arborescente dans laquelle chaque nœud interne représente une règle de décision basée sur une caractéristique des données, tandis que chaque feuille correspond à une prédiction (soit une classe pour la classification, soit une valeur pour la régression). Pour choisir la meilleure division à chaque nœud, des critères tels que l'impureté de Gini, l'entropie ou l'écart-type peuvent être utilisés afin de maximiser le gain d'information. Le processus de division continue jusqu'à ce qu'une condition d'arrêt soit atteinte, telle que :

- Toutes les données d'un nœud appartiennent à la même classe.
- Le nombre d'observations d'un nœud est inférieur à un seuil minimal fixé.
- La profondeur de l'arbre atteint une limite maximale prédéfinie.

Dans une forêt aléatoire, le mécanisme de bagging intervient de la manière suivante : À partir d'un ensemble de d observations, on génère n sous-échantillons (avec remise), chacun comportant d observations tirées aléatoirement. Un modèle (arbre) est entraîné sur chaque sous-échantillon, produisant ainsi n prédictions indépendantes. L'agrégation finale des prédictions s'effectue soit par vote majoritaire (pour la classification), soit par moyenne (pour la régression) [12].

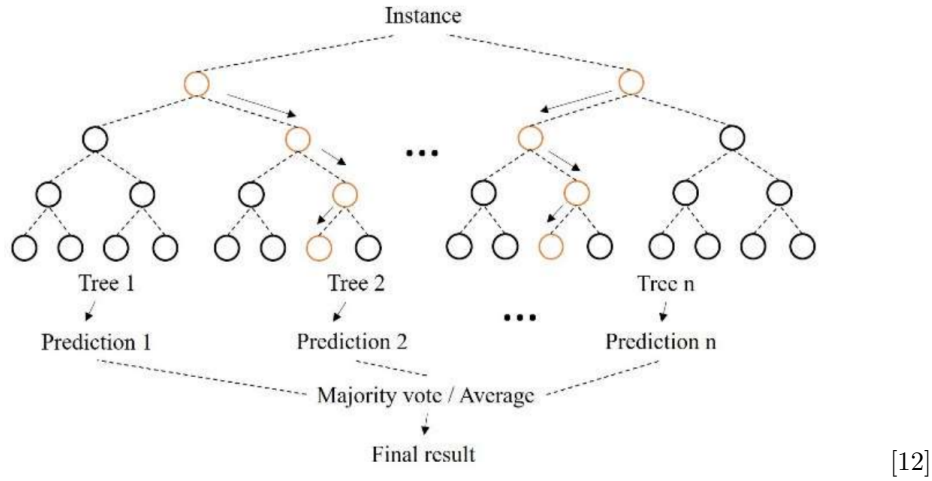


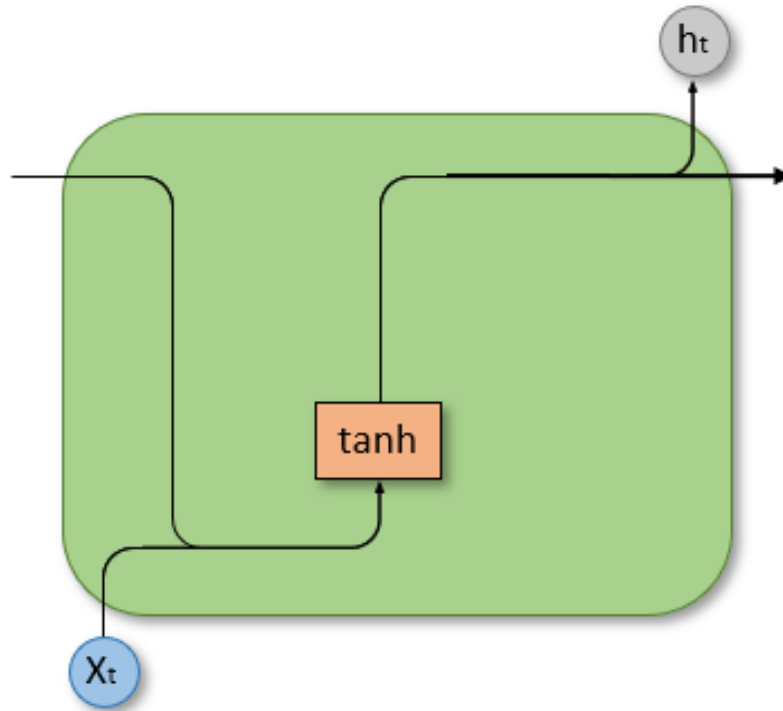
FIGURE 3.1 – illustration du fonctionnement global d’un modèle Random Forest. [12]

Dans le cadre de la prédiction de variables critiques dans l’unité RFCC, les forêts aléatoires présentent plusieurs avantages. Elles sont capables de modéliser des relations complexes entre les variables d’entrée, même en présence de bruit ou de corrélations non linéaires, ce qui en fait un outil robuste pour prédire, par exemple, l’évolution de la température dans le régénérateur. De plus, elles nécessitent peu de réglages, s’adaptent bien à des jeux de données hétérogènes et permettent d’identifier les variables les plus influentes dans la prise de décision. Toutefois, les forêts aléatoires ne tiennent pas compte de la dimension temporelle des données, ce qui limite leur capacité à capturer des dynamiques séquentielles ou des tendances progressives. Dans le contexte de séries temporelles industrielles, il est donc souvent nécessaire de leur adjoindre un prétraitement par fenêtrage ou extraction de caractéristiques temporelles pour préserver les dépendances temporelles entre les observations.

3.2.2 RNN

Les réseaux de neurones récurrents (RNN) sont couramment utilisés pour la modélisation et la prédiction de données temporelles. Grâce à leur capacité à exploiter la structure séquentielle des données, ils permettent de prédire les valeurs futures d’une série à partir de ses états passés [13].

Cette capacité en fait une approche pertinente pour la prédiction dans des contextes industriels, où les anomalies peuvent se manifester par des déviations progressives par rapport au comportement attendu. En comparant les valeurs prédites aux observations réelles, il devient alors possible de détecter des écarts significatifs, indicateurs potentiels d’un dysfonctionnement [13].



[14]

FIGURE 3.2 – Structure interne d'un RNN

Afin de surmonter certaines limitations des RNN classiques (notamment leur difficulté à modéliser les dépendances à long terme en raison du phénomène de disparition du gradient, ainsi que leur performance souvent dégradée sur des séquences longues ou complexes), des variantes améliorées ont été développées, telles que les LSTM (Long Short-Term Memory) et GRU (Gated Recurrent Units), qui seront abordées plus loin [13].

3.2.3 LSTM

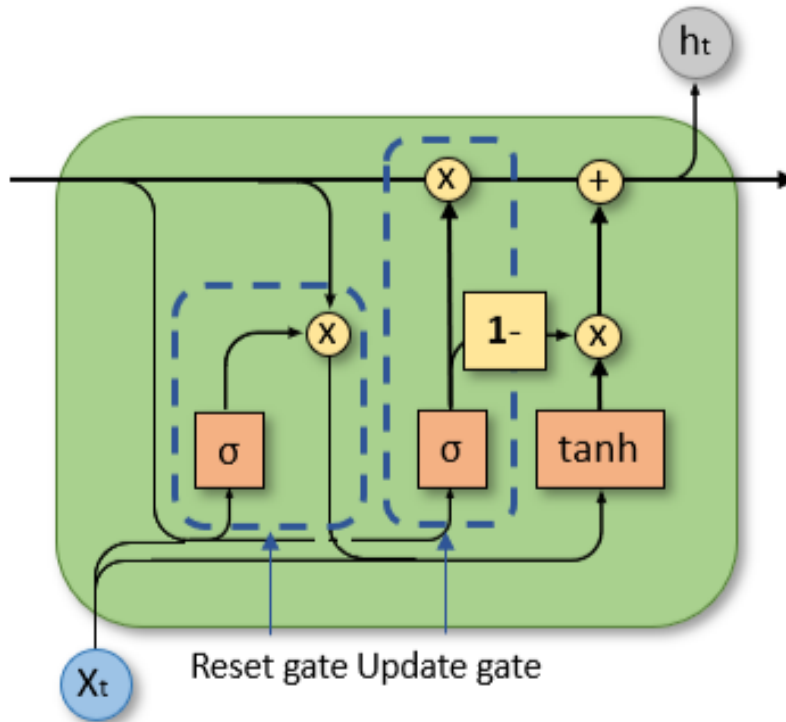
Les réseaux LSTM peuvent également être utilisés dans un cadre purement prédictif, en modélisant directement l'évolution temporelle d'une variable. Dans ce cas, l'objectif n'est pas de détecter une anomalie via une reconstruction, mais bien de prévoir les valeurs futures à partir d'un historique de mesures. Par exemple, un LSTM entraîné sur des séries de températures peut prédire la valeur attendue à l'instant suivant, ou sur plusieurs horizons temporels [13].

Cette approche est particulièrement pertinente dans des contextes industriels comme celui de l'unité RFCC, où il est essentiel d'anticiper des dérives thermiques ou des changements de pression. Une fois entraîné, le modèle fournit des prédictions qui peuvent ensuite être confrontées aux mesures réelles pour évaluer des écarts anormaux, ou utilisées directement pour la régulation du processus.

L'usage du LSTM dans la prédiction permet donc de modéliser la dynamique normale du système et d'anticiper son comportement futur, ce qui en fait une base solide pour des systèmes de surveillance ou de maintenance prédictive.

3.2.4 Gated Recurrent Units (GRU)

Les Gated Recurrent Units (GRU) sont une variante simplifiée des réseaux de neurones récurrents (RNN), spécifiquement conçue pour modéliser des dépendances temporelles tout en réduisant la complexité structurelle des modèles comme les LSTMs. Dans le cadre de la prédiction d'anomalies sur des séries temporelles, les GRUs sont utilisés comme modèles autorégressifs qui apprennent les dynamiques normales des séquences pour ensuite anticiper les valeurs futures. Une anomalie est ainsi détectée lorsque l'écart entre la valeur prédite et la valeur réelle dépasse un certain seuil [13].



[14]

FIGURE 3.3 – Structure interne d'un GRU

Grâce à leur architecture plus légère (moins de portes que les LSTMs), les GRUs permettent des entraînements plus rapides et sont donc adaptés aux applications en temps réel ou à grande échelle. De plus, leur capacité à effectuer des mises à jour incrémentales les rend particulièrement utiles pour les environnements de données en flux continu [13].

Toutefois, les GRUs présentent aussi certaines limitations : leur performance peut dépendre fortement du choix de la fenêtre temporelle, ils nécessitent toujours un calibrage précis des seuils de détection et leur interprétabilité reste limitée par rapport à des modèles plus explicites. En résumé, les GRU offrent un bon compromis entre efficacité et performance pour la prédiction d'anomalies dans les séries temporelles industrielles, notamment lorsque les contraintes computationnelles sont importantes [13].

3.2.5 AutoRegressive Integrated Moving Average (ARIMA)

Le modèle ARIMA (AutoRegressive Integrated Moving Average) est l'une des approches classiques les plus couramment utilisées pour la prédiction de séries temporelles ; il repose sur trois composantes fondamentales :

- **La partie autorégressive (AR) :** Une composante qui modélise la dépendance entre une observation et plusieurs observations retardées. Un modèle AR(p) s'exprime par :

$$\text{AR}(p) : X_t = \phi_1 X_{t-1} + \phi_2 X_{t-2} + \dots + \phi_p X_{t-p} + \epsilon_t$$

où ϕ_i sont les coefficients d'auto-corrélation, ϵ_t représente un bruit blanc et p est l'ordre du modèle AR déterminant le nombre de termes retardés utilisés.

- **La moyenne mobile (MA) :** Une composante qui modélise la dépendance entre une observation et les erreurs de prévision précédentes. Un modèle MA(q) s'exprime par :

$$\text{MA}(q) : X_t = \epsilon_t - \theta_1 \epsilon_{t-1} - \theta_2 \epsilon_{t-2} - \dots - \theta_q \epsilon_{t-q}$$

où ϕ_i sont les coefficients des moyennes mobiles, ϵ_t représente l'erreur de prévision au temps t et q est l'ordre du modèle MA.

- **L'intégration (I) :** Cette composante implique la différenciation de la série temporelle pour la rendre stationnaire. Pour un ordre de différenciation d=1, on calcule :

$$\hat{X}_t = X_t - X_{t-1},$$

Un modèle ARIMA(p,d,q) combine ces trois composantes et se formule comme suit :

$$\hat{y}_t = \mu + \underbrace{\phi_1 y_{t-1} + \phi_2 y_{t-2} + \dots + \phi_p y_{t-p}}_{\text{AR}(p)} - \underbrace{\theta_1 \epsilon_{t-1} - \theta_2 \epsilon_{t-2} - \dots - \theta_q \epsilon_{t-q}}_{\text{MA}(q)},$$

où μ est une constante et y_t représente la série différenciée ($y_t = Y_t - Y_{t-1}$ pour $d = 1$).

Cette formulation montre que chaque valeur à un temps spécifique est influencée à la fois par les observations précédentes (composante AR) et par les erreurs de prévision antérieures (composante MA). ARIMA est particulièrement efficace sur les séries non stationnaires une fois qu'elles ont été stationnarisées par différenciation.

Cette combinaison permet de capturer à la fois la tendance et la structure de dépendance temporelle d'une série. Le modèle ARIMA est donc efficace pour prédire les valeurs futures dans une série univariée, en tenant compte à la fois des observations passées et des erreurs résiduelles.

Toutefois, il présente certaines limites : il suppose que la série soit stationnaire (ou rendue stationnaire par différenciation) et il n'est pas conçu pour traiter directement des données multivariées. Dans ces cas, des extensions telles que le modèle ARIMAX (qui intègre des variables exogènes) ou le modèle VAR (Vector AutoRegression) sont utilisées.

De plus, lorsqu'une série présente des variations saisonnières marquées, le modèle SARIMA (Seasonal ARIMA) peut être employé, en introduisant des paramètres spécifiques pour modéliser la saisonnalité. Bien qu'ARIMA ne repose pas sur des techniques d'apprentissage profond, il reste un outil robuste et interprétable pour la modélisation de séries temporelles dans des contextes industriels.

3.2.6 Transformateurs

Le transformateur, introduit en 2017 pour les tâches de traduction automatique, a rapidement inspiré de nombreux modèles adaptés à divers domaines. Certains conservent l'architecture complète d'origine, tandis que d'autres n'utilisent que le module d'encodeur ou de décodeur, selon les



FIGURE 3.4 – Architecture du transformateur

besoins spécifiques de la tâche. Quel que soit le cas, le **mécanisme d'attention**, en particulier l'*attention multi-tête* (*multi-head attention*), constitue l'élément central de l'apprentissage dans les transformateurs. Ce mécanisme permet au modèle de pondérer différemment les parties d'une séquence d'entrée, en capturant efficacement les dépendances globales [15].

Le transformateur repose sur une architecture de transduction de séquences auto-régressive composée de deux modules principaux : l'**encodeur** et le **décodeur**, chacun constitué de plusieurs couches empilées [15].

Encodeur Chaque couche de l'encodeur comprend :

- une couche d'attention multi-tête (*Multi-Head Attention*) ;
- une couche *feed-forward* (réseau entièrement connecté) ;
- des connexions résiduelles (*Residual Connections*) ;
- des couches d'addition et de normalisation (*Add & Norm*).

L'entrée est d'abord encodée sous forme d'un vecteur d'embedding enrichi d'un encodage positionnel. Ensuite, les matrices de requêtes (*Query*), de clés (*Key*) et de valeurs (*Value*) sont calculées puis traitées par la couche d'attention. La couche *feed-forward* assure une transformation non linéaire de l'information, tandis que les normalisations stabilisent l'apprentissage [15].

Décodeur Le décodeur reprend la même structure que l'encodeur, avec l'ajout d'une couche d'attention **masquée** (*Masked Multi-Head Attention*) afin d'empêcher l'accès aux positions futures. Il applique ensuite une deuxième attention, dite croisée, sur les sorties de l'encodeur. Les vecteurs résultants passent ensuite dans un réseau *feed-forward*, puis dans une couche linéaire et une fonction

SoftMax pour générer les sorties séquentiellement. Ce processus est répété jusqu’à l’obtention du dernier jeton de la séquence [15].

Dans le cadre de la prédiction d’anomalies chimiques au sein de l’unité RFCC, les modèles transformateurs présentent plusieurs avantages significatifs. Leur principal point fort réside dans leur capacité à modéliser des dépendances temporelles à long terme de manière efficace, ce qui est essentiel pour anticiper des anomalies dans un procédé chimique complexe. Contrairement aux RNN ou LSTM qui traitent les séquences de manière séquentielle, les transformateurs s’appuient sur un mécanisme d’attention globale qui leur permet de traiter l’ensemble de la séquence simultanément. Cela favorise une meilleure compréhension des relations entre différentes phases du processus, même éloignées dans le temps, comme les interactions entre la température du régénérateur et la composition du flux en sortie.

De plus, les transformateurs offrent une grande flexibilité architecturale et peuvent être facilement adaptés à des séquences multivariées typiques des capteurs utilisés dans l’unité RFCC. Grâce à leur architecture modulaire, ils permettent d’intégrer efficacement plusieurs types de signaux (température, pression, concentration, etc.), ce qui est crucial pour prédire des comportements anormaux avant qu’ils ne deviennent critiques. Le fait qu’ils puissent être pré-entraînés puis ajustés sur des jeux de données spécifiques renforce leur intérêt pour des environnements comme le RFCC, où les données labellisées sont rares.

Cependant, cette puissance s’accompagne de limitations non négligeables. Tout d’abord, la complexité computationnelle des transformateurs est élevée, en particulier pour les longues séquences temporelles souvent générées par les capteurs industriels. Cela peut poser problème si l’on souhaite un déploiement en temps réel ou sur des systèmes à ressources limitées. Par ailleurs, l’efficacité des transformateurs dépend souvent d’un volume important de données d’entraînement, ce qui peut être un frein dans le cas d’anomalies rares ou de périodes de fonctionnement limitées à analyser.

3.3 Travaux existants

La détection et la prédiction des conditions anormales dans les unités de craquage catalytique à lit fluidisé (FCC) sont devenues des axes de recherche majeurs dans le domaine du raffinage, en raison des risques industriels, des coûts d’arrêt et des pertes de rendement qu’elles impliquent. Face aux limites des approches traditionnelles, de nombreuses études récentes se sont orientées vers des solutions intelligentes, notamment fondées sur l’apprentissage profond, les modèles hybrides et l’analyse causale. Ce chapitre présente une synthèse des contributions majeures dans ce domaine, mettant en lumière des méthodes innovantes alliant robustesse, précision prédictive et capacité d’interprétation. Ces travaux servent de base de comparaison et d’inspiration pour le développement de notre propre solution de détection d’anomalies dans l’unité RFCC de la raffinerie d’Alger.

3.3.1 Travaux de Chenwei Tang et al.

Parmi les contributions récentes les plus prometteuses dans le domaine de la surveillance intelligente des unités FCC, on retrouve le travail autour du cadre AEW-AOC (Attention-based Early Warning for Abnormal Operating Conditions). Cette approche innovante propose une solution de détection précoce des conditions de fonctionnement anormales dans les unités de craquage catalytique à lit fluidisé, en s’appuyant sur les capacités avancées des réseaux de neurones profonds.

L'intérêt principal de cette méthode réside dans sa capacité à dépasser les limites des modèles classiques, notamment ceux à deux étapes, en intégrant une architecture end-to-end qui anticipe les anomalies à partir de séries temporelles industrielles complexes. AEW-AOC repose sur trois modules clés : un Self-Correlation Denoiser (SCD) qui exploite les corrélations spatio-temporelles pour réduire le bruit des données collectées en conditions extrêmes ; un Conv-LSTM, capable de capturer les variations dynamiques et retardées des paramètres critiques de l'unité FCC et un module Anomaly Pattern Attention (APA) qui renforce la capacité du système à distinguer les comportements anormaux à partir de motifs historiques. Ce dernier point est particulièrement intéressant dans un contexte où les classes sont fortement déséquilibrées, les anomalies étant naturellement plus rares que les conditions normales. Les résultats obtenus dans des scénarios industriels réels montrent des performances remarquables, avec une précision supérieure à 90% et soulignent le potentiel de ces techniques à améliorer la fiabilité, la sécurité et la rentabilité des unités FCC [17].

3.3.2 Travaux de Ge He et al.

Une autre approche récente et particulièrement innovante a été proposée pour améliorer la prédiction des rendements dans les unités de craquage catalytique à lit fluidisé (FCC), en exploitant les capacités combinées du deep learning et des algorithmes évolutionnaires. Les auteurs de cette étude ont conçu un modèle hybride appelé Differential Evolutionary Dual-Stage Attention-Based LSTM, qui vise à prédire avec précision les rendements de produits tels que le gaz sec, le GPL, l'essence et le diesel. Ce modèle repose sur deux couches d'attention : la première identifie les paramètres de procédé et les propriétés du catalyseur les plus influents, tandis que la seconde attribue dynamiquement des poids aux valeurs temporelles les plus pertinentes. Pour éviter que l'apprentissage ne stagne dans des solutions locales, un algorithme d'évolution différentielle adaptatif avec stratégie de mutation progressive a été intégré afin d'optimiser les poids de l'attention. En exploitant également le clustering des données catalytiques, les chercheurs ont significativement amélioré les performances prédictives, réduisant les erreurs quadratiques moyennes (RMSE) jusqu'à 95%. Cette étude illustre parfaitement la tendance actuelle à associer réseaux LSTM, mécanismes d'attention et optimisation heuristique, pour modéliser avec précision des processus industriels complexes, hautement non linéaires et dynamiques comme ceux du FCC [18].

3.3.3 Travaux de Wende Tian et al.

Un autre travail remarquable dans le domaine de la prédiction et de l'alerte précoce des anomalies dans les unités FCC est celui qui propose une approche hybride fondée à la fois sur les données et la connaissance, appelée DL-SDG (Deep Learning - Signed Directed Graph). Cette méthode combine la puissance du deep learning avec les capacités explicatives des graphes causaux signés pour anticiper les conditions anormales dans un environnement industriel particulièrement complexe comme le FCC. L'approche commence par l'identification des variables clés via des mesures de centralité dans un réseau complexe, afin de cibler les paramètres les plus critiques du procédé. Ensuite, la réduction de dimension est assurée par le coefficient de corrélation de Spearman, permettant de filtrer les variables redondantes ou bruitées, optimisant ainsi la qualité des données en entrée. Le cœur du modèle repose sur un réseau LSTM enrichi par des mécanismes d'attention et de convolution, qui apprend les évolutions temporelles fines du comportement du système. La grande originalité de ce travail réside dans l'intégration d'un modèle SDG, utilisé après la prédiction pour retracer la propagation probable de l'anomalie dans le système, facilitant ainsi l'interprétation par les opérateurs [19].

3.4 Conclusion

Dans ce chapitre, nous avons exploré les principales approches de prédiction utilisées pour modéliser l'évolution des systèmes industriels à partir de séries temporelles. Qu'il s'agisse de modèles traditionnels ou de réseaux neuronaux profonds, ces techniques constituent le socle des solutions modernes d'anticipation des anomalies.

Avant de concevoir notre propre pipeline prédictif, il est essentiel d'examiner les solutions développées par la communauté scientifique dans des contextes similaires. C'est pourquoi le chapitre suivant présente un état de l'art ciblé des travaux récents appliqués spécifiquement aux unités FCC. Cette revue critique permettra de dégager les tendances actuelles et les bonnes pratiques, tout en identifiant les approches les plus prometteuses à intégrer ou à adapter dans notre propre démarche.

Partie II : Conception et réalisation

Chapitre 4 : Conception

4.1 Introduction

Notre solution doit alerter les ingénieurs bien avant que les pannes n’affectent l’indicateur critique actuellement utilisé par les équipes de Sonatrach.

Cette démarche soulève plusieurs questions :

Quels modèles de prédiction choisir, capables d’anticiper une distribution proche de celle des données réelles ? Quels modèles de détection utiliser, capables de distinguer entre une distribution normale et une distribution anormale sans supervision explicite ? Comment capturer les dynamiques complexes des signaux temporels ? Comment pallier le problème de forte corrélation entre les variables ? Et surtout, comment garantir une bonne généralisation, quelles que soient les données d’inférence ?

Nous présentons donc dans ce chapitre les différentes méthodes utilisées pour détecter et prédire les anomalies dans les séries temporelles issues de l’unité RFCC.

4.2 Données

Dans le cadre de notre projet, nous exploitons des séries temporelles collectées sur une période d’un an au sein de l’unité RFCC. Ces données numériques proviennent de capteurs industriels mesurant différentes grandeurs physiques telles que la température, la pression, le débit et autres. Chaque capteur est identifié par un TAG et les mesures sont enregistrées à intervalles réguliers.

Nous avons étudié plusieurs granularités temporelles avant de retenir un pas d’échantillonnage de 3 minutes. En effet, nous avons initialement commencé avec des données capturées toutes les 10 minutes, mais cela entraînait une perte significative de motifs temporels importants, en plus de générer un nombre de lignes trop faible pour entraîner correctement les modèles.

Nous avons ensuite testé un pas de 30 secondes afin de capturer les motifs courts, mais cela a posé un problème technique : même avec notre modèle le plus léger, les séquences exploitables sur GPU n’excédaient pas une vingtaine de lignes, soit à peine 10 minutes de prévision, ce qui s’est révélé peu rentable et inefficace dans un cadre industriel. Finalement, un compromis a été trouvé avec une granularité de 3 minutes, qui assure un équilibre satisfaisant entre richesse des motifs, taille du dataset et contraintes matérielles.

Pour pallier à la problématique du faible volume de données et ainsi nourrir davantage le dataset d’entraînement, nous avons artificiellement généré un grand nombre de séquences à partir d’un découpage en fenêtres par recouvrement (overlapping) selon une règle où nous décalions d’un pas chaque nouvelle séquence par rapport à la précédente. Cela nous a permis d’augmenter fortement la taille du jeu de données ($30\,104 * 32 = 963\,328$ lignes) tout en préservant la brièveté des motifs.

Cependant cette méthode de génération a eu des effets négatifs majeurs. D’une part, de nom-

breuses séquences contenaient un grand nombre des mêmes lignes de données, ce qui a eu pour conséquence de sur-pondérer artificiellement certains motifs. Le modèle prédictif s’est ainsi sur-spécialisé sur ces zones où la distribution des données est plus ou moins stationnaire, au détriment de sa capacité de généralisation à d’autres zones du processus. D’autre part, ce biais a eu pour conséquence d’induire un phénomène de stationnarité dans les prédictions : pour certaines séquences, le modèle prédisait un comportement quasiment constant, à savoir la répétition identique de la même valeur sur les 32 pas à prédire.

Ces constats nous ont poussés à abandonner ce schéma de génération, au profit d’un découpage par chaînes de séquences disjointes, mieux adapté aux contraintes du problème et plus fidèle aux variations naturelles des données industrielles.

Chaque ligne de données contient un horodatage suivi de dizaines de mesures issues des différents équipements de l’unité. À ce stade, notre objectif est d’analyser la structure de ces données afin de jeter les bases d’un système de détection et de prédiction adéquat.

4.2.1 Prétraitement des données

Le jeu de données comportait initialement 73 variables. Un travail de prétraitement a permis de ramener ce nombre à 23 variables en deux étapes complémentaires. La première a consisté à appliquer des outils statistiques pour identifier et éliminer les variables fortement corrélées, dans le but de supprimer des redondances et de réduire la dimensionnalité du jeu de données. La seconde a fait appel à l’expertise industrielle des ingénieurs de Sonatrach qui ont pu identifier plusieurs variables comme n’étant pas pertinentes pour le diagnostic : certaines présentaient des valeurs systématiquement négatives (indiquant que les capteurs liés n’étaient pas en fonctionnement), d’autres ont été jugées non informatives ou sans lien apparent avec les phénomènes de postcombustion ou de régénération incomplète. Au total, 4 variables ont été écartées à ce stade, permettant d’affiner la qualité du jeu d’entraînement tout en allégeant la charge de calcul.

Par souci de cohérence et de fiabilité dans l’analyse, les données issues des phases de démarrage de l’unité RFCC ont été exclues. Seules les observations correspondant à des régimes stables ont été conservées, ces dernières étant considérées comme plus pertinentes pour l’entraînement des modèles.

Cette réduction du nombre de lignes engendre des discontinuités dans notre dataset. Pour contourner ce problème, nous avons identifié plusieurs fenêtres temporelles exploitables. Toutefois, nous avons volontairement retenu une seule fenêtre (la plus longue) composée de 30 104 lignes. Cette fenêtre présente l’avantage d’être structurellement homogène et centrée sur des conditions opératoires typiques. Les autres fenêtres, bien que potentiellement utilisables, contenaient des incidents liés à des pannes matérielles ou des états transitoires différents de notre cible, ce qui aurait nui à la cohérence temporelle de l’apprentissage. En concaténer plusieurs aurait compromis l’intégrité des motifs, en brisant la continuité temporelle indispensable à une bonne modélisation séquentielle. Ce choix garantit ainsi que notre solution s’applique au bon moment et dans le bon contexte, en ciblant spécifiquement les anomalies liées à la postcombustion et à la régénération incomplète.

Bien que la fenêtre sélectionnée pour l’exploitation ne contienne que 30 104 lignes, ce volume s’est révélé suffisant pour entraîner efficacement les modèles proposés. D’une part, ce choix s’est imposé par contrainte : il s’agissait de la seule fenêtre temporelle disponible présentant à la fois une stabilité des conditions opérationnelles, une continuité temporelle exploitable et une absence de perturbations extérieures majeures (comme des pannes matérielles ou des redémarrages) et aussi le fait que la fin de cette fenêtre (utilisée pour les tests) contient suffisamment d’anomalies qui

nous intéressent (afterburning ou régénération incomplète) pour montrer l’efficacité de notre modèle sur des données jamais vues. D’autre part, les résultats expérimentaux que nous présentons dans les chapitres suivants démontrent clairement que cette quantité de données permet non seulement d’entraîner des modèles performants, mais aussi de garantir une généralisation raisonnable sur des cas non vus.

Il est important de noter que ce jeu de données, bien que modeste en taille par rapport à certains standards du deep learning, possède une densité informationnelle très riche : chaque ligne contient plus d’une vingtaine de mesures physiques critiques, capturant l’état du procédé industriel à un instant donné. Enfin, la réussite de notre approche ne réside pas uniquement dans le volume brut, mais dans la pertinence de la fenêtre choisie, la qualité du prétraitement appliqué et la justesse des méthodes utilisées. Ce compromis entre quantité et qualité a été l’un des leviers clés du succès de notre pipeline, comme en témoigneront les évaluations présentées dans le reste de ce mémoire.

Nous avons fait passer les séquences du training set dans le modèle de forecast afin d’en prédire l’intégralité, séquence par séquence. Ces prédictions ont ensuite été lissées manuellement en retirant les anomalies identifiées dans le régénérateur (grâce à un seuil défini ci-dessous) et le résultat a été utilisé comme dataset d’apprentissage pour le modèle de détection. Cette stratégie nous permet de fournir au modèle de détection un dataset propre, représentatif des situations normales.

Le seuil appliqué pour nettoyer les anomalies du régénérateur repose sur la variable clé suivante :
— TI0037 avec un seuil haut à 730 °C

Ce seuil ne provient pas du manuel d’exploitation, mais de l’expertise directe des ingénieurs procédés. En effet, les manuels ne reflètent que des conditions idéales qui ne correspondent pas à la réalité de l’unité de Sidi Arcine, laquelle a subi plusieurs modifications au fil des années.

Par ailleurs, deux MinMaxScaler distincts ont été utilisés : l’un pour la prédiction et l’autre pour la détection. Cette décision est justifiée par le fait que le dataset utilisé pour la détection est une version prédite (et lissée) du dataset original. Les distributions statistiques étant modifiées par la prédiction, une normalisation séparée s’impose afin de garantir une cohérence d’échelle et de traitement.

Enfin, les ensembles de test ont été définis différemment selon les tâches. Le test set pour la prédiction correspond aux 20% les plus récents de la fenêtre temporelle retenue. Pour la détection, un jeu de 1 442 lignes a été sélectionné à partir du test-set et il a été labellisé manuellement par les ingénieurs procédés à l’aide de leurs comptes rendus hebdomadaires (voir chapitre *Évaluation* pour plus de détails). Cette labellisation manuelle, bien que partielle, constitue une base solide pour évaluer la qualité des détections générées par notre système.

4.3 Conception de notre solution

Dans cette partie, nous exposons l’approche de conception choisie pour apporter une réponse à notre problématique. Cette démarche est synthétisée dans le pipeline illustré dans la Figure 4.1.

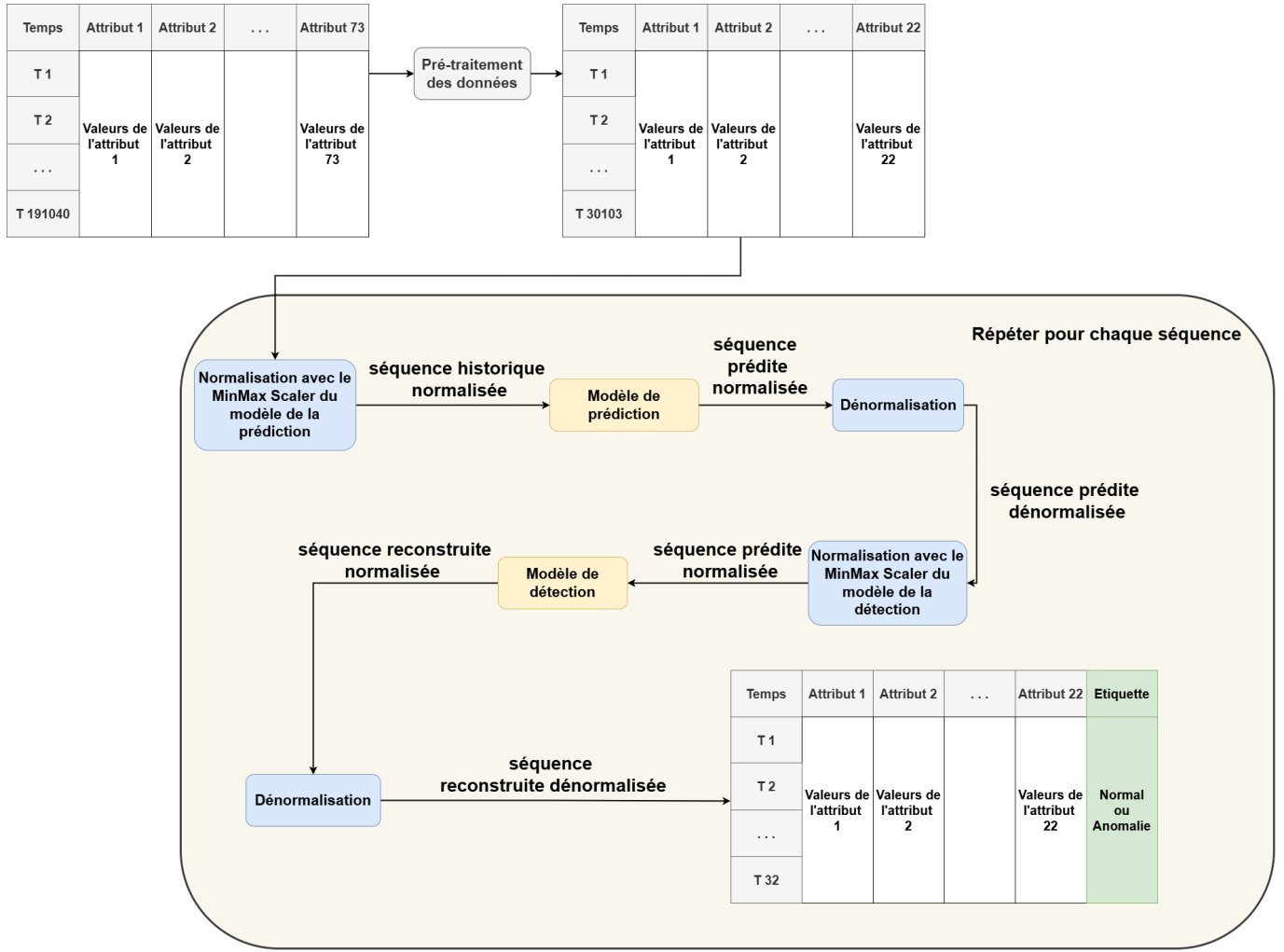


FIGURE 4.1 – Architecture globale de notre système

4.3.1 Prédiction

La notion de prédiction est essentielle dans notre démarche, elle consiste à estimer comment va évoluer notre unité à partir de l'observation de son comportement passé, donc à prévoir la survenue de dysfonctionnements. Cette propriété est notamment essentielle dans le contexte où l'on souhaite détecter des anomalies avant qu'elles ne se manifestent par une dérive observable sur les métriques traditionnellement utilisées.

Dans cette partie, nous allons présenter les différents modèles de prédiction que nous avons explorés au cours de notre travail.

Il faut noter aussi que les données utilisées pour l'entraînement et la validation des modèles ont été organisées sous forme de fenêtres temporelles glissantes. Chaque séquence d'entrée (*input_steps*) correspond à un historique de 32 pas de temps consécutifs, à partir desquels le modèle est chargé de prédire les 32 pas suivants (*forecast_steps*).

4.3.1.1 Modèle Transformateur

Dans notre cas, nous avons choisi une architecture encodeur-seul, sans décodeur, une approche bien adaptée aux séries temporelles. Cette configuration diffère de l'architecture *Transformateur*

classique, illustrée dans la Figure 3.4, initialement conçue pour des tâches de traitement du langage naturel (NLP). L'idée est d'exploiter la capacité du *Transformateur* à modéliser les dépendances à long terme entre différentes variables de capteurs au fil du temps, tout en restant flexible et performant sur des données non structurées de manière séquentielle stricte, comme est souvent le cas dans les procédés industriels.

1. Architecture du modèle

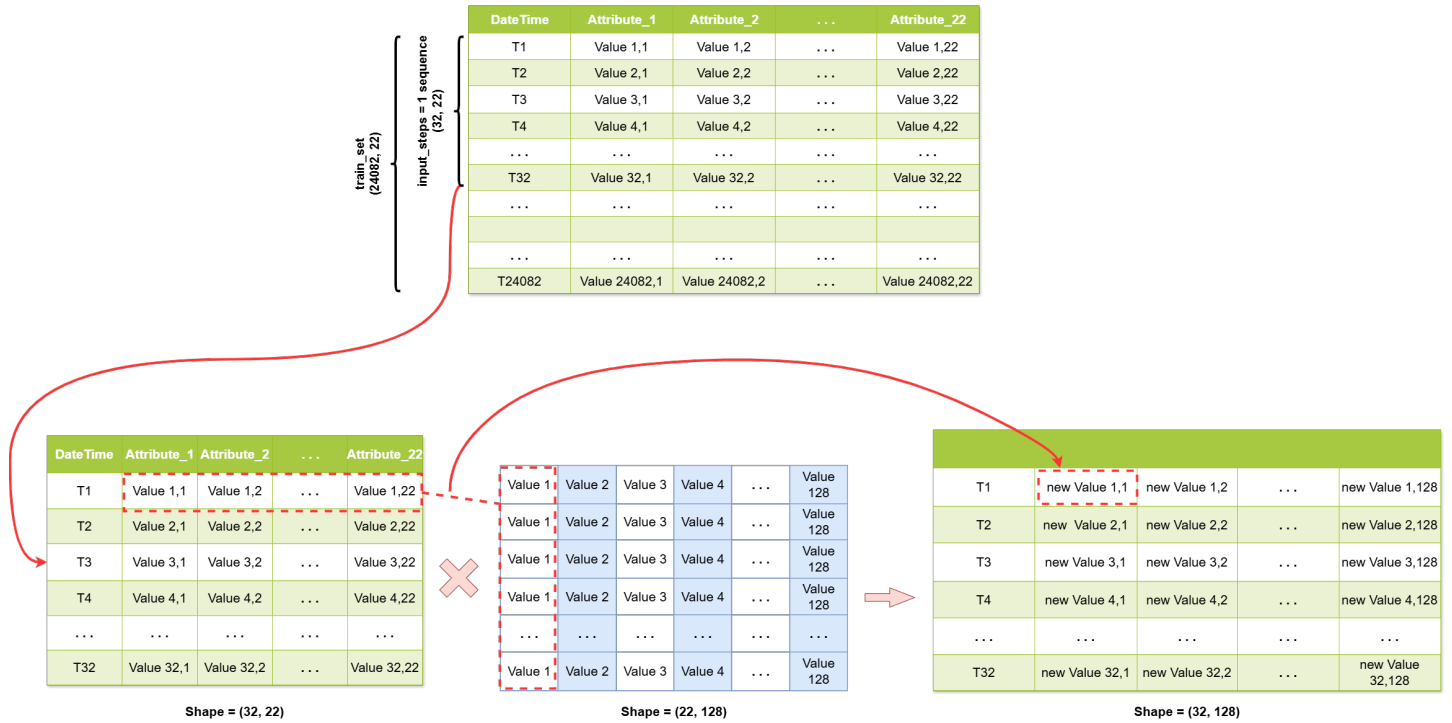
Notre modèle est structuré selon l'architecture suivante :

- Avant l'encodeur :

Pour chaque pas de temps, on a un vecteur multivarié qui est projeté dans un espace de dimension fixe grâce à un embedding dimension. Par exemple, un vecteur à 2 dimensions([val1, val2]), est projeté dans un espace de dimension "embed_dim" ([val1, val2, ..., valembed_dim]).

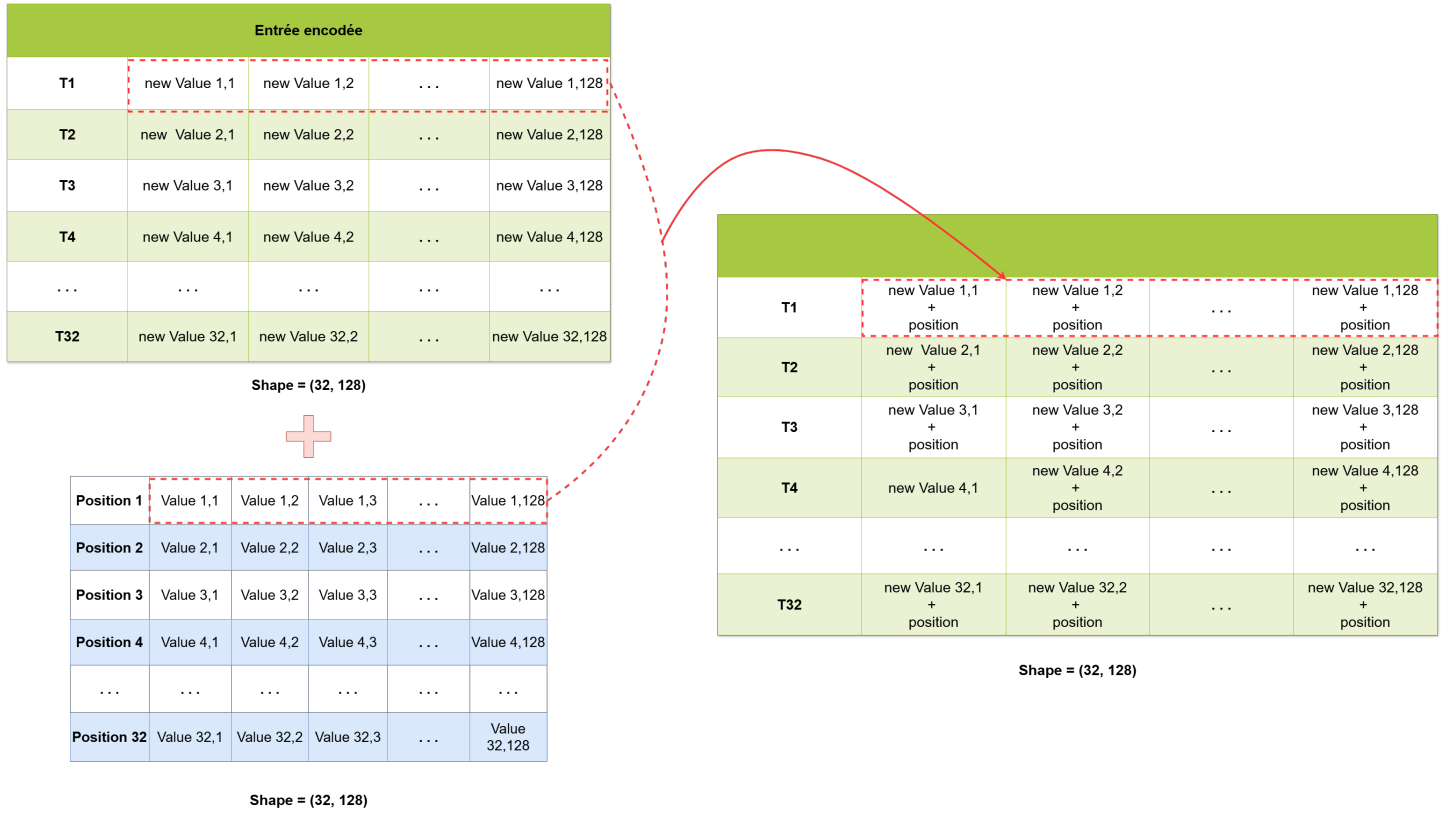
À cette représentation, on ajoute un encodage positionnel, non appris, qui permet au modèle d'intégrer la notion de position temporelle (ex. : la donnée provient-elle du jour 1 ou du jour 15 ?), une information indispensable dans un contexte de séries temporelles.

Ce processus est exécuté sur chaque pas de temps de la séquence en entrée indépendamment et de façon parallèle.



[16]

FIGURE 4.2 – Illustration de la projection linéaire



[16]

FIGURE 4.3 – Illustration de l'encodage positionnel

- Encodeur :

Nous avons 2 encodeurs successifs, chacun étant un bloc transformateur constitué de trois couches principales :

— Attention Multi-têtes :

L'Attention Multi-têtes est une extension du mécanisme d'attention classique. Plutôt que d'avoir une seule vue sur la séquence, le modèle en a plusieurs simultanément, via différentes "têtes". Chaque tête apprend à se concentrer sur un type de relation particulier et ce, en appliquant sa propre transformation aux vecteurs de requêtes (Q), de clés (K) et de valeurs (V) à l'aide matrices spécifiques (W_Q^i, W_K^i, W_V^i) [16].

Les résultats de ces différentes têtes sont ensuite concaténés pour former un vecteur global et projetés à nouveau, donnant une vision globale enrichie de la séquence. Cela permet au modèle d'être plus flexible et performant, sans augmenter considérablement la complexité [16].

La formule générale du processus s'exprime ainsi :

$$MultiHead(Q, K, V) = Concat(head_1, head_2, \dots, head_d).W_O$$

où

$$head_i = Attention(QW_Q^i, KW_K^i, VW_V^i)$$

En d'autres termes, ce bloc permet à chaque pas de temps de "regarder" tous les

autres pas grâce au mécanisme de *self-attention*, qui apprend à pondérer différemment les contributions des autres instants. Par exemple, pour analyser le comportement au pas de temps $t=10$, le modèle peut juger que le pas $t=2$ contient une information très similaire et donc importante [16].

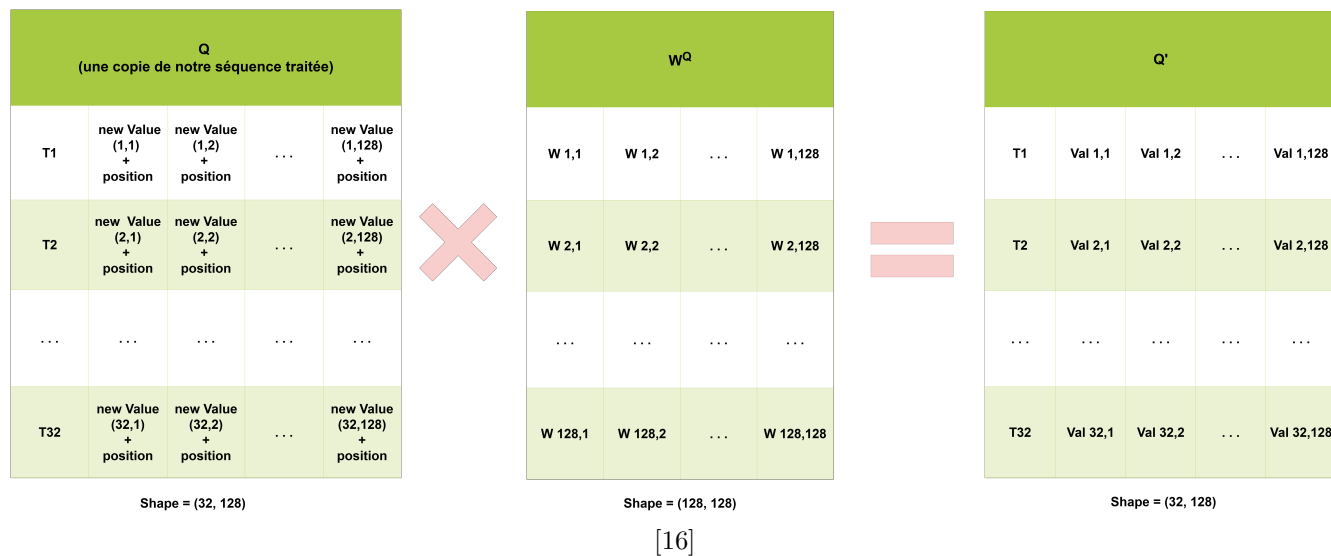


FIGURE 4.4 – Projection linéaire de la matrice Q

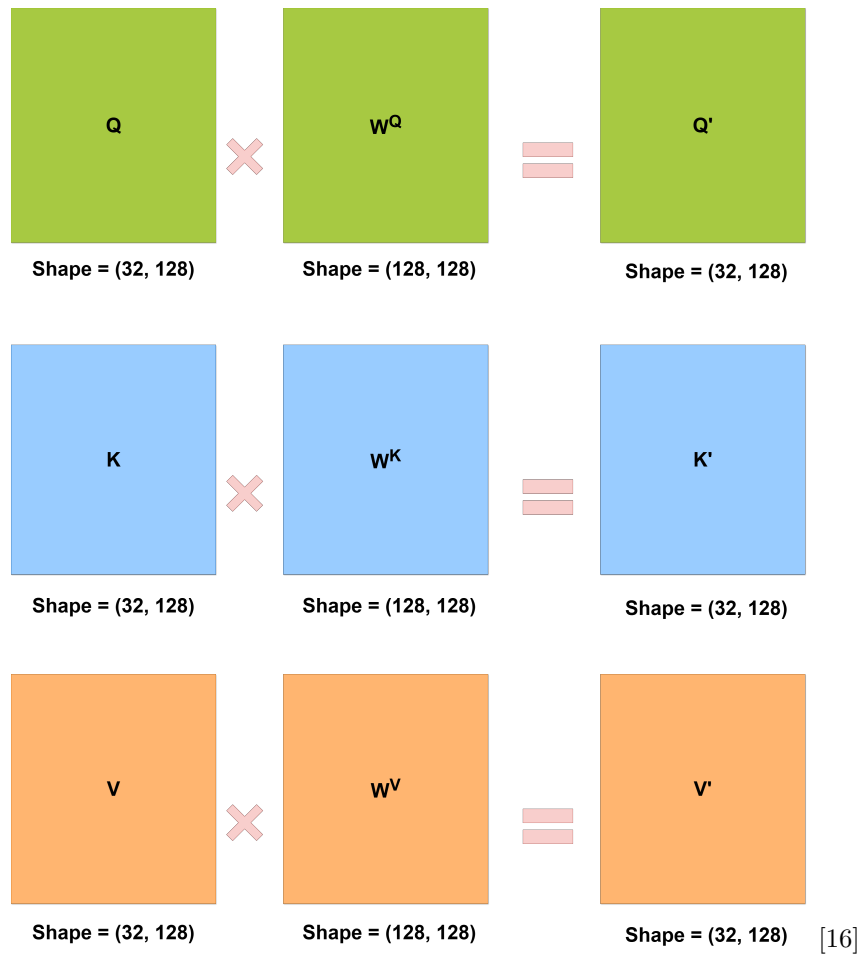
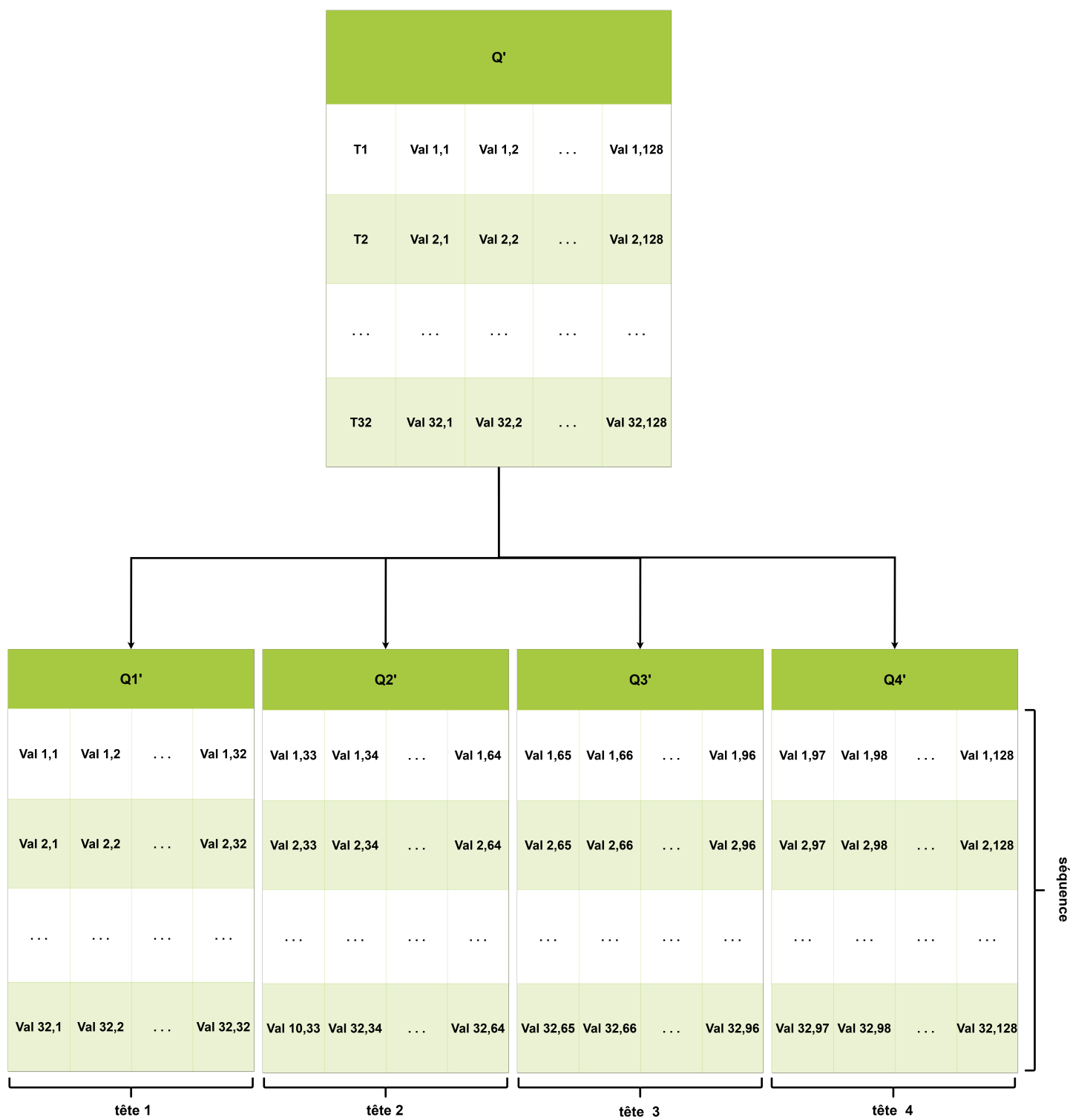


FIGURE 4.5 – Projection linéaire des matrices Q , K , V

Note : L'opération réalisée dans la figure ci-dessus est reproduite de la même manière pour les matrices K et V .



[16]

FIGURE 4.6 – Découpage de représentation Q en sous-espaces

Note : Ceci est un découpage sur un exemple dont le `input_steps=32`, le `nb_heads=4` et l'encodage=128.

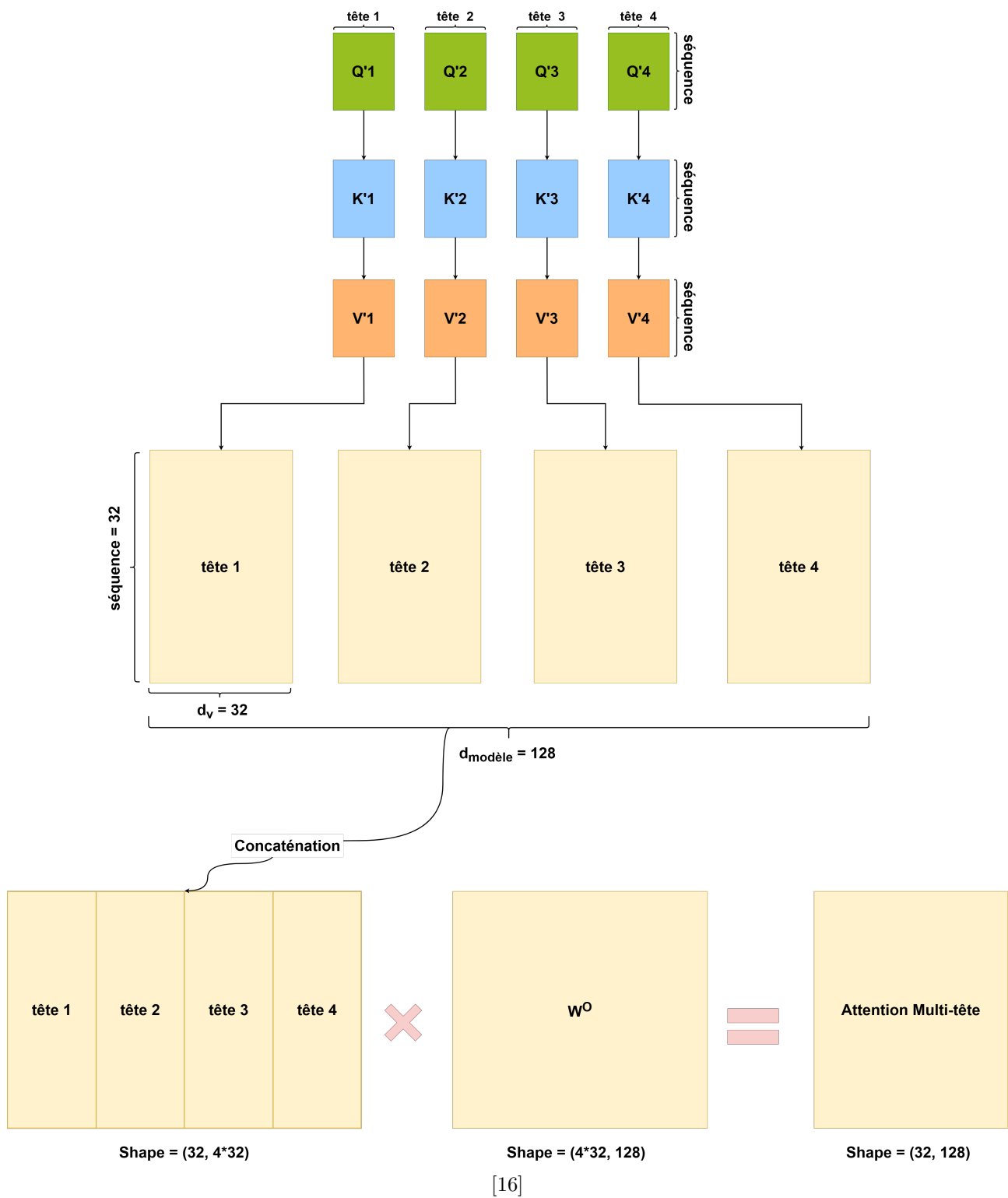


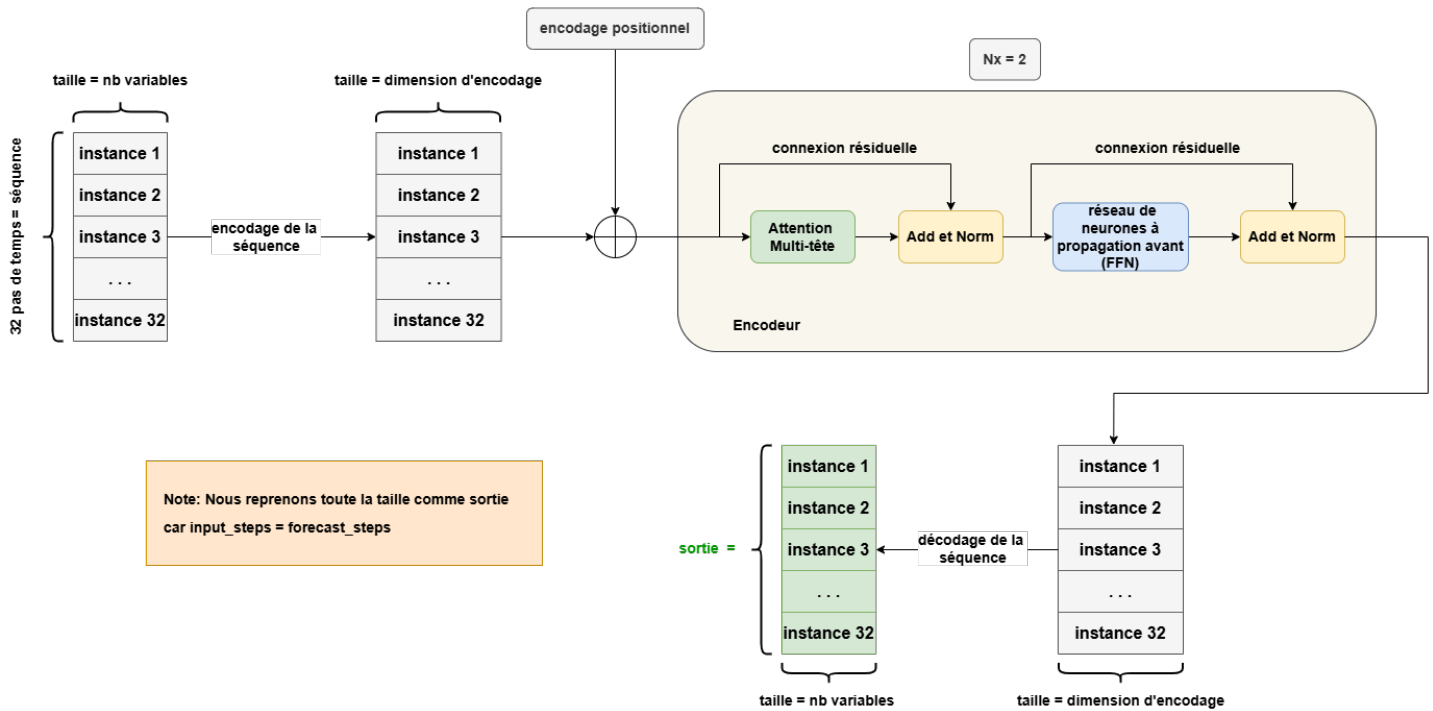
FIGURE 4.7 – Fonctionnement de l'Attention multi-têtes dans un encodeur transformateur

— **Normalisation :**

Cette couche prend en entrée la sortie de la couche précédente, additionnée à la connexion résiduelle (skip connection), afin de stabiliser et d'accélérer l'apprentissage. Cette somme est ensuite soumise à une normalisation.

— **Feed Forward Network :**

Chaque pas de temps est ensuite traité de manière indépendante par un petit réseau de neurones entièrement connecté (Dense $\rightarrow$ ReLU $\rightarrow$ Dense), identique pour chaque pas. La sortie de ce réseau est ajoutée à une connexion résiduelle, puis une normalisation est appliquée de nouveau.



[16]

FIGURE 4.8 – Architecture Transformateur à encoder seul

Algorithm 1 Entraînement_modèle_Transformateur

```
1 : function INITIALIZE_MODELE_TRANSFORMATEUR
Require : longueur de séquence d'entrée  $L$ , longueur de séquence de sortie  $S$ , dimension d'entrée
           $D$ , dimension d'embedding  $E$ , nombre de têtes  $H$ , dimension de la couche feed-forward  $F$ 
Ensure : Modèle  $\mathcal{M}$  initialisé
2 :    $inputs \leftarrow$  séquence en entrée de forme  $(L, D)$ ;
3 :    $x \leftarrow$  projection positionnel de  $inputs$  de la forme  $(L, D, E)$ ;
4 :    $x \leftarrow$  encodeur transformateur de la forme  $(E, H, F)$ ;
5 :    $x \leftarrow$  second encodeur transformateur de la forme  $(E, H, F)$ ;
6 :    $x \leftarrow$  couche dense avec  $E$  unités, activation ReLU;
7 :    $x \leftarrow$  seconde couche dense avec  $E$  unités;
8 :    $outputs \leftarrow x[:, -S :, :];$  /*Sélection des pas à prédire*/
9 :   Compiler  $\mathcal{M}$  avec l'optimiseur Adam et la perte MSE;
10 :  return  $\mathcal{M}$ ;
11 : end function
```

Require : Données d'entraînement $\mathcal{D}_{\text{train}}$, longueur de séquence d'entrée $input_steps$, longueur de séquence de sortie $forecast_steps$, dimension d'entrée $input_dim$, unités LSTM $units$
Ensure : Modèle $\mathcal{M}$ entraîné

Début

```
12 :  $X \leftarrow$  création de séquences à partir de  $\mathcal{D}_{\text{train}}$  avec une fenêtre de taille  $input\_steps$ ;
13 : // indice_debut_X = i
14 :  $Y \leftarrow$  création de séquences à partir de  $\mathcal{D}_{\text{train}}$  avec une fenêtre de taille  $forecast\_steps$ ;
15 : // indice_debut_Y = indice_debut_X + forecast_steps
16 :  $\mathcal{M} \leftarrow$  INITIALIZE_TRANSFORMATEUR( $input\_steps, forecast\_steps, \dim(X), units$ );
17 : Initialiser EarlyStopping avec patience 20 et restauration des meilleurs poids;
18 : Entraîner  $\mathcal{M}$  sur  $(X, Y)$  avec 10% de validation,  $E$  époques et batch de taille  $B$ , sans shuffle;
Fin
```

4.3.1.2 Modèle Informeur

Dans cette étude, nous nous appuyons sur une architecture inspirée des *Informeurs*, un modèle dérivé des *Transformateurs*, spécifiquement optimisé pour la modélisation des longues séries temporelles. À la base, l'encodeur des *Informeurs* reprend la structure du *Transformateur*, avec ses mécanismes d'attention sophistiqués. En revanche, notre approche adopte un décodeur basé sur un *LSTM*, ce qui constitue une hybridation originale entre : l'attention globale pour capter les dépendances longues et la mémoire séquentielle pour la prédiction.

1. Architecture du modèle

Notre modèle est structuré selon l'architecture suivante :

- Avant l'encodage

L'entrée est traitée de la même manière qu'avec un *Transformateur*, la séquence subit un encodage de données (*Data Embedding*), qui combine à la fois l'encodage des valeurs et l'encodage positionnel.

- **Encodeur**

Nous utilisons un seul encodeur, dont la structure interne est très proche de celle d'un *Transformateur*, à quelques détails près, que nous citons ci-dessous :

— **Auto-Attention ProbSparse :**

Cette couche permet à chaque pas de temps de regarder les autres pas et de se concentrer uniquement sur les éléments les plus pertinents de la séquence ; elle ignore ou approxime le reste, pour être plus précis, plutôt que de considérer toutes les requêtes (Query) de manière égale, seules les plus informatives sont retenues. Pour déterminer quelles requêtes sont les plus utiles, une probabilité de sparsité est utilisée, qui repose sur une divergence de *Kullback-Leibler* (KL divergence) entre chaque vecteur de Query et une distribution uniforme. Les requêtes qui s'écartent le plus de cette distribution (c'est-à-dire celles qui portent le plus d'information) sont conservées [16].

Cette technique permet de réduire la complexité de calcul de l'Attention tout en maintenant un bon niveau de performance [16].

Étape	Opération	Forme du tenseur (shape)
1	Entrées Q, K, V	(B, L, D_{model})
2	Projections linéaires : $Q' = QW^Q, K' = KW^K, V' = VW^V$	(B, L, D_{model})
3	Split en têtes	(B, H, L, D_h) où $D_h = D_{\text{model}}/H$
4	Produits scalaires $QK^\top / \sqrt{D_h}$	(B, H, L, L)
5	Calcul de la mesure de sparsité (max - mean)	(B, H, L)
6	Sélection des top- k requêtes selon sparsité	(B, H, k)
7	Masquage des requêtes non sélectionnées	(B, H, L, L)
8	Application du softmax sur les scores filtrés	(B, H, L, L)
9	Produit attention $\times V$	(B, H, L, D_h)
10	Concaténation des têtes	(B, L, D_{model})
11	Projection finale (couche Dense)	(B, L, D_{model})

TABLE 4.1 – Étapes du mécanisme de Attention Multi-tête ProbSparse suivies des formes du tenseur

Où B désigne la taille du batch, L la longueur de la séquence temporelle, H le nombre de têtes d'attention, D_{model} la dimension d'embedding globale et D_h la dimension par tête (i.e. D_{model}/H). k représente le nombre de requêtes sélectionnées selon le critère de sparsité.

— **Normalisation :**

Les sorties obtenues sont additionnées à une connexion résiduelle, la somme est ensuite normalisée, tout comme on le voit dans les *Transformateurs*.

— **Distilling Encoder Layer :**

Le Distilling Encoder Layer, inspiré de l'architecture Informeur, s'appuie dans notre implémentation sur deux couches Conv1D jumelles, un choix délibéré pour

renforcer la robustesse des caractéristiques temporelles.

La première convolution joue le rôle de filtre intelligent : avec son noyau de taille 3 et son padding identique, elle scanne la séquence, capturant les motifs locaux tout en lissant les micro-variations. Le ReLU qui suit agit comme un filtre passe-haut, éliminant le bruit tout en préservant l'essence du signal.

La seconde convolution, son alter ego, opère un second passage sur ces caractéristiques déjà transformées. Comme un éditeur affinant un texte déjà relu, elle peaufine les représentations en détectant des relations plus subtiles. Cette duplication n'est pas redondante : en empilant deux convolutions identiques, nous créons un effet d'approfondissement progressif similaire à une vision binoculaire : chaque couche apporte sa propre perspective sur les mêmes données.

Cette approche, bien que différente du distilling original de l'*Informeur*, offre un compromis idéal entre simplicité d'implémentation et efficacité pour nos données de taille modérée. La normalisation et la connexion résiduelle qui encadrent ces couches agissent comme des garde-fous, garantissant que chaque nouvelle transformation préserve l'information essentielle.

- Décodeur

Dans sa version originale, l'*Informeur* utilise un décodeur basé sur l'architecture *Transformateur*, un choix puissant mais parfois gourmand en ressources. Il commence par un signal initial (un simple "start token"), puis intègre progressivement le contexte global fourni par l'encodeur. Son secret ? Une double mécanique d'attention : d'une part, il analyse les relations entre les différents pas de temps à prédire (auto-attention), d'autre part, il s'aligne en permanence avec les informations de l'encodeur (attention croisée).

Notre adaptation apporte une touche plus pragmatique en remplaçant ce décodeur *Transformateur* par un décodeur à une couche *LSTM*, comme un artisan qui choisirait des outils plus spécialisés. Voici comment cela fonctionne : l'encodeur nous livre son essence sous forme d'un vecteur de contexte (h_T, c_T). Plutôt que de manipuler des mécanismes d'attention complexes, nous dupliquons simplement cette capsule pour chaque pas de temps à prédire. Le *LSTM* joue alors le rôle d'interprète attentif, traitant séquentiellement ces informations tout en préservant leur richesse. Enfin, une dernière transformation (*TimeDistributed*) adapte le résultat à notre problème spécifique.

Cette approche hybride combine le meilleur des deux mondes : la vision globale de l'encodeur *Transformateur* avec l'efficacité et la sensibilité temporelle du *LSTM*. C'est particulièrement adapté quand on travaille avec des séries présentant des motifs locaux marqués ou des ressources de calcul limitées. Notre modèle sait donc extraire l'essentiel tout en restant agile dans sa manière de le restituer.

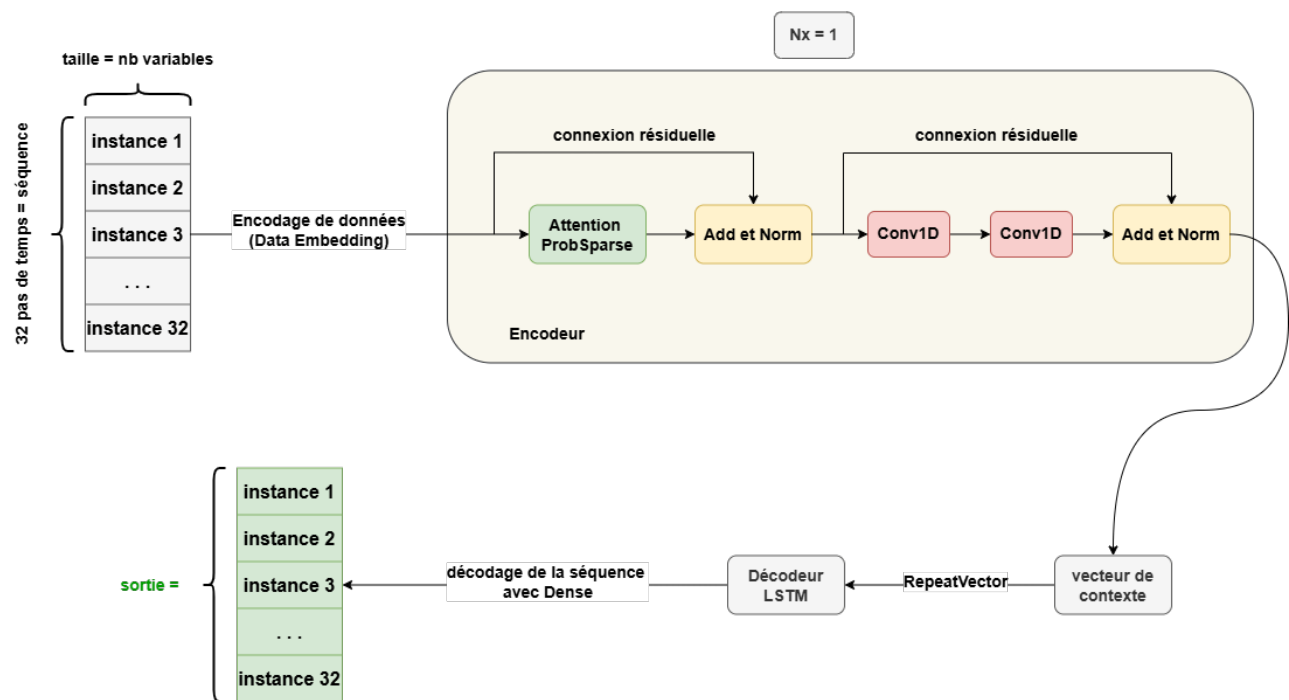


FIGURE 4.9 – Architecture de notre Informer

Algorithm 2 Entraînement_modèle_Informer

1 : **function** INITIALISER_INFORMER
Require : longueur de séquence d'entrée L , longueur de séquence de sortie S , dimension d'entrée D , dimension d'embedding E , nombre de têtes H
Ensure : Modèle $\mathcal{M}$ initialisé
2 : $inputs \leftarrow$ séquence en entrée de forme (L, D) ;
3 : $x \leftarrow$ projection dense vers E dimensions ;
4 : $attn_out \leftarrow$ attention Multi-têtes probabiliste avec H têtes et top-k = K
5 : $x \leftarrow x + attn_out$; //connexion résiduelle
6 : $x \leftarrow$ normalisation par couche ;
7 : $x \leftarrow$ deux convolutions 1D avec E filtres, noyau 3, activation ReLU ;
8 : $x \leftarrow$ duplication du dernier pas de temps sur S étapes par RepeatVector ;
9 : $x \leftarrow$ décodage par LSTM avec E unités, séquences retournées ;
10 : $outputs \leftarrow$ projection dense sur D par TimeDistributed ;
11 : $\mathcal{M} \leftarrow \text{Model}(inputs, outputs)$;
12 : Compiler $\mathcal{M}$ avec Adam et la perte MSE ;
13 : **return** $\mathcal{M}$;
14 : **end function**

Require : Données d'entraînement $\mathcal{D}_{\text{train}}$, longueur de séquence d'entrée $input_steps$, longueur de séquence de sortie $forecast_steps$, dimension d'entrée $input_dim$, unités LSTM $units$
Ensure : Modèle $\mathcal{M}$ entraîné

Début

15 : $X \leftarrow$ création de séquences à partir de $\mathcal{D}_{\text{train}}$ avec une fenêtre de taille $input_steps$;
16 : // indice_debut_X = i
17 : $Y \leftarrow$ création de séquences à partir de $\mathcal{D}_{\text{train}}$ avec une fenêtre de taille $forecast_steps$;
18 : // indice_debut_Y = indice_debut_X + forecast_steps
19 : $\mathcal{M} \leftarrow \text{INITIALISER_INFORMER}(input_steps, forecast_steps, \dim(X), units)$;
20 : Initialiser EarlyStopping avec patience 20 et restauration des meilleurs poids ;
21 : Entraîner $\mathcal{M}$ sur (X, Y) avec 10% de validation, E époques et batch de taille B , sans shuffle ;
Fin

4.3.1.3 Modèle Patch Time Series Transformateur (PatchTST)

Le modèle exploré dans cette phase de prédiction est *PatchTST* (Patch Time Series Transformateur), qui se distingue par son architecture innovante fondée sur les *Transformateurs*. Ce modèle a été spécifiquement conçu pour la prévision de séries temporelles multivariées en introduisant deux mécanismes clés : le **patching** (segmentation des données en blocs temporels) et le **channel-independence** (traitement indépendant des canaux) [20].

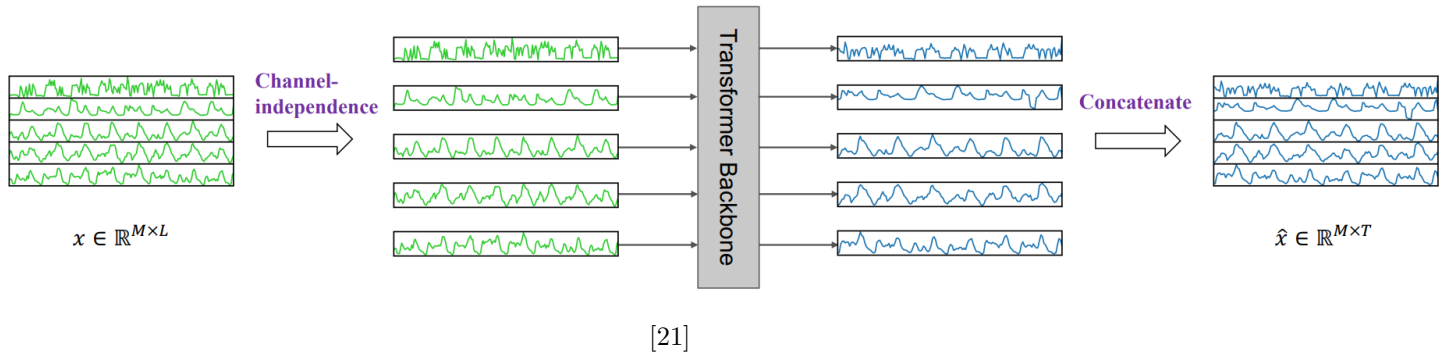


FIGURE 4.10 – Vue globale du modèle PatchTST

1. Architecture du modèle

Nous allons à présent développer la structure du modèle :

- Indépendance des canaux :

L'Indépendance des canaux est un concept clé du *PatchTST*, permettant ainsi de traiter chaque canal (variable) indépendamment des autres, préservant ainsi ses caractéristiques uniques tout au long du processus. [21].

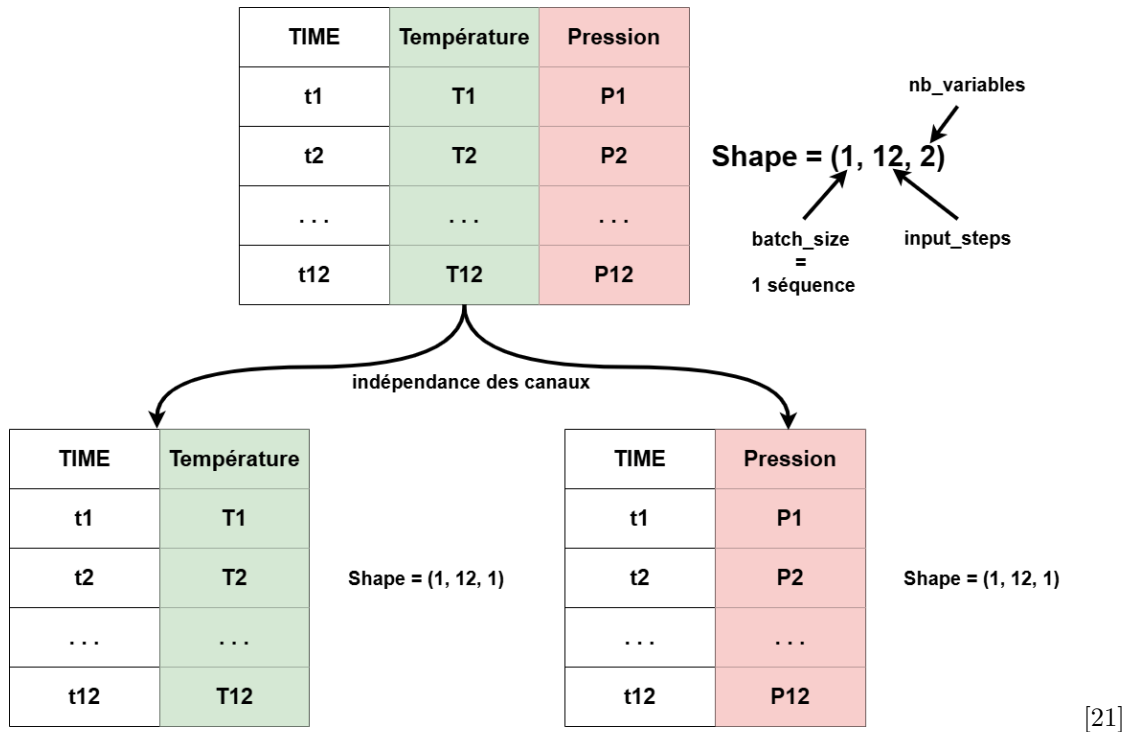


FIGURE 4.11 – Illustration du concept indépendance des canaux

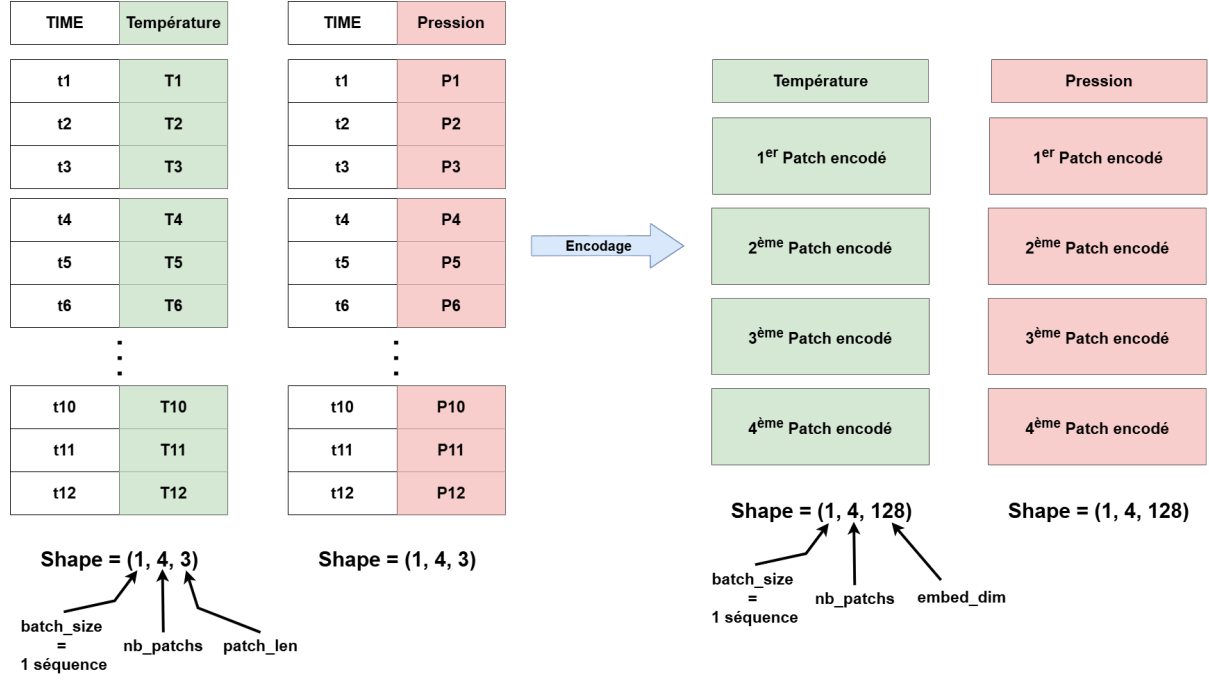
- Découpage en Patch :

Le modèle applique une autre innovation clé, à savoir, le découpage de chaque séquence en fragments de temps (patches) de *patch_len* pas.

Grâce à l'indépendance des canaux, cette transformation s'effectue séparément pour

chaque variable, évitant tout mélange artificiel entre des signaux potentiellement non corrélés [20].

Ces derniers sont ensuite projetés, un par un, dans un espace de dimension défini par *embed_dim*, révélant ainsi des motifs complexes qui n'étaient pas visibles dans les données brutes. Les données prennent alors la forme $(batch_size * num_features, num_patches, embed_dim)$, par exemple dans la Figure 4.12, les données passent du Shape $(1*2, 4, 3)$ à $(1*2, 4, 128)$



[21]

FIGURE 4.12 – Illustration du mécanisme de *Patching* suivi de l'encodage

- Encodeur :

Les représentations obtenues grâce aux deux étapes précédentes sont ensuite traitées par un bloc *Transformateur*, appliqué indépendamment à chaque variable. Ce bloc exploite les mêmes composants que ceux décrits précédemment, garantissant un encodage riche tout en préservant la structure propre à chaque canal.

Tout au long de cette phase, la forme du tenseur reste inchangée, soit $(batch_size * num_features, num_patches, embed_dim)$. À l'issue du bloc, chaque patch de chaque variable est représenté par un vecteur contextualisé, intégrant l'information temporelle pertinente sans introduire de mélange entre les canaux.

- Sorties :

Une fois les représentations enrichies extraites par l'encodeur, le modèle sélectionne uniquement le dernier patch de chaque variable, considéré comme contenant les informations les plus récentes et donc les plus pertinentes pour la prédiction. Cette opération de réduction temporelle permet d'obtenir un tenseur de forme $(batch_size * num_features, embed_dim)$, qui est ensuite réorganisé pour retrouver la structure par batch et par canal, donnant $(batch_size, num_features, embed_dim)$.

Chaque vecteur ainsi obtenu est ensuite projeté, via une couche dense, vers une sé-

quence temporelle de taille égale au nombre d’instants à prédire, noté *forecast_steps*. Cela donne une sortie de forme $(batch_size, num_features, forecast_steps)$, qui est finalement transposée pour adopter le format standard $(batch_size, forecast_steps, num_features)$.

Le modèle produit donc, pour chaque échantillon du batch, une prédiction multivariée sur les *forecast_steps* à venir, en maintenant une séparation explicite entre les canaux tout au long du processus.

Étape	Forme du tenseur (shape)
Entrée brute	$(batch_size, input_steps, num_features)$
Reshape pour canal indépendant	$(batch_size \times num_features, input_steps, 1)$
Découpage en patches	$(batch_size \times num_features, num_patches, patch_len)$
Projection (embedding)	$(batch_size \times num_features, num_patches, embed_dim)$
Sortie de l’encodeur Transformateur	$(batch_size \times num_features, num_patches, embed_dim)$
Sélection du dernier patch	$(batch_size \times num_features, embed_dim)$
Reshape vers batch + canal	$(batch_size, num_features, embed_dim)$
Projection vers les pas à prédire	$(batch_size, num_features, forecast_steps)$
Transposition finale (sortie du modèle)	$(batch_size, forecast_steps, num_features)$

TABLE 4.2 – Résumé des étapes du modèle PatchTST et des transformations successives des dimensions des tenseurs

Algorithm 3 Entraînement_modèle_PatchTST_multi-couches

```
1 : function INITIALISATION_PATCHTST_MULTI-COUCHES( $L, D, P, E, H, N, S$ )
Require :  $L$  : Longueur des séquences en entrée,  $D$  : Nombre de features (dimensions),  $P$  :
          Longueur des patches,  $E$  : Dimension de l'embedding,  $H$  : Nombre de têtes d'attention,  $N$  :
          Nombre de blocs Transformateur,  $S$  : Longueur de la séquence de sortie
Ensure : Modèle  $\mathcal{M}$  initialisé
2 :   Vérifier que  $L$  est divisible par  $P$ 
3 :    $N_p \leftarrow L/P$  ▷ Nombre de patches
4 :    $inputs \leftarrow$  séquence d'entrée de forme  $(L, D)$ 
      // Indépendance des canaux (features traités séparément)
5 :    $x \leftarrow \text{reshape}(inputs, \text{forme} = (\text{batch} \times D, L, 1))$ 
      // Découpage en patches
6 :    $x \leftarrow \text{reshape}(x, \text{forme} = (\text{batch} \times D, N_p, P))$ 
      // Projection des patches
7 :    $x \leftarrow \text{Dense}(E)$  appliquée à chaque patch
      // Blocs Transformateur
8 :   for  $i = 1$  to  $N$  do
9 :      $attn \leftarrow \text{MultiHeadAttention}(H, \text{key\_dim} = E)$  sur  $x$ 
10 :     $x \leftarrow \text{LayerNormalization}(\text{Add}(x, attn))$ 
11 :     $feed\_forward\_network \leftarrow \text{Dense}(4E, \text{activation}=\text{gelu})(x)$ 
12 :     $feed\_forward\_network \leftarrow \text{Dense}(E)(feed\_forward\_network)$ 
13 :     $x \leftarrow \text{LayerNormalization}(\text{Add}(x, feed\_forward\_network))$ 
14 :   end for
      // Sélection du dernier patch
15 :    $x \leftarrow x[:, -1, :]$  ▷ Dernier patch de chaque séquence
16 :    $x \leftarrow \text{reshape}(x, \text{forme} = (\text{batch}, D, E))$ 
      // Projection vers les pas de prévision
17 :    $x \leftarrow \text{Dense}(S)$  appliquée à chaque feature
18 :    $output \leftarrow \text{transpose}(x, \text{axes} = (\text{batch}, S, D))$ 
19 :    $\mathcal{M} \leftarrow \text{Model}(inputs, output)$ 
20 :   Compiler  $\mathcal{M}$  avec Adam ( $lr = 10^{-3}$ ) et perte MSE
21 :   return  $\mathcal{M}$ 
22 : end function
```

Require : Données d'entraînement $\mathcal{D}_{\text{train}}$, hyperparamètres $(L, S, D, P, E, H, N, B, epochs)$

Ensure : Modèle $\mathcal{M}$ entraîné

Début

```
23 :  $X \leftarrow$  création de séquences d'entrée ( $L$ ) depuis  $\mathcal{D}_{\text{train}}$ 
24 :  $Y \leftarrow$  séquences cibles alignées ( $S$ ) avec  $X$ 
25 :  $\mathcal{M} \leftarrow \text{INITIALISATION\_PATCHTST\_MULTI-COUCHES}(L, D, P, E, H, N, S)$ 
26 : Initialiser EarlyStopping (patience = 20, restore_best_weights = True)
27 : Entraîner  $\mathcal{M}$  sur  $(X, Y)$  avec validation 10%, batch  $B$ , époques  $epochs$ , sans shuffle
28 : return  $\mathcal{M}$ 
```

Fin

4.3.1.4 Modèle LSTM Sequence-to-Sequence (Seq2Seq)

Le modèle exploré dans cette phase de prédiction est basé sur l'architecture *LSTM* de type sequence-to-sequence. Il repose sur une architecture Encodeur-Décodeur, bien adaptée aux tâches de prévision multivariée séquentielle. Elle se compose de deux modules principaux : un encodeur et un décodeur, tous deux basés sur des cellules *LSTM*. Le principe consiste à encoder une séquence d'entrée (d'une durée fixe) en un vecteur latent, puis à la décoder en une séquence de sortie de même taille.

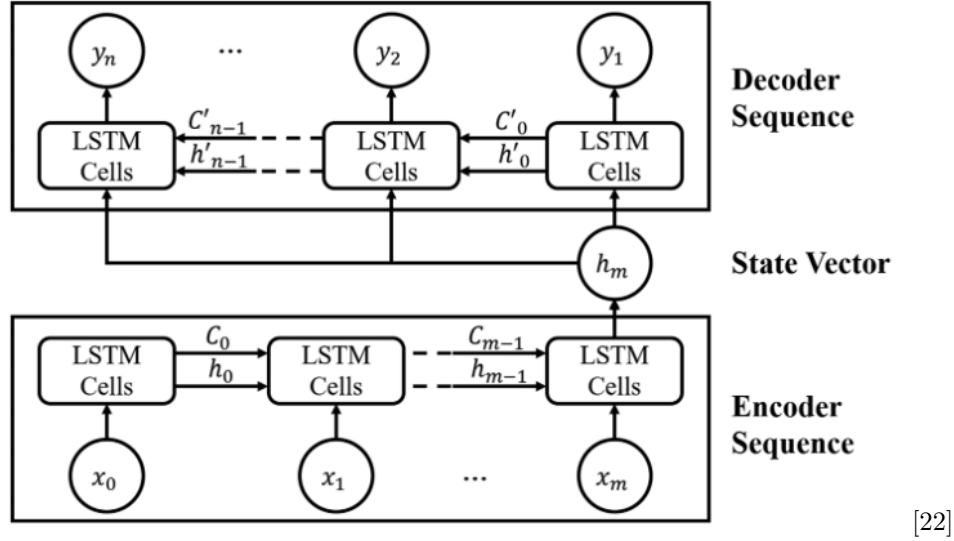


FIGURE 4.13 – Architecture générale LSTM seq2seq

1. Architecture du modèle

Le modèle est structuré selon l'architecture suivante :

- Encodeur :

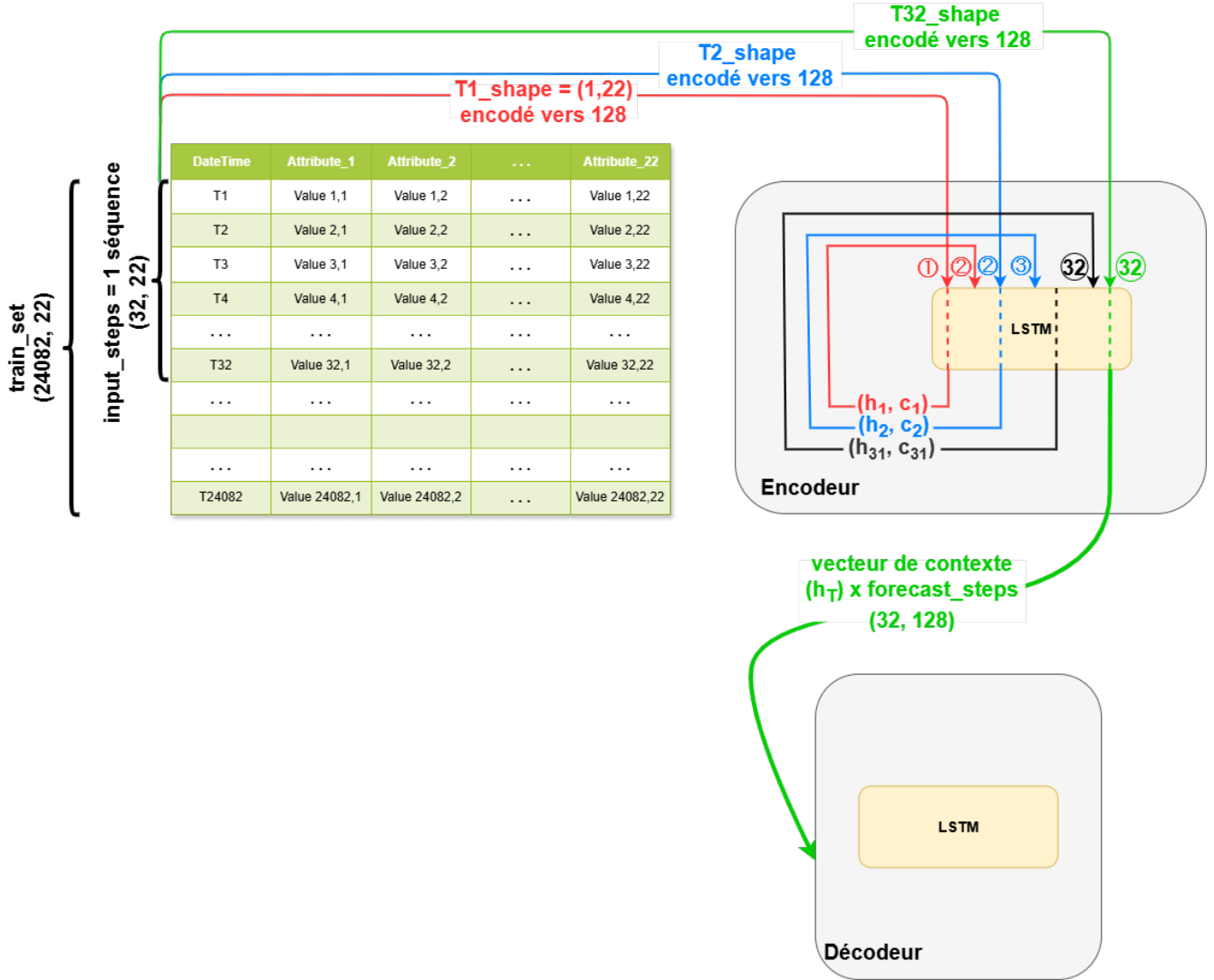
L'encodeur consiste en une couche *LSTM* simple, il traite séquentiellement une séquence d'entrée de dimension (32, 22), où chaque pas de temps correspond à un vecteur de 22 attributs. Concrètement, cela signifie qu'il analyse successivement 32 étapes, chacune représentant un instantané de nos données.

À chaque étape, le *LSTM* effectue une projection du vecteur d'entrée (1, 22) dans un espace latent de dimension *embed_dimension*, correspondant au nombre d'unités de la couche. Ce processus s'apparente à une traduction intelligente, où chaque observation brute est convertie en une représentation riche en informations.

Le mécanisme repose sur une mémoire dynamique : la couche reçoit à la fois le pas de temps courant et ses états internes précédents (h_{t-1}, c_{t-1}). Elle produit alors de nouveaux états (h_t, c_t), mettant à jour en permanence sa compréhension de la séquence. L'état caché h_t capture les motifs immédiats, tandis que l'état de cellule c_t conserve une

mémoire à long terme des tendances globales. Nous précisons aussi que les valeurs h_0 et c_0 sont initialisées à 0.

À l'issue de ce traitement, seule la dernière sortie est conservée : le vecteur de contexte h_T (1, $embed_dimension$). Ce choix architectural est délibéré, ce vecteur condense l'information essentielle de toute la séquence. Notons qu'avec $return_sequences = False$, nous privilégions cette synthèse finale au détriment des états intermédiaires, simplifiant ainsi la transmission d'informations entre les modules.



[22]

FIGURE 4.14 – Architecture de l'encodeur de notre modèle LSTMseq2seq

Note : Ceci est un déroulement sur un $input_steps=32$ et un $embed_dimension=128$.

- Vecteur de contexte :

Le h_T obtenu à la sortie de l'encodeur est répliqué autant de fois qu'il y a de pas de temps à prédire (32 fois). Cela signifie qu'on crée une séquence de taille $(32, embed_dimension)$, où chaque pas à prédire est initialisé avec le même vecteur h_T .

Cette duplication permet au décodeur de disposer d'un contexte constant pour générer chaque prédiction temporelle.

- **Décodeur :**

Le décodeur, aussi comprenant une couche LSTM, reçoit en entrée une séquence constante de taille $(32, embed_dimension)$, obtenue en dupliquant le vecteur de contexte h_T (produit par l'encodeur) $forecast_steps$ fois à l'aide d'un *RepeatVector()*. À chaque pas de temps, le décodeur reçoit en entrée le même vecteur de contexte (identique pour tous les pas), ainsi que les états internes précédents (h_{t-1}, c_{t-1}) ; ce sont ces états internes qui évoluent au fil du temps, permettant au modèle de générer une séquence de sorties cohérente malgré des entrées constantes. Contrairement à une approche classique avec *Teacher Forcing*, ici les vraies sorties précédentes ne sont pas utilisées pendant l'entraînement. Dans un modèle utilisant *Teacher Forcing*, le vecteur de contexte ne serait utilisé que pour initialiser les états du décodeur, à savoir l'état caché h_0 et l'état de cellule c_0 ; ensuite, chaque pas recevrait comme entrée la valeur réelle précédente ainsi que ses propres états internes mis à jour au pas précédent (voir la Figure 4.15).

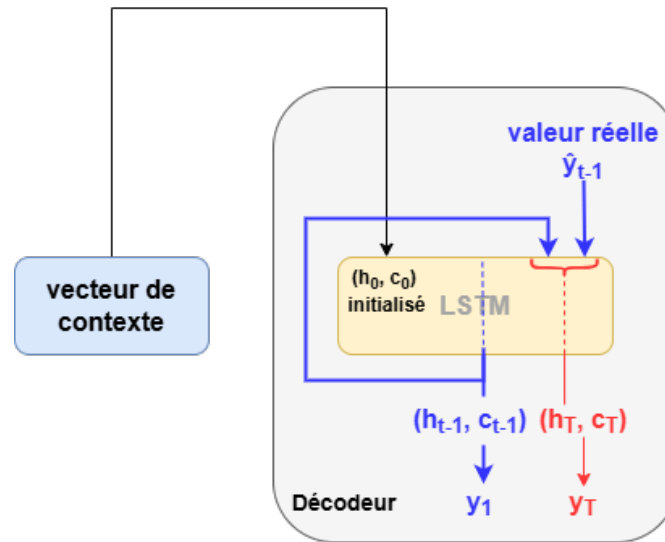


FIGURE 4.15 – Architecture décodeur à une cellule LSTM avec la méthode *Teacher Forcing*

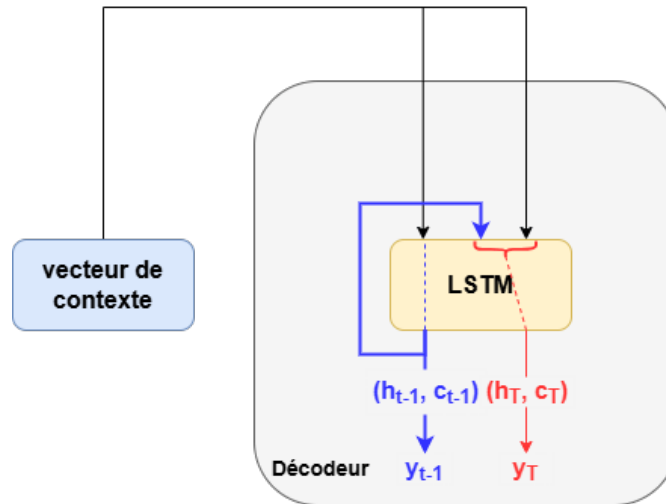


FIGURE 4.16 – Architecture décodeur à une cellule LSTM sans la méthode *Teacher Forcing*

- **Sortie :**

Les sorties du décodeur sont des vecteurs de dimension $(1, embed_dimension)$. Pour les ramener à une sortie compatible avec la dimension des variables initiales, on utilise une couche Dense.

Étant donné que la sortie est une séquence, on utilise la couche `TimeDistributed(Dense(22))` pour appliquer une même couche Dense à chaque pas de temps, sans devoir écrire une boucle manuellement.

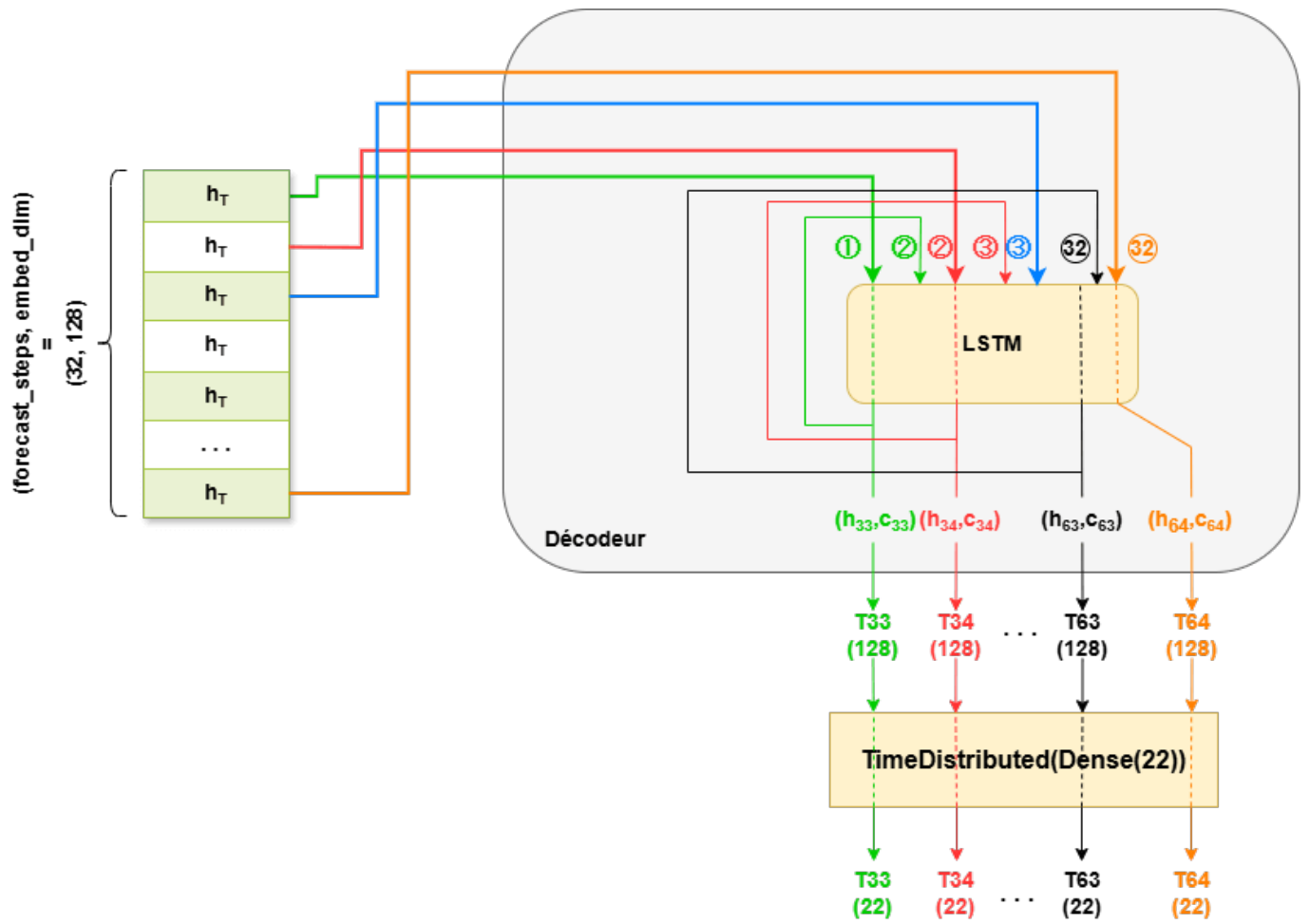


FIGURE 4.17 – Mécanisme du décodeur de notre modèle LSTM seq2seq

Algorithm 4 Entraînement_modèle_LSTM_Seq2Seq

```
1 : function INITIALISATION_LSTM_SEQ2SEQ
Require : longueur de séquence entrante  $L$ , longueur de séquence sortante  $S$ , dimension d'entrée
            $D$ , unités LSTM  $U$ 
Ensure : Modèle  $\mathcal{M}$  initialisé
2 :    $inputs \leftarrow$  séquence en entrée de forme  $(L, D)$ ;
3 :    $x \leftarrow$  encodage LSTM de  $inputs$  avec  $U$  unités;
4 :    $x \leftarrow$  duplication de  $x$  avec RepeatVector sur  $S$  pas de temps;
5 :    $x \leftarrow$  décodage LSTM de  $x$  avec  $U$  unités;
6 :    $outputs \leftarrow$  reconstruction par TimeDistributed(Dense( $D$ ));
7 :    $\mathcal{M} \leftarrow$  Model( $inputs, outputs$ );
8 :   Compiler  $\mathcal{M}$  avec l'optimiseur Adam et la perte MSE;
9 :   return  $\mathcal{M}$ ;
10 : end function
```

Require : Données d'entraînement $\mathcal{D}_{\text{train}}$, longueur de séquence $input_steps$, longueur de séquence de sortie $forecast_steps$, dimension d'entrée $input_dim$, unités LSTM $units$
Ensure : Modèle $\mathcal{M}$ entraîné

Début

```
11 :  $X \leftarrow$  création de séquences à partir de  $\mathcal{D}_{\text{train}}$  avec une fenêtre de taille  $input\_steps$ ;
12 : // indice_debut_X = i
13 :  $Y \leftarrow$  création de séquences à partir de  $\mathcal{D}_{\text{train}}$  avec une fenêtre de taille  $forecast\_steps$ ;
14 : // indice_debut_Y = indice_debut_X +  $forecast\_steps$ 
15 :  $\mathcal{M} \leftarrow$  INITIALIZE_LSTM_AE( $input\_steps, forecast\_steps, \dim(X), units$ );
16 : Initialiser EarlyStopping avec patience 20 et restauration des meilleurs poids;
17 : Entraîner  $\mathcal{M}$  sur  $(X, Y)$  avec 10% de validation,  $E$  époques et batch de taille  $B$ , sans shuffle;
Fin
```

4.3.1.5 Modèle PatchTST personnalisé

Le modèle présenté ici est une adaptation du *PatchTST* à notre jeu de données, dans laquelle le mécanisme de l'indépendance des canaux a été exclu en raison de la forte corrélation entre les attributs et un décodeur de type *LSTM* a été intégré afin d'améliorer la capacité de prédiction.

1. Architecture du modèle

Nous allons à présent présenter le fonctionnement du modèle *PatchTST* adapté à notre problématique :

- **Découpage en patches :**

Le découpage en patches s'exécute comme nous l'avons décrit pour le modèle *PatchTST* standard.

- **Encodeur Transformateur :**

Le bloc *Transformateur* nommé ici *Transformateur backbone*, entre alors en action avec son mécanisme d'attention sophistiqué, ce mécanisme est identique à celui que

nous avons exploré précédemment dans la section *Transformateurs* en excluant la *Feed Forward Network*.

- **Global Pooling :**

Après l'analyse attentive des patches, nous appliquons un *Global Average Pooling* qui agit comme un condensateur d'information. Cette étape cruciale extrait l'essentiel en calculant la moyenne des caractéristiques sur l'ensemble des patches, produisant ainsi une signature unique qui capture les motifs dominants de la séquence. Comme un résumé expert, il distille les informations clés tout en réduisant la dimensionnalité, préparant le terrain pour la phase de prédiction.

- **Décodeur LSTM :**

Nous avons adapté ce modèle à notre jeu de données en y intégrant un décodeur basé sur une architecture *LSTM* (une seule couche). Celui-ci génère les prédictions de manière autoregressive à partir d'une représentation globale compressée, répétée *forecast_steps* fois à l'aide de la couche *RepeatVector*, de façon à produire une sortie temporelle complète.

Le fonctionnement détaillé du décodeur *LSTM*, ainsi que l'obtention des sorties sont présentés dans la section précédente.

Algorithm 5 Entraînement_modèle_PatchTST_personnalisé

```
1 : function INITIALISATION_PATCHTST_PERSONNALISÉ( $L, D, P, E, H, S$ )  
Require :  $L$  : longueur des séquences en entrée,  $D$  : nombre de features (dimension d'entrée),  
           $P$  : longueur des patches,  $E$  : dimension d'embedding,  $H$  : nombre de têtes d'attention,  $S$  :  
          longueur de la séquence en sortie  
Ensure : Modèle  $\mathcal{M}$  initialisé  
2 :   Vérifier que  $L$  est divisible par  $P$   
3 :    $N_p \leftarrow L/P$  ▷ Nombre de patches  
4 :    $inputs \leftarrow$  séquence d'entrée de forme  $(L, D)$   
5 :    $patches \leftarrow$  réarrangement de  $inputs$  en  $(N_p, P \times D)$   
6 :    $embedded \leftarrow$  couche Dense( $E$ ) appliquée à chaque patch  
7 :    $attn\_output \leftarrow$  MultiHeadAttention( $num\_heads = H, key\_dim = E$ ) sur  $embedded$   
8 :    $x \leftarrow$  Add( $embedded, attn\_output$ )  
9 :    $x \leftarrow$  Normalisation( $x$ )  
10 :   $x \leftarrow$  GlobalAveragePooling1D( $x$ )  
11 :   $x \leftarrow$  RepeatVector( $S$ ) de  $x$   
12 :   $x \leftarrow$  LSTM( $E, return\_sequences=True$ ) sur  $x$   
13 :   $output \leftarrow$  TimeDistributed(Dense( $D$ )) sur  $x$   
14 :   $\mathcal{M} \leftarrow$  Model( $inputs, output$ )  
15 :  Compiler  $\mathcal{M}$  avec Adam et perte MSE  
16 :  return  $\mathcal{M}$   
17 : end function
```

Require : Données d'entraînement $\mathcal{D}_{\text{train}}$, hyperparamètres $(L, S, D, P, E, H, B, epochs)$

Ensure : Modèle $\mathcal{M}$ entraîné

Début

```
18 :  $X \leftarrow$  création de séquences d'entrée de taille  $L$  depuis  $\mathcal{D}_{\text{train}}$   
19 :  $Y \leftarrow$  création de séquences cibles de taille  $S$  alignées avec  $X$   
20 :  $\mathcal{M} \leftarrow$  INITIALISATION_PATCHTST_PERSONNALISÉ( $L, D, P, E, H, S$ )  
21 : Initialiser EarlyStopping avec patience 20 et restauration des meilleurs poids ;  
22 : Entraîner  $\mathcal{M}$  sur  $(X, Y)$  avec validation_split = 0.1, epochs =  $epochs$ , batch_size =  $B$ , sans  
    shuffle  
23 : return  $\mathcal{M}$   
    Fin
```

4.3.1.5 Modèle LSTM Attention Multi-tête

Ce modèle est une combinaison du mécanisme *LSTM* seq2seq et de l'Attention multi-tête que nous avons évoquée plus tôt dans les *Transformateurs*. Il repose donc sur : un encodeur LSTM et un décodeur enrichi par le mécanisme d'attention Multi-tête.

1. Architecture du modèle

Nous allons à présent présenter le fonctionnement du modèle combiné :

- Encodeur LSTM :

l'encodeur ici la même structure que celle observée dans la rubrique consacrée au

modèle LSTM seq2seq classique. Il produit à sa sortie les états finaux (h_T, c_T), h_T fera office de vecteur de contexte

- **Décodeur LSTM avec Attention multi-tête :**

Ce dernier, à une couche LSTM, prend en entrée le vecteur de contexte (résumé global de la séquence d'entrée) qui est répété autant de fois que le nombre de pas de prédiction souhaités. Cette couche est initialisée avec les états finaux de l'encodeur : l'état caché initial est h_T et l'état de cellule initial est c_T . À chaque pas de temps, le décodeur produit un vecteur de sortie, formant ainsi une séquence *decoder_outputs*.

Ces sorties alimentent ensuite un module d'attention Multi-têtes, où les requêtes (queries) proviennent du décodeur, tandis que les clés (keys) et valeurs (values) proviennent des sorties de l'encodeur. Ce mécanisme permet au décodeur de porter une attention ciblée sur différentes parties de la séquence d'entrée à chaque instant.

Enfin, les sorties du décodeur et celles de l'attention sont concaténées, normalisées par une couche de Layer Normalization, puis traitées par une couche Dense appliquée de manière temporelle (TimeDistributed(Dense)) afin de générer les prédictions finales, de dimension *input_dim*, pour toute la fenêtre de prédiction.

Algorithm 6 Entraînement modèle LSTM Attention Multi-tête

1 : **function** INITIALISATION_LSTM_MULTIHEAD_ATTENTION

Require : longueur de séquence entrante L , longueur de séquence sortante S , dimension d'entrée D , unités LSTM U

Ensure : Modèle $\mathcal{M}$ initialisé

```
2 :    $inputs \leftarrow$  séquence en entrée de forme  $(L, D)$  ;
3 :    $encoder\_outputs, h_T, c_T \leftarrow$  encodage LSTM de  $inputs$  avec  $U$  unités ;
4 :    $context\_vector \leftarrow h_T$  ;
5 :    $decoder\_inputs \leftarrow$  duplication de  $context\_vector$  avec RepeatVector sur  $S$  pas de temps ;
6 :    $decoder\_lstm \leftarrow$  LSTM(units) ;
7 :    $decoder\_outputs \leftarrow$  decoder_lstm(decoder_inputs, initial_state =  $[h_T, c_T]$ ) ;
8 :    $attention \leftarrow$  MultiheadAttention(num_heads=4, key_dim=units) ;
9 :    $attention\_output \leftarrow$  attention(query=decoder_outputs, value=encoder_outputs,
    key=encoder_outputs) ;
10 :   $combined \leftarrow$  concatenation(decoder_outputs, attention_output) ;
11 :   $combined \leftarrow$  Normalisation( $combined$ ) ;
12 :   $output \leftarrow$  reconstruction de  $combined$  par TimeDistributed(Dense( $D$ )) ;
13 :   $\mathcal{M} \leftarrow$  Model( $encoder\_inputs, output$ ) ;
14 :  Compiler  $\mathcal{M}$  avec l'optimiseur Adam et la perte MSE ;
15 :  return  $\mathcal{M}$  ;
16 : end function
```

Require : Données d'entraînement $\mathcal{D}_{\text{train}}$, longueur de séquence $input_steps$, longueur de séquence de sortie $forecast_steps$, dimension d'entrée $input_dim$, unités LSTM $units$

Ensure : Modèle $\mathcal{M}$ entraîné

Début

```
17 :  $X \leftarrow$  création de séquences à partir de  $\mathcal{D}_{\text{train}}$  avec une fenêtre de taille  $input\_steps$  ;
18 : // indice_debut_X = i
19 :  $Y \leftarrow$  création de séquences à partir de  $\mathcal{D}_{\text{train}}$  avec une fenêtre de taille  $forecast\_steps$  ;
20 : // indice_debut_Y = indice_debut_X + forecast_steps
21 :  $\mathcal{M} \leftarrow$  INITIALISATION_LSTM_MULTIHEAD_ATTENTION( $input\_steps, forecast\_steps,$ 
     $\dim(X), units$ ) ;
22 : Initialiser EarlyStopping avec patience 20 et restauration des meilleurs poids ;
23 : Entraîner  $\mathcal{M}$  sur  $(X, Y)$  avec 10% de validation,  $E$  époques et batch de taille  $B$ , sans shuffle ;
Fin
```

4.3.2 Détection

La détection d'anomalies pourrait être perçue comme une simple tâche de classification binaire (anomalie ou pas). Pourtant, cette vision est un peu réductrice. Dans notre approche, le modèle *Autoencodeur* basé sur les *LSTM* ne cherche pas à classer les données de manière directe ; il tente avant tout de reproduire fidèlement les séquences qu'on lui soumet, en s'appuyant sur ce qu'il a appris du comportement habituel des données saines.

Ce que le modèle apprend, c'est d'effectuer une transformation de séquence à séquence, appelée une transduction. Ce n'est qu'après cette étape, en comparant la séquence d'entrée avec sa

reconstruction, que l'on peut identifier les écarts significatifs et ainsi déduire la présence éventuelle d'anomalies. La décision finale (normale ou anormale) repose donc sur cette reconstruction, mais l'apprentissage en lui-même ne porte pas sur la classification. Cela distingue clairement notre approche d'un modèle de classification classique, puisque notre objectif principal est la modélisation et la restitution du comportement normal, non la prédiction directe d'étiquettes.

Concernant le choix du modèle, nous avons volontairement écarté les architectures basées sur les transformateurs pour la détection. Bien que puissants et à la pointe dans de nombreuses applications de traitement séquentiel, les transformateurs sont souvent réservés à des tâches plus complexes, comme la prédiction multivariée à long terme, où leur capacité à capturer les dépendances globales devient un réel avantage. En revanche, dans les scénarios de détection d'anomalies, où l'on cherche principalement à modéliser fidèlement un comportement « normal » sur des fenêtres relativement courtes, les autoencodeurs LSTM restent une référence robuste, éprouvée et mieux adaptée (au moins à l'heure actuelle).

Les LSTM sont également plus légers en ressources que les Transformateurs, ce qui facilite leur entraînement et leur intégration dans un pipeline industriel en temps réel. De plus, ils ont montré une stabilité plus grande en reconstruction sur nos données, tout en permettant une interprétation plus directe des erreurs temporelles. Ainsi, le choix de l'autoencodeur LSTM s'est imposé naturellement pour répondre aux contraintes de notre contexte industriel tout en garantissant une efficacité optimale.

4.3.2.1 Autoencodeur BiLSTM

Dans cette version autoencodeur du *LSTM*, le principe fondamental est de transformer une séquence d'entrée en une représentation latente condensée à l'aide d'un encodeur de deux couches : la première avec une couche LSTM unidirectionnelle, la seconde avec une couche *BiLSTM* ; Ce bloc encodeur est suivi d'un décodeur constitué d'une couche LSTM, chargé de reconstruire la séquence d'origine à partir de la représentation latente.

Contrairement au modèle de prédiction (cf. §4.3.1.4), qui apprend à projeter une séquence sur un futur proche ($X \rightarrow Y$), ici, le réseau est entièrement tourné vers la restitution du passé ($X \rightarrow X$).

L'idée est simple mais puissante : si le modèle a uniquement vu des comportements normaux pendant son entraînement, il devient naturellement très bon pour les reproduire, mais échoue dès que les motifs changent, comme c'est souvent le cas lors d'un incident ou d'un dysfonctionnement. Cette propriété en fait une approche particulièrement adaptée aux environnements où les anomalies sont rares ou mal identifiées, comme est souvent le cas dans les systèmes industriels.

Tout d'abord, l'encodeur reçoit une séquence d'entrée de forme (`input_steps=32, input_dim=22`) représentant des séries temporelles multivariées, issues des résultats de notre modèle de prédiction. Il commence par une couche LSTM(`units=128, return_sequences=True`) qui extrait des représentations temporelles à chaque pas de temps. Ensuite, une couche Bidirectionnel(LSTM(`units=128`), `return_sequences=False`) est appliquée. Cette couche contient deux sous-couches LSTM : une qui lit la séquence dans l'ordre normal (`forward`), autrement dit, dans l'ordre chronologique et une autre qui la lit en sens inverse (`backward`). Elle ne retourne qu'un seul vecteur par séquence (car `return_sequences=False`), obtenu en concaténant les derniers états cachés `forward` et `backward`. C'est ce vecteur compressé qui résume toute la séquence d'entrée.

Le vecteur de contexte est donc directement la sortie de la couche Bidirectionnelle(LSTM) avec `return_sequences=False`. Il correspond à la concaténation des deux derniers états cachés des

LSTM forward et backward (h_T^f, h_T^b). Ce vecteur, de taille $2 \times \text{units}$, capture une représentation globale de la séquence en prenant en compte les dépendances temporelles dans les deux directions.

Ce vecteur de contexte est répété autant de fois que `input_steps=32` grâce à la couche Repeat-Vector. Cela permet de créer une séquence constante en entrée du décodeur.

Cette séquence est ensuite traitée par une couche LSTM(`units=128, return_sequences=True`) qui constitue notre décodeur, tentant de reconstruire dynamiquement la séquence d'origine pas à pas. Enfin, une couche TimeDistributed(Dense(`input_dim=22`)) est appliquée pour reconstruire les valeurs originales à chaque pas de temps, avec la même dimension que l'entrée. Le modèle est entraîné pour minimiser l'erreur quadratique moyenne (MSE) entre la séquence d'entrée et sa reconstruction.

Nous avons opté pour une couche bidirectionnelle, car, contrairement à la prédiction où le respect de la chronologie est crucial, ici, l'objectif est de reconstruire fidèlement la séquence d'entrée. Dans ce contexte, comprendre la séquence globalement (dans les deux sens) peut aider le modèle à mieux capter les motifs. De plus, cette même couche supplémentaire nous évite de tomber dans une architecture trop simple. Cela permet au modèle de construire des représentations intermédiaires plus riches avant d'arriver au goulot d'étranglement (le vecteur de contexte).

Algorithm 7 Entraînement BiLSTM Autoencoder pour détection d'anomalies

```

1 : function INITIALIZE_BiLSTM_AUTOENCODER
Require : Longueur de séquence  $L$ , dimension d'entrée  $D$ , nombre d'unités LSTM  $U$ 
Ensure : Modèle  $\mathcal{M}$  initialisé
2 :   Début
3 :      $\text{inputs} \leftarrow$  séquence d'entrée de forme  $(L, D)$ ;
4 :      $x \leftarrow$  LSTM( $U$ , return_sequences=True) appliqué à  $\text{inputs}$ ;
5 :      $x \leftarrow$  Bidirectional(LSTM( $U$ )) appliqué à  $x$  ▷ Encodeur BiLSTM (bottleneck);
6 :      $x \leftarrow$  RepeatVector( $L$ ) ▷ Reproduction du vecteur de contexte;
7 :      $x \leftarrow$  LSTM( $U$ , return_sequences=True) appliqué à  $x$ ;
8 :      $\text{outputs} \leftarrow$  TimeDistributed(Dense( $D$ )) appliqué à  $x$ ;
9 :      $\mathcal{M} \leftarrow$  Model( $\text{inputs}, \text{outputs}$ );
10 :    Compiler  $\mathcal{M}$  avec l'optimiseur Adam et la perte MSE;
11 :    return  $\mathcal{M}$ 
12 : end function
Require : Données d'entraînement  $\mathcal{D}_{\text{train}}$ , taille de fenêtre  $L$ 
Ensure : Modèle entraîné  $\mathcal{M}$ 
   Début
13 :  $X \leftarrow$  Création de séquences depuis  $\mathcal{D}_{\text{train}}$  de forme  $(L, D)$ ;
14 :  $\mathcal{M} \leftarrow$  INITIALIZE_BiLSTM_AUTOENCODER( $L, D, U$ );
15 : Initialiser EarlyStopping avec patience 20 et restauration des meilleurs poids;
16 : Entraîner  $\mathcal{M}$  sur  $(X, X)$  avec 10% de validation,  $E$  époques et batch de taille  $B$ ;
   Fin

```

4.4 Conclusion

Dans ce chapitre, nous avons présenté les modèles candidats pour la prédiction et la détection des anomalies dans les séries temporelles issues de l'unité RFCC. Six modèles ont été décrits pour la tâche de prédiction multivariée, ainsi qu'un modèle basé sur un autoencodeur pour la détection.

Ces choix s'appuient sur leurs propriétés théoriques et leur pertinence pour la modélisation séquentielle. Dans le chapitre suivant, consacré à la réalisation, nous procéderons à leur implémentation, entraînement et évaluation comparative sur les données réelles. Cette analyse expérimentale nous permettra de sélectionner le modèle le plus performant pour la prédiction dans notre contexte industriel.

Chapitre 5 : Réalisation

5.1 Introduction

Dans ce chapitre, nous détaillons les outils utilisés, les environnements d'exécution, l'architecture de l'application développée, ainsi que le processus d'intégration de notre application au système de Sonatrach. Nous présentons aussi la comparaison des performances des modèles du chapitre précédent ainsi qu'une évaluation finale de notre pipeline.

5.2 Réalisation de notre approche

Après avoir défini une stratégie méthodologique rigoureuse et sélectionné les modèles d'apprentissage appropriés, nous avons entamé la phase de mise en œuvre de notre solution. Cette phase a nécessité l'intégration d'outils de traitement de données, de frameworks de modélisation avancés ainsi que de solutions modernes pour le développement d'interfaces utilisateur interactives. L'ensemble de ces composants a été orchestré dans le but de produire une application robuste et évolutive, capable de répondre aux contraintes spécifiques du secteur énergétique et industriel.

5.2.1 Outils utilisés

La mise en œuvre de notre approche repose sur un ensemble d'outils technologiques reconnus pour leur efficacité dans les domaines de l'intelligence artificielle et du développement logiciel.

5.2.1.1 Python

Python est un langage de programmation interprété, polyvalent, orienté objet et dynamique. Il est largement utilisé dans tout ce qui est du data science, de l'intelligence artificielle et de l'ingénierie logicielle pour sa syntaxe claire, sa flexibilité et sa richesse en terme de librairie[23].

5.2.1.2 TensorFlow

TensorFlow est une bibliothèque Google open-source pour le calcul numérique qui se base sur des graphes de flux de données. Elle est particulièrement adaptée à l'apprentissage profond (deep learning) et prend en charge des architectures complexes comme les réseaux de neurones convolutifs (CNN), récurrents (RNN/LSTM), ainsi que le traitement distribué sur GPU [24].

5.2.1.3 scikit-learn

scikit-learn est aussi une bibliothèque open-source conçue pour l'apprentissage automatique en Python. Elle fournit des outils efficaces pour le prétraitement des données, la modélisation statis-

tique(régression, classification, réduction de dimensionnalité...) et l'évaluation de modèles à l'aide de métriques standards [25].

5.2.1.4 Conda

Conda est un gestionnaire open-source d'environnements et de paquets , qui sert à installer, exécuter et mettre à jour des bibliothèques dans des environnements isolés [26].

5.3 Environnement matériel

Le prétraitement des données, l'entraînement et l'évaluation des modèles de prédiction et de détection d'anomalies ont été réalisés à l'aide de la machine GIGABYTE présente ci-dessous, qui a des performances similaires à celles de la raffinerie.

De plus, nous avons utilisé une autre machine HP pour les mêmes buts, mais l'exécution finale a été réalisée sur la première machine pour des raisons de performance.

PC	SE	CPU	GPU	Mémoire GPU	RAM
HP	Windows	Intel Core i7-13700H, 2.40–2.60 GHz	Nvidia RTX A500	4 GO	32 GO
GIGABYTE	Windows	Intel Core i5-12500H, 3.20–3.50 GHz	Nvidia RTX 4050	6 GO	16 GO

TABLE 5.1 – Environnement matériel utilisé

5.4 Implémentation de notre approche

5.4.1 Technologies utilisées

Ci-dessous sont présentées les principales technologies employées pour le développement de notre solution, chacune choisie pour sa fiabilité, sa performance et son adéquation aux exigences des systèmes modernes distribués.

5.4.1.1 FastAPI

FastAPI est un framework web open-source pour la création d'API RESTful avec Python et très connu pour sa rapidité. Il repose sur le standard ASGI et intègre la gestion automatique de la documentation via OpenAPI [27].

5.4.1.2 Uvicorn

Uvicorn est un serveur web léger et performant, conçu pour exécuter des applications Python asynchrones. Compatible avec le standard ASGI, il utilise des outils optimisés comme uvloop (pour accélérer la gestion des tâches) et httptools (pour traiter rapidement les requêtes HTTP), ce qui lui permet d'offrir des performances très élevées [28].

5.4.1.3 ReactJS

ReactJS est une bibliothèque JavaScript développée par Meta (anciennement Facebook) pour la création d'interfaces utilisateur dynamiques et modulaires. Elle repose sur un système de composants réutilisables et un DOM virtuel performant [29].

5.4.1.4 Git

Git est un système de contrôle de version distribué qui permet de suivre les modifications apportées au code source d'un projet, de faciliter la collaboration en équipe et de maintenir l'historique des changements [30].

5.4.2 Architecture logicielle

La solution que nous avons développée repose sur une architecture logicielle bien structurée, articulée autour de deux composants principaux : un **backend** et un **frontend**.

Le **backend**, développé à l'aide du framework *FastAPI*, est responsable du traitement des données d'entrée (une séquence temporelle sous format CSV), de l'exécution des modèles d'inférence (prédiction et détection d'anomalies), ainsi que de l'envoi des résultats à l'interface utilisateur (sous format JSON).

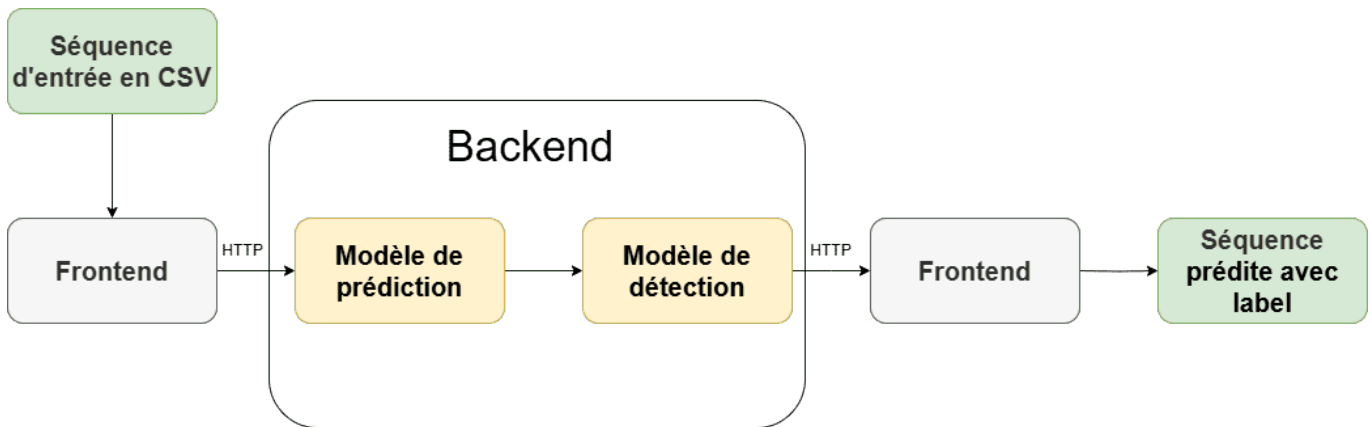


FIGURE 5.1 – Le pipeline de l'application.

Le **frontend**, conçu avec *ReactJS*, représente l'interface utilisateur de l'application. Il permet aux utilisateurs d'importer le fichier d'entrée, le visualiser, inspecter les prédictions générées, les anomalies détectées, ainsi que des statistiques descriptives. Une attention particulière a été portée à l'ergonomie de l'interface afin d'offrir une expérience fluide, intuitive et interactive, notamment grâce à l'intégration de visualisations graphiques dynamiques (graphiques linéaires avec zoom et surlignage des anomalies).

La communication entre ces deux composants s'effectue via des appels HTTP, selon une architecture de type client-serveur. Ce découplage entre backend et frontend présente de nombreux avantages : il permet une meilleure maintenabilité du code, facilite l'évolutivité de la solution (ajout de fonctionnalités ou remplacement de composants) et autorise un déploiement distribué dans des environnements hétérogènes.

5.4.3 Fonctionnalités de l'application

L'application développée offre une interface intuitive et moderne permettant d'exploiter la puissance de modèles prédictifs et de détection d'anomalies appliqués à des séries temporelles industrielles. Elle se compose d'un tableau de bord interactif accessible via une interface web développée avec ReactJS, connectée à un backend FastAPI assurant les traitements de données et l'inférence des modèles.

Les fonctionnalités principales de l'application sont les suivantes :

- **Téléversement de fichiers CSV** : l'utilisateur peut charger un fichier contenant une séquence historique via un champ de sélection de fichier. Le nom du fichier est ensuite affiché pour confirmation. Ce CSV est extrait par l'ingénieur manuellement depuis la base de données de l'unité RFCC via une requête SQL.
- **Aperçu des données d'entrée** : après téléversement, la séquence d'entrée est présentée dans un tableau stylisé, permettant de vérifier visuellement les colonnes, les valeurs ainsi que les timestamps. Cela garantit que les données soient correctement reconnues par le système.
- **Prévision du futur comportement** : une fois le fichier traité, l'application affiche les résultats des prévisions produites par le modèle de prédiction. Les valeurs prévues sont affichées dans un tableau comportant les horodatages et les mesures pour chaque variable ainsi qu'un indicateur binaire de statut (Normal ou Anomalie).
- **Téléchargement des résultats** : un bouton permet de télécharger la séquence prévisionnelle avec les flags d'anomalies au format CSV, facilitant l'analyse ultérieure ou l'archivage des résultats.
- **Visualisation graphique personnalisée** : une section dédiée permet à l'utilisateur de choisir une variable à visualiser sous forme de courbe. Les anomalies détectées sont mises en évidence en rouge. Un outil de zoom et de balayage temporel permet une inspection précise du signal.
- **Statistiques globales** : des statistiques descriptives (moyenne, minimum, maximum) sont automatiquement calculées sur la séquence prévisionnelle pour chaque variable. Celles-ci sont présentées dans un tableau clair et consultable horizontalement.
- **Ergonomie moderne et responsive** : l'application adopte une charte graphique sombre avec des accents néon pour renforcer la lisibilité, offrir un aspect professionnel et s'adapter à différents types d'écrans (desktop, laptop, etc.). Ce choix esthétique est également motivé par des considérations de santé : selon les recommandations d'un médecin du travail, les ingénieurs et techniciens en milieu industriel sont souvent exposés à des interfaces à fond clair de manière prolongée, ce qui peut engendrer plusieurs troubles visuels tels que la fatigue oculaire, les migraines, la sécheresse des yeux et une diminution de la concentration. L'utilisation d'un thème sombre contribue ainsi à améliorer le confort visuel, en réduisant la lumière bleue émise par l'écran et en limitant l'éblouissement lors d'une utilisation continue.

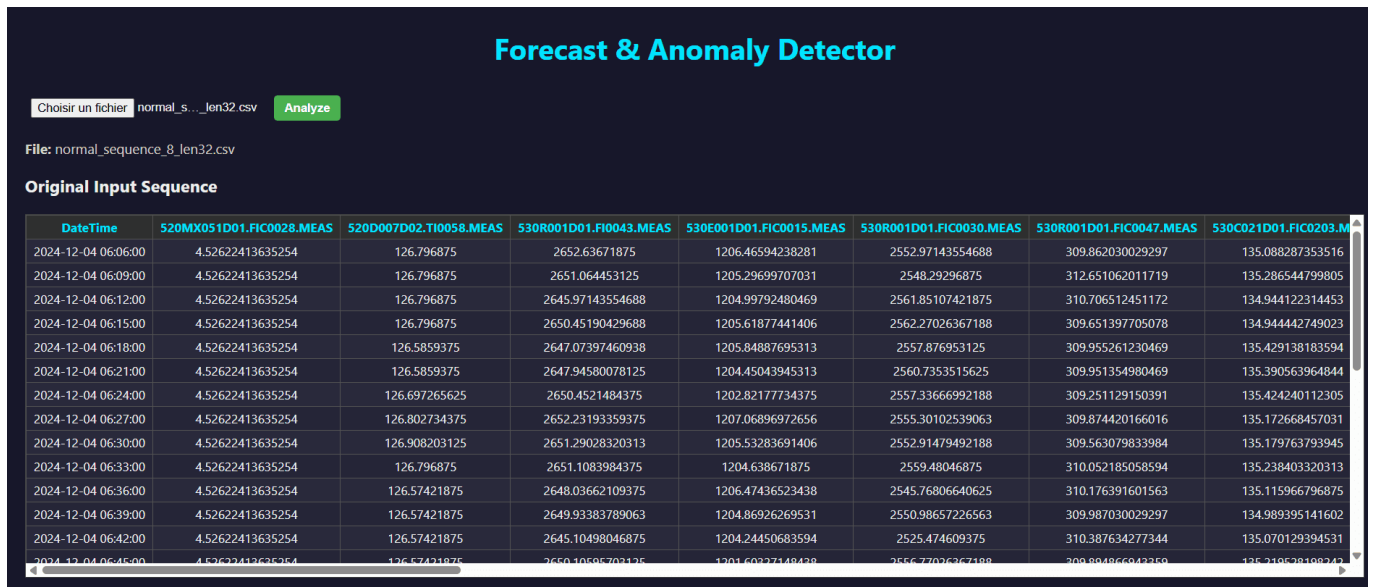


FIGURE 5.2 – La séquence historique introduite dans l’interface de l’application.

La figure 5.2 illustre l’aperçu de la séquence passée telle qu’elle est affichée à l’utilisateur après téléversement du fichier CSV. Elle permet une vérification immédiate de la structure des données avant le traitement.

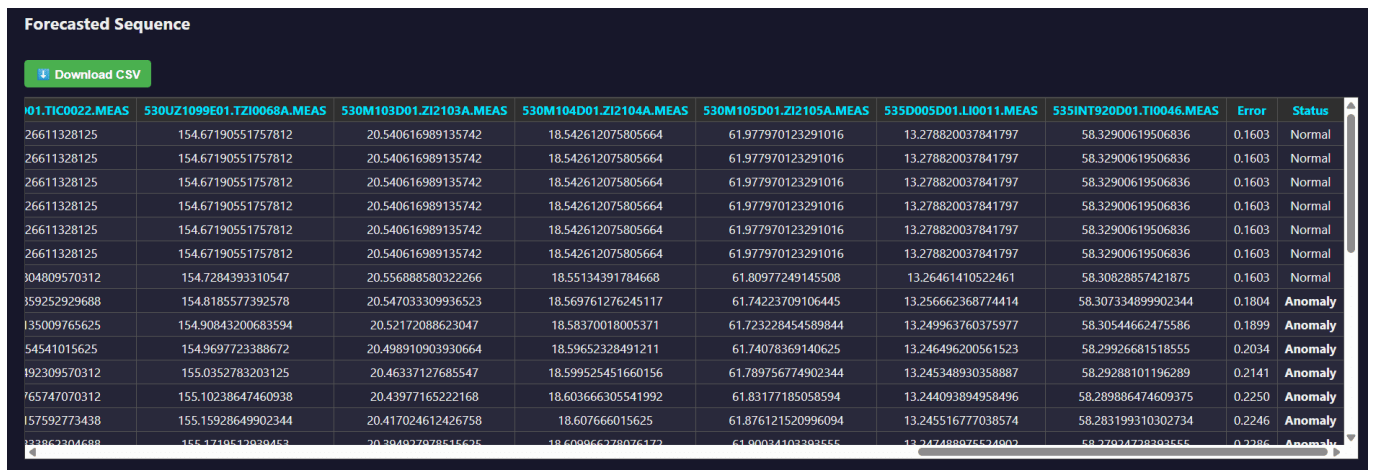


FIGURE 5.3 – La séquence prédite résultante dans l’interface de l’application.

La figure 5.3 présente un tableau qui regroupe les valeurs prédites par le modèle, accompagnées des statuts (Normal ou Anomalie), facilitant ainsi l’interprétation rapide des résultats.



FIGURE 5.4 – Les statistiques de la séquence prédite dans l’interface de l’application.

La figure 5.4 montre le graphe de valeurs prédites pour un attribut choisi avec les labels ainsi qu’un résumé statistique (moyenne, minimum, maximum) de chaque variable prédite, permettant d’obtenir une vue d’ensemble synthétique du comportement du système analysé.

Ces fonctionnalités ont été conçues en étroite cohérence avec les besoins réels de surveillance industrielle, notamment dans des contextes critiques comme ceux des installations de Sonatrach. L’accent a été mis sur l’automatisation, la simplicité d’utilisation, la lisibilité des résultats et la capacité d’intégration dans un environnement industriel existant.

5.4.4 Déploiement

Le déploiement de notre solution au sein de l’unité RFCC de Sonatrach a nécessité une approche prudente et conforme aux contraintes de sécurité en vigueur. En effet, en tant que stagiaires, nous n’avons pas été autorisés à intégrer directement notre application dans l’environnement informatique de l’unité, celle-ci étant classée comme infrastructure critique pour la sécurité nationale. Toute intervention sur ce système est strictement réglementée et nécessite des autorisations spécifiques, réservées aux ingénieurs accrédités.

Initialement, nous avons développé un prototype complet intégrant un flux Kafka pour assurer la transmission des données en temps réel. Ce prototype a été présenté à l’équipe technique de l’unité, accompagné d’une requête formelle pour procéder à des tests d’intégration. Cependant, pour des raisons de conformité avec la politique interne de cybersécurité, cette demande n’a pas été retenue.

Face à ces contraintes, une alternative a été validée en collaboration avec les ingénieurs responsables : nous avons livré les deux composants de l’application (le backend FastAPI et le frontend ReactJS) sous forme de solutions indépendantes prêtes à être exécutées dans leur environnement. L’intégration prévue repose sur l’interconnexion avec le système d’information existant via Kafka, permettant une alimentation automatique de l’application avec les séquences de données collectées en temps réel, sans recourir à des extractions manuelles via des requêtes SQL.

Afin de faciliter l’adoption et l’évolutivité de la solution, nous avons pris soin d’installer l’ensemble de l’environnement nécessaire à son bon fonctionnement (Python, Node.js, bibliothèques,

dépendances, etc.) et de documenter toutes les étapes d'exécution et de modification. Ainsi, les ingénieurs de Sonatrach disposent non seulement d'une application fonctionnelle, mais aussi des moyens de la maintenir, de la faire évoluer ou de l'adapter à de nouveaux besoins de manière autonome.

5.5 Comparaisons des modèles de prédiction

À présent, nous allons tester nos six modèles. D'abord, pour chaque architecture, nous explorerons plusieurs configurations afin de dénicher celle qui donne les meilleurs résultats (voir Annexe). Ensuite, munis de ces versions optimales, nous mettrons tous les modèles face à face pour voir lequel excelle le plus dans la prédiction de nos séries temporelles.

Les figures 5.5 à 5.10 présentent les courbes de Training Loss et Validation Loss :

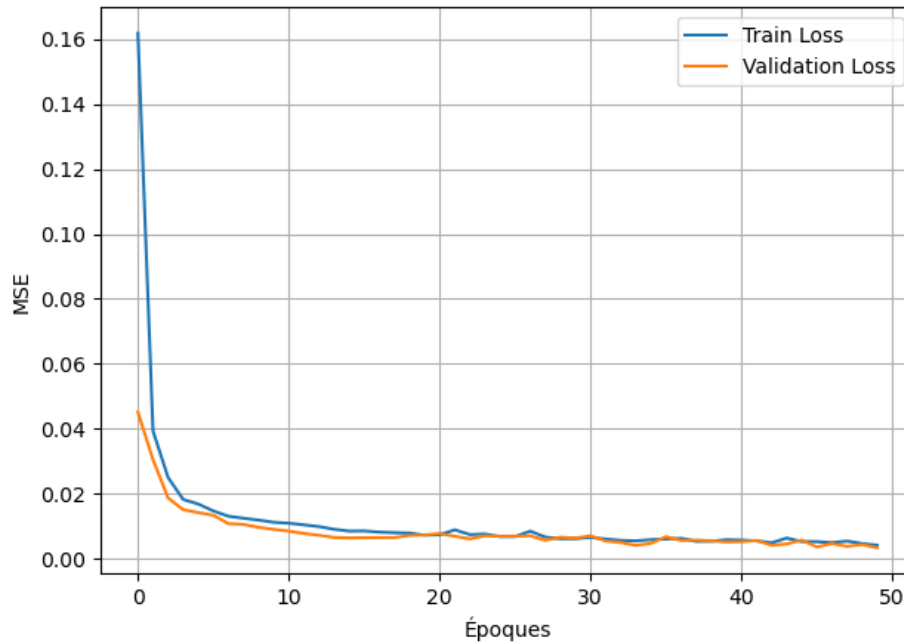


FIGURE 5.5 – Courbe de perte – LSTM seq2seq

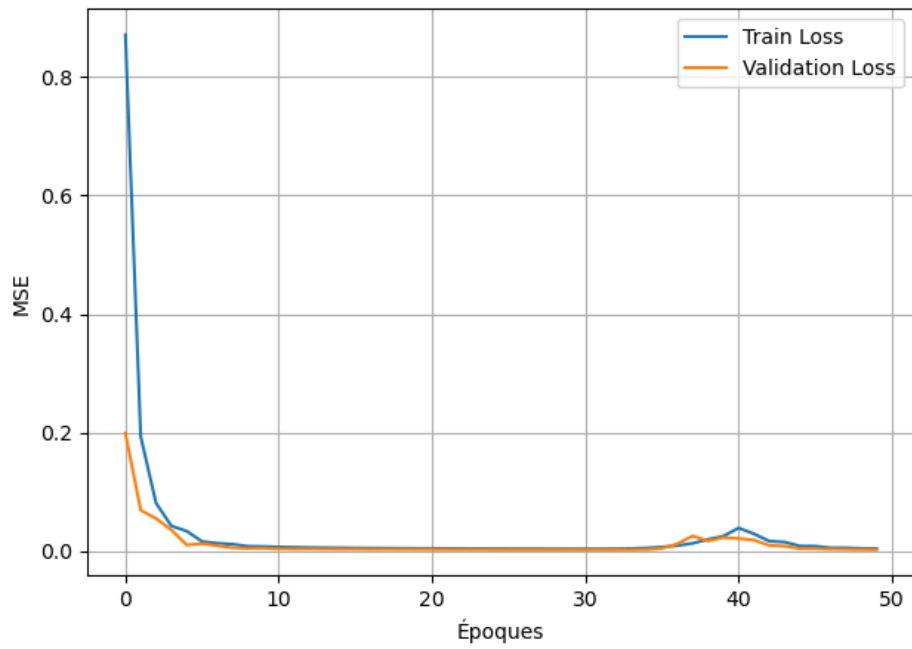


FIGURE 5.6 – Courbe de perte – LSTM Attention multi-tête

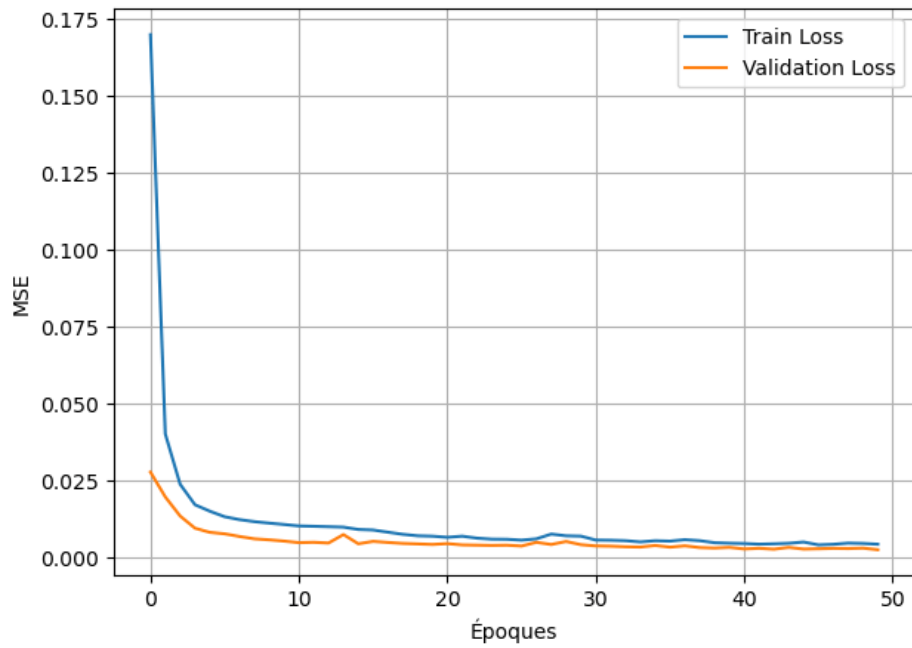


FIGURE 5.7 – Courbe de perte – Informeurs

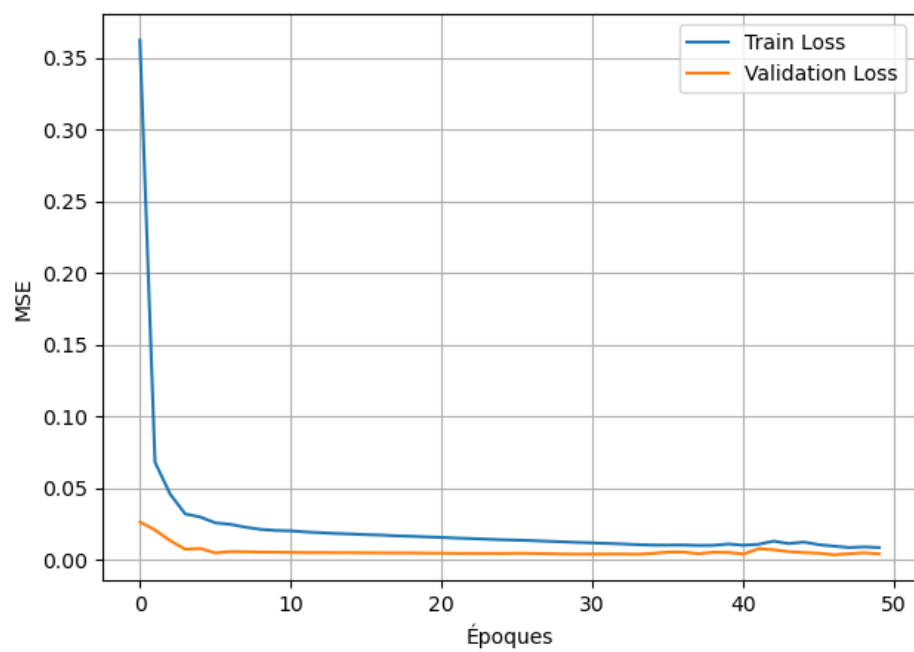


FIGURE 5.8 – Courbe de perte – Transformateurs

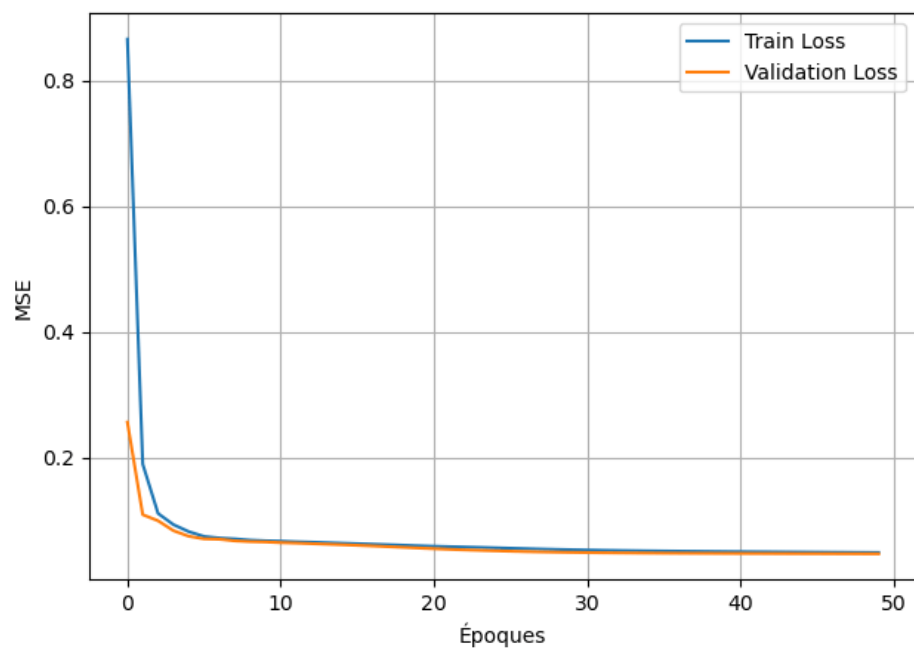


FIGURE 5.9 – Courbe de perte – PatchTST (standard)

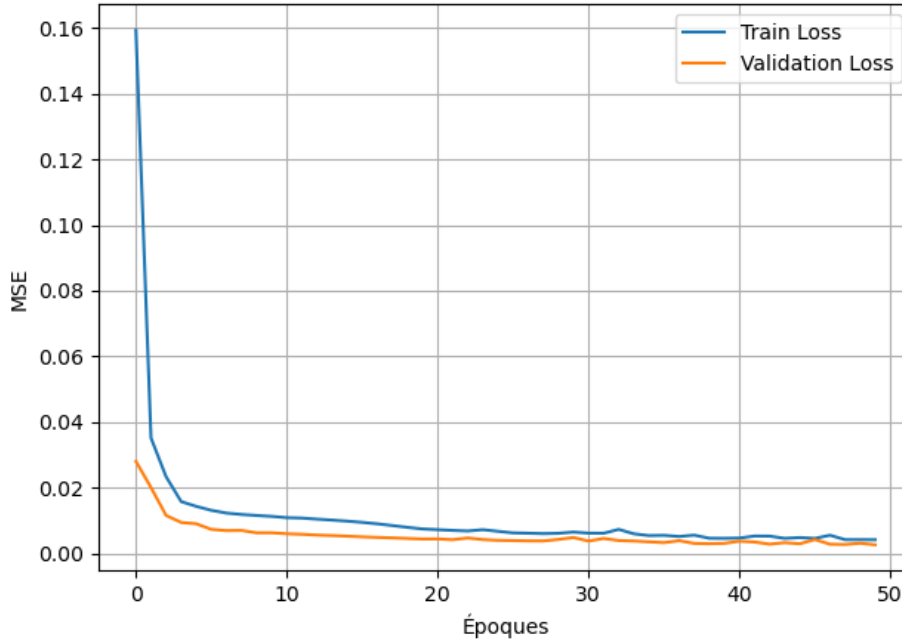


FIGURE 5.10 – Courbe de perte – PatchTST (adapté)

L'ensemble des modèles montre une convergence relativement rapide dès les premières époques. Cela témoigne de leur capacité à s'adapter aux données industrielles utilisées. On observe cependant quelques phénomènes intéressants :

- Pour plusieurs modèles (notamment pour les modèles basés transformateurs), la validation loss est parfois inférieure à la loss d'entraînement. Ce phénomène peut sembler contre-intuitif mais il n'est pas nécessairement problématique. Il peut s'expliquer par une régularisation implicite ou une structure de données de validation plus homogène que le training set.
- L'absence de surapprentissage est globalement confirmée par la stabilité des courbes sur les 50 époques.
- Les pertes se stabilisent à des niveaux différents selon les modèles, ce qui correspond aux scores de MSE/MAE reportés dans le tableau de comparaison.

Pour évaluer la qualité des prédictions, nous avons utilisé deux métriques standard dans le domaine des séries temporelles multivariées :

- **Mean Squared Error (MSE)** : c'est la moyenne des carrés des erreurs entre les valeurs réelles et les valeurs prédites. Le MSE est toujours positif et plus il est proche de zéro, mieux le modèle prédit. Cette métrique pénalise fortement les grandes erreurs, ce qui la rend sensible aux écarts importants.

$$\text{MSE} = \frac{1}{N} \sum_{n=1}^N (y_n - \hat{y}_n)^2$$

avec :

- N : nombre total d'observations,
- y_n : valeur réelle pour l'observation n ,
- $\hat{y}_n$: valeur prédite pour l'observation n .

- **Mean Absolute Error (MAE)** : c'est la moyenne des valeurs absolues des erreurs entre les valeurs réelles et prédites. Moins sensible aux valeurs extrêmes que le MSE, le MAE donne une estimation plus linéaire de l'erreur moyenne et est souvent plus interprétable dans des contextes industriels.

$$\text{MAE} = \frac{1}{N} \sum_{n=1}^N |y_n - \hat{y}_n|$$

avec :

- N : nombre total d'observations,
- y_n : valeur réelle pour l'observation n ,
- $\hat{y}_n$: valeur prédite pour l'observation n ,
- $|x|$: valeur absolue de x .

Ces deux métriques sont largement utilisées dans les travaux récents de l'état de l'art en prévision de séries temporelles, notamment dans les articles introduisant les modèles *PatchTST* [21] et *Informer* [31], ce qui nous permet également de comparer nos résultats dans un cadre académique cohérent.

Modèle	Validation MSE	Validation MAE	Temps (s)	Complexité Spatiale (Big-O)
LSTM seq2seq	0.003224	0.036543	9.9	$O(u^2 + u \cdot id)$
LSTM+Attention	0.004267	0.046870	11.1	$O(u^2 + u \cdot id + nh \cdot u^2)$
Transformeur	0.005926	0.056789	8.7	$O(ed^2 + ed \cdot ffd + sl \cdot ed)$
Informeur	0.003800	0.041697	17.3	$O(ed^2 + ed \cdot id + k \cdot sl)$
PatchTST	0.051289	0.184532	13.2	$O(np \cdot pl \cdot ed + nl \cdot ed^2)$
PatchTST perso.	0.003849	0.042100	11.1	$O(np \cdot pl \cdot ed + ed^2)$

TABLE 5.2 – Résultats du tuning des hyperparamètres des modèles de prédiction

u : unités LSTM, id : input_dim, nh : num_heads, ed : embed_dim, ffd : ff_dim, sl : sequence_length, np : num_patches, pl : patch_len, nl : num_layers, k : top_k (ProbSparse)

Remarque sur la complexité et le temps d'exécution : Dans notre contexte d'inférence sur des séries temporelles, ni la complexité spatiale ni le temps d'exécution à l'entraînement ne constituent une contrainte. En effet, en production, l'entrée ne sera composée que d'une seule séquence de 32 pas de temps, ce qui rend le temps d'inférence négligeable, même pour les modèles complexes. C'est pourquoi nous avons volontairement choisi de ne pas mentionner la complexité temporelle et nous avons utilisé uniquement la complexité spatiale à titre indicatif. Cette dernière aussi n'est pas discriminante dans notre cas, car la taille mémoire des modèles reste acceptable sur les GPU actuels.

Par ailleurs, nous avons conservé une taille de séquence de 32 afin de maximiser le nombre d'itérations d'entraînement. Augmenter cette taille réduirait fortement le nombre de fenêtres possibles dans les séries et donc la quantité totale de données d'entraînement.

Globalement, les architectures LSTM se sont montrées plus performantes sur notre cas d'usage. Le modèle *LSTM seq2seq* a obtenu les meilleurs résultats en termes de MSE, tandis que le *LSTM-MultiheadAttention* a offert un compromis intéressant entre complexité, stabilité et précision.

Cependant, notre choix final ne s’est pas appuyé uniquement sur les scores de validation (MSE et MAE). Nous avons également évalué la capacité des modèles à restituer fidèlement la distribution statistique des données réelles, en observant notamment la variance, les amplitudes des signaux prédits, ainsi que leur comportement visuel par rapport aux séries temporelles d’origine. Le modèle LSTM-MultiheadAttention a su mieux préserver cette cohérence globale, en évitant les prédictions trop lissées ou stationnaires. Ce critère est fondamental dans un contexte industriel, où la qualité de la forme des signaux est aussi critique que leur exactitude numérique.

Le Transformateur et l’Informer, bien qu’efficaces dans les benchmarks classiques, ont montré des performances légèrement inférieures dans notre contexte industriel, probablement en raison de la taille relativement modeste de notre jeu de données d’entraînement (80% de la fenêtre, soit environ 24 000 lignes), ainsi que de la longueur limitée des séquences (32 pas en entrée et 32 en sortie).

Concernant *PatchTST*, la version de base a donné des résultats moins satisfaisants. Cela peut s’expliquer par le fait qu’il est initialement conçu pour des prévisions à long terme sur de très gros volumes de données. En revanche, après personnalisation (élimination du concept Channel independence et l’ajout d’un décodeur basé LSTM), le modèle a atteint d’excellents scores, comparables aux LSTM, ce qui prouve qu’il peut être adapté à des contextes contraints, sous réserve de le reconfigurer intelligemment.

Ainsi, bien que les modèles Transformateur-based soient puissants, ce sont les architectures récurrentes (en particulier les variantes LSTM enrichies d’un mécanisme d’attention) qui se sont révélées les plus adaptées à notre cas d’étude, tant sur le plan des métriques que sur celui de la fidélité aux signaux physiques du procédé.

Il est d’ailleurs important de souligner que de bonnes performances en MSE ou MAE ne garantissent pas nécessairement la qualité des prédictions en pratique. Ces métriques peuvent parfois être trompeuses, notamment dans le cas de modèles ayant tendance à lisser les séquences ou à produire des moyennes, ce qui minimise mathématiquement l’erreur, mais ne reflète pas fidèlement la dynamique réelle du système. C’est pourquoi l’analyse visuelle et la cohérence statistique des prédictions ont été des critères essentiels dans notre processus de sélection.

Les figures suivantes montrent, pour chaque modèle évalué, un exemple de prédiction sur un seul attribut, en comparant la séquence réelle (en bleu) et la séquence prédite (en orange). Bien que tous ces modèles affichent de bons scores MSE et MAE, ces courbes révèlent des différences importantes au niveau de la distribution, des décalages de valeur et du comportement temporel.

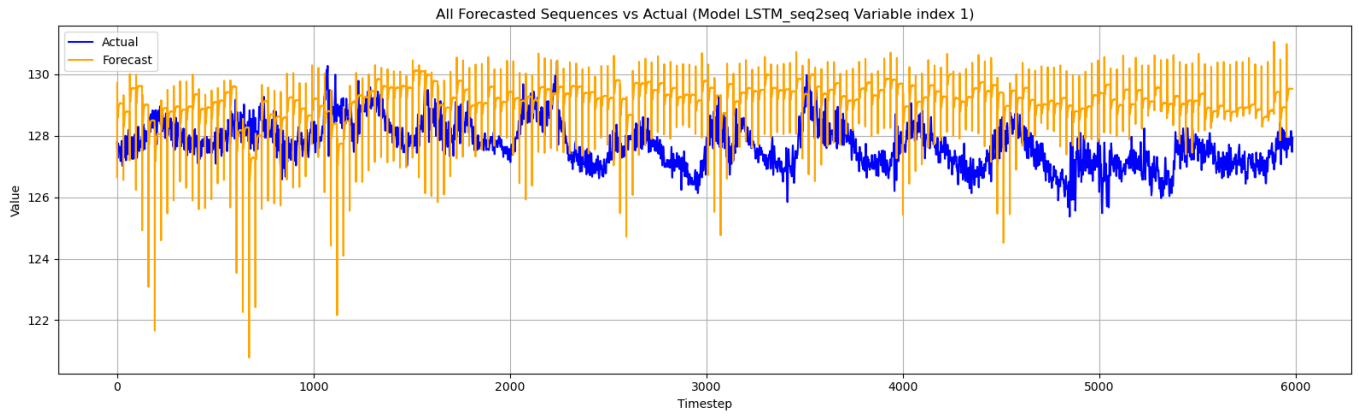


FIGURE 5.11 – Prévision réelle vs prédite modèle LSTM seq2seq

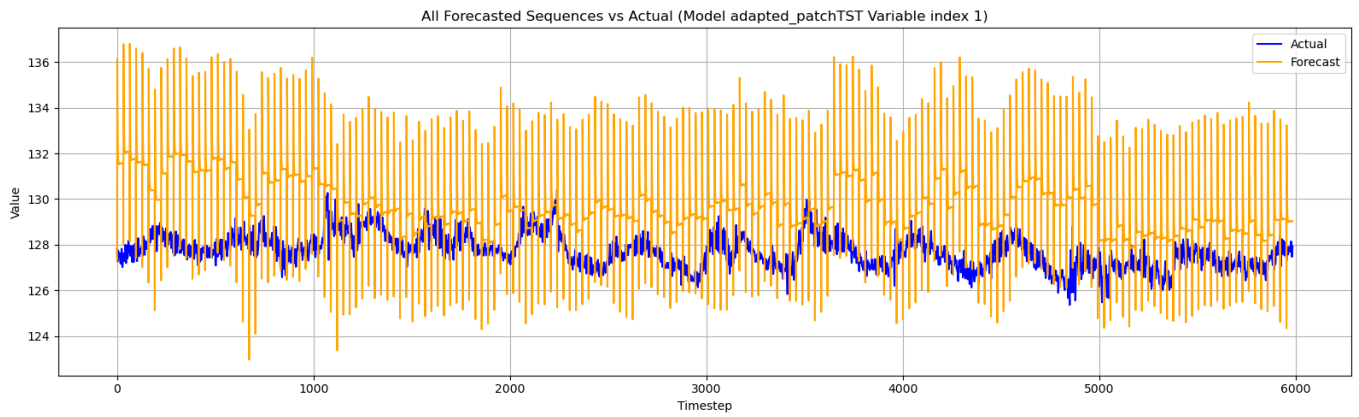


FIGURE 5.12 – Prévision réelle vs prédite modèle PatchTST adapté

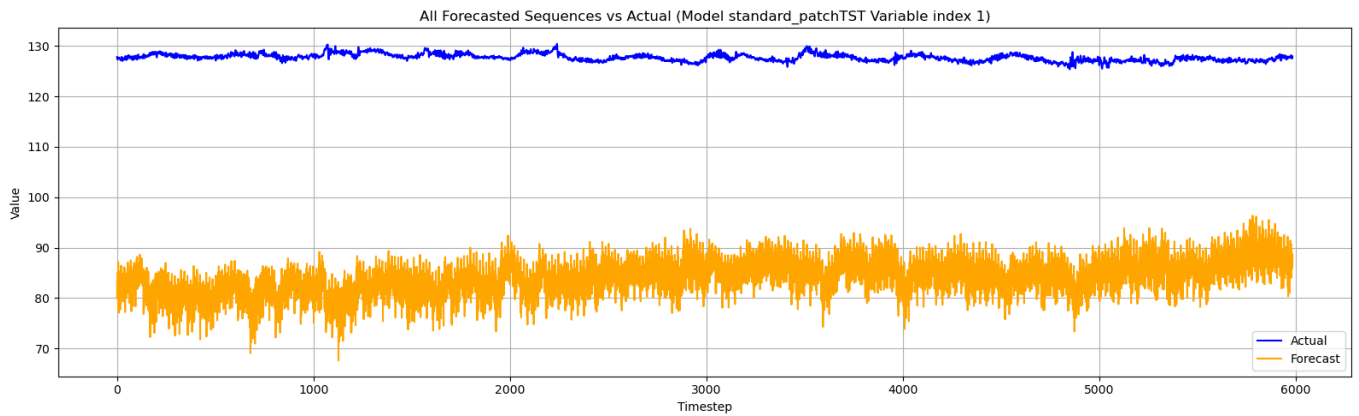


FIGURE 5.13 – Prévision réelle vs prédite modèle PatchTST standard

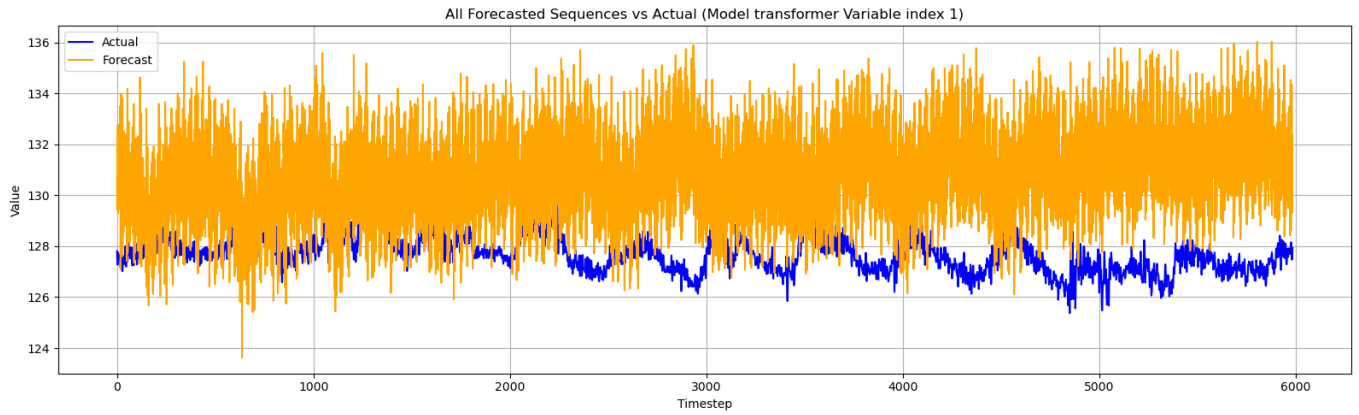


FIGURE 5.14 – Pr vision r elle vs pr dite du mod le Transformateur

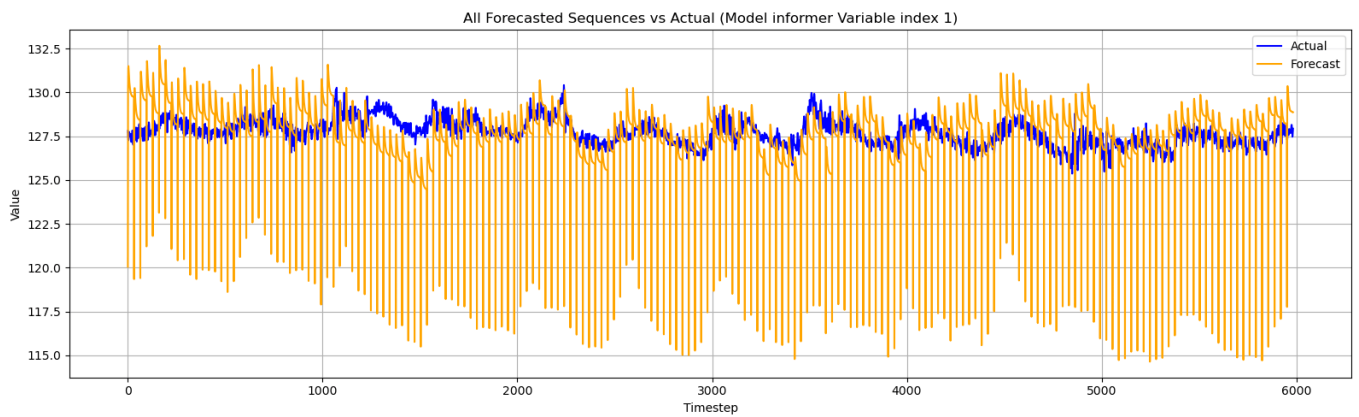


FIGURE 5.15 – Pr vision r elle vs pr dite du mod le Informeur

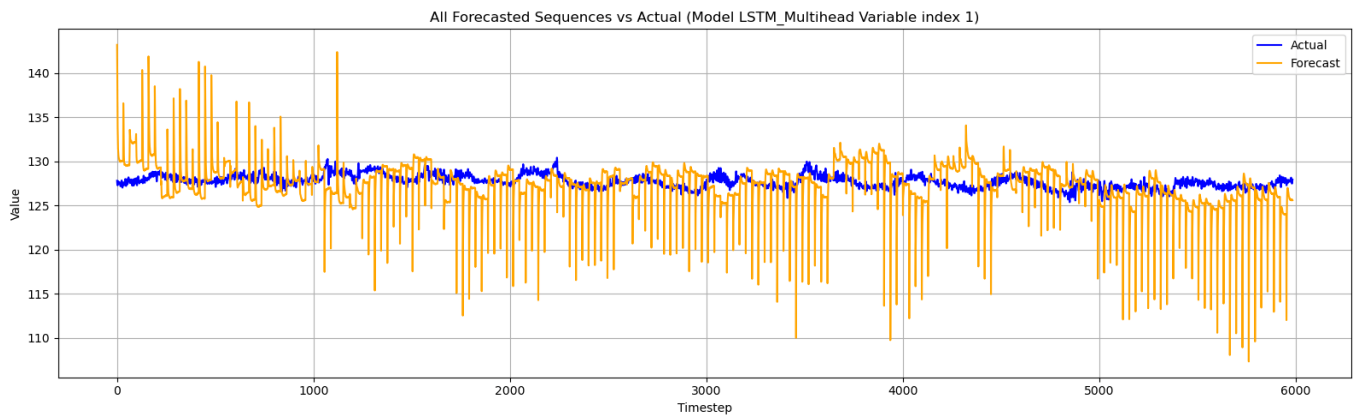


FIGURE 5.16 – Pr vision r elle vs pr dite du mod le LSTM Attention Multi-t te

Une observation attentive des courbes des figures 5.11   5.16 met en  vidence les points suivants :

- Ce type d'instabilit  au d but des s quences est courant dans les mod les de pr vision, en particulier ceux utilisant une architecture   encodeur. Le mod le manque de contexte au

démarrage, ce qui entraîne des fluctuations brusques sur les premières valeurs prédites. Pour corriger cela en production, un lissage est appliqué : la sixième valeur est assignée aux cinq premières, ce qui supprime les pics initiaux sans altérer la dynamique globale.

- Certains modèles comme **PatchTST** ou **Transformateur** produisent des prédictions qui s'écartent fortement en termes de niveau ou d'amplitude, malgré des MSE acceptables. La distribution des valeurs prédites ne reflète pas fidèlement la dynamique réelle du signal.
- D'autres modèles, comme **Informer** ou **LSTM seq2seq**, arrivent à capturer la forme globale, mais souffrent de décalages constants (biais en valeur), ce qui pénalise la précision métier.
- Le modèle **PatchTST adapté** améliore partiellement la distribution, mais présente encore une forte dispersion, ce qui le rend instable dans les environnements industriels.
- Enfin, le **LSTM Multihead Attention** se distingue nettement. Il parvient non seulement à respecter l'allure globale du signal, mais aussi à rester dans une plage cohérente de valeurs. Sa capacité à suivre la distribution réelle, tout en limitant les écarts ponctuels, justifie son intégration dans notre pipeline final.

Ce type de visualisation est crucial pour compléter les métriques classiques comme le MSE ou le MAE. En effet, un bon score agrégé peut masquer des biais systématiques, une perte de variance, ou des erreurs sur des attributs spécifiques. D'où la nécessité d'une validation visuelle avant le déploiement industriel.

5.6 Validation du modèle de détection

Afin de s'assurer de la stabilité et de l'efficacité du modèle *Autoencodeur BiLSTM* destiné à la détection d'anomalies, une analyse de son comportement durant l'entraînement a été menée. Le tableau ci-dessous montre que le modèle converge correctement, avec une perte sur l'ensemble de validation (0.001532) très proche de celle de l'entraînement (0.001814), ce qui indique une bonne capacité de généralisation et l'absence de surapprentissage.

Modèle	Loss validation	Loss entraînement	Temps d'exécution	Complexité spatiale
Autoencodeur BiLSTM	0.0015	0.0018	17.4 s	$\mathcal{O}(u^2 + ud + u + d)$

TABLE 5.3 – Performances du modèle Autoencodeur BiLSTM après ajustement des hyperparamètres

La figure 5.17 montre l'évolution de la fonction de perte (MSE) pendant l'entraînement du modèle Autoencodeur basé sur une architecture LSTM. On observe une diminution rapide de la perte sur l'ensemble d'entraînement (Train Loss) au cours des premières époques, suivie d'une convergence progressive.

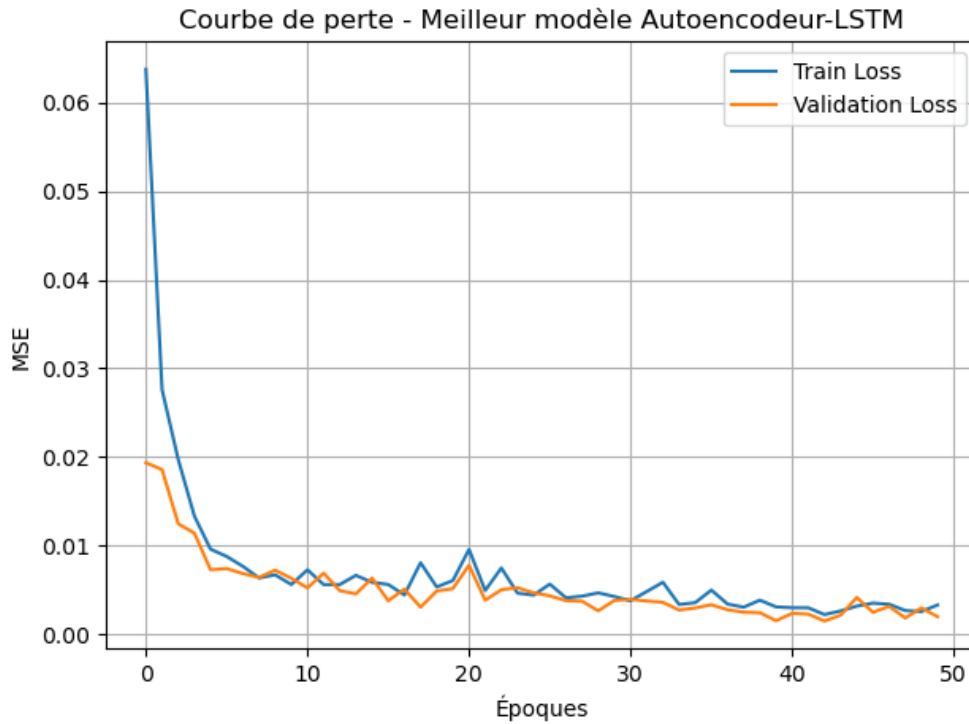


FIGURE 5.17 – Courbe d’apprentissage du modèle Autoencodeur BiLSTM pour la détection d’anomalies.

La figure 5.17 représente la courbe de perte sur les données de validation (Validation Loss) qui suit une tendance très proche de celle de l’entraînement, ce qui indique que le modèle ne souffre ni de surapprentissage (overfitting), ni de sous-apprentissage (underfitting). La faible valeur de perte en fin d’entraînement, ainsi que la stabilité des deux courbes, témoignent de la capacité du modèle à généraliser correctement à des données non vues.

Cette stabilité est essentielle pour un système de détection d’anomalies, où le moindre surapprentissage pourrait conduire à des fausses alertes, ou à l’inverse, à des anomalies non détectées.

La performance réelle du modèle a été testée sur un jeu de données séparé, labellisé manuellement par les ingénieurs procédés. Pour évaluer sa qualité, trois métriques issues de la classification binaire ont été utilisées :

- **Précision (Precision)** : c’est le rapport entre le nombre d’instances correctement identifiées comme anomalies et le nombre total d’instances prédites comme anomalies. Cette métrique indique la capacité du modèle à éviter les faux positifs. Plus la précision est élevée, plus le modèle est fiable lorsqu’il signale une anomalie.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

- **Rappel (Recall)** : c’est le rapport entre le nombre d’anomalies correctement détectées et le nombre total d’anomalies réellement présentes dans les données. Le rappel mesure la capacité du modèle à détecter les vraies anomalies (éviter les faux négatifs). Il est particulièrement important dans les contextes industriels, où il vaut mieux générer quelques fausses alertes que de manquer une vraie anomalie.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

- **F1-score** : c’est la moyenne harmonique entre la précision et le rappel. Il fournit un compromis équilibré entre ces deux métriques, en tenant compte à la fois des faux positifs et des faux négatifs.

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Les résultats obtenus sur le jeu de test sont les suivants :

- **Précision** : 0.7273
- **Rappel** : 0.8889
- **F1-score** : 0.8000

Ces métriques sont standard dans la littérature pour les systèmes de détection d’événements rares ou critiques [32]. Dans le cadre d’un système de surveillance industrielle, la priorité est donnée au **Rappel**, car il vaut mieux générer quelques fausses alertes que manquer des anomalies réelles susceptibles d’endommager l’installation ou de compromettre la sécurité. Ce niveau élevé de rappel (88.89%) confirme la pertinence du modèle dans ce contexte.

5.7 Pipeline d’évaluation

Dans cette section, nous présentons le choix final des modèles à intégrer dans notre pipeline. Ce choix repose sur des critères de performance objectifs, adaptés à chaque tâche (prédiction et détection), ainsi que sur des règles métier spécifiques dans le cas de la détection.

4.3.3.1 Modèle de prédiction

Le modèle retenu pour la prédiction est le LSTM Attention multi-tête. Ce choix repose sur les métriques et les méthodes évoquées supra.

4.3.3.2 Modèle de détection

Le modèle utilisé pour la détection des anomalies est un Autoencodeur basé sur une architecture LSTM. Il a été sélectionné en fonction de ses performances en *Training loss* et en *Validation loss*.

En complément, cette approche a été enrichie par une logique métier spécifique : si la valeur du tag TI0037 dépasse 729 °C, alors la séquence correspondante est automatiquement considérée comme anormale, indépendamment de la sortie du modèle. Ce seuil a été fixé à 729, légèrement en dessous de la valeur critique réelle estimée à 730, dans une logique de prudence. L’objectif est d’anticiper au maximum l’apparition de conditions dangereuses, quitte à générer occasionnellement de fausses alertes. Cette marge préventive permet de garantir une réactivité accrue du système, essentielle dans un environnement industriel à haut risque.

Cette règle, construite en collaboration avec les ingénieurs procédés, permet de capturer des situations critiques non apprises par le modèle de détection. Il s’agit ici d’un mécanisme de sécurité fondé sur des seuils connus par expertise, visant à renforcer la robustesse et la fiabilité globale du système.

Par ailleurs, une autre étape a été intégrée dans le pipeline avant la détection, visant à atténuer un biais introduit par la prédiction. En effet, les sorties du modèle de forecast ont tendance à être

légèrement lissées, ce qui réduit artificiellement les variations naturelles présentes dans les données réelles. Pour corriger cela, nous avons appliqué une amplification des écarts autour de la moyenne, selon un facteur contrôlé.

Mathématiquement, pour chaque variable X , la transformation suivante a été appliquée :

$$X_{\text{amplifié}} = \mu_X + \min \left(\frac{\sigma_{\text{réel}}}{\sigma_{\text{forecast}}}, \alpha \right) \cdot (X - \mu_X)$$

où :

- μ_X est la moyenne de la variable dans les données forecastées,
- $\sigma_{\text{réel}}$ est l'écart-type de la variable dans les vraies données,
- σ_{forecast} est l'écart-type dans les données forecastées,
- α est un facteur limite d'amplification fixé empiriquement (dans notre cas à 6) pour éviter une surcorrection.

Cette correction permet de réaligner la distribution des données prédites avec celle du jeu réel, facilitant ainsi le travail du modèle de détection et évitant qu'il n'interprète à tort des séquences normales comme étant sans variation (et donc faussement normales). Elle améliore également la détection des pics discrets qui peuvent apparaître dans des situations anormales localisées.

En résumé, ce modèle de détection repose sur une combinaison harmonieuse entre apprentissage profond (autoencodeur LSTM), expertise métier (seuil fixe sur TI.0037) et ingénierie des données (amplification statistique), ce qui lui confère à la fois précision, flexibilité et robustesse dans le contexte complexe d'une raffinerie industrielle.

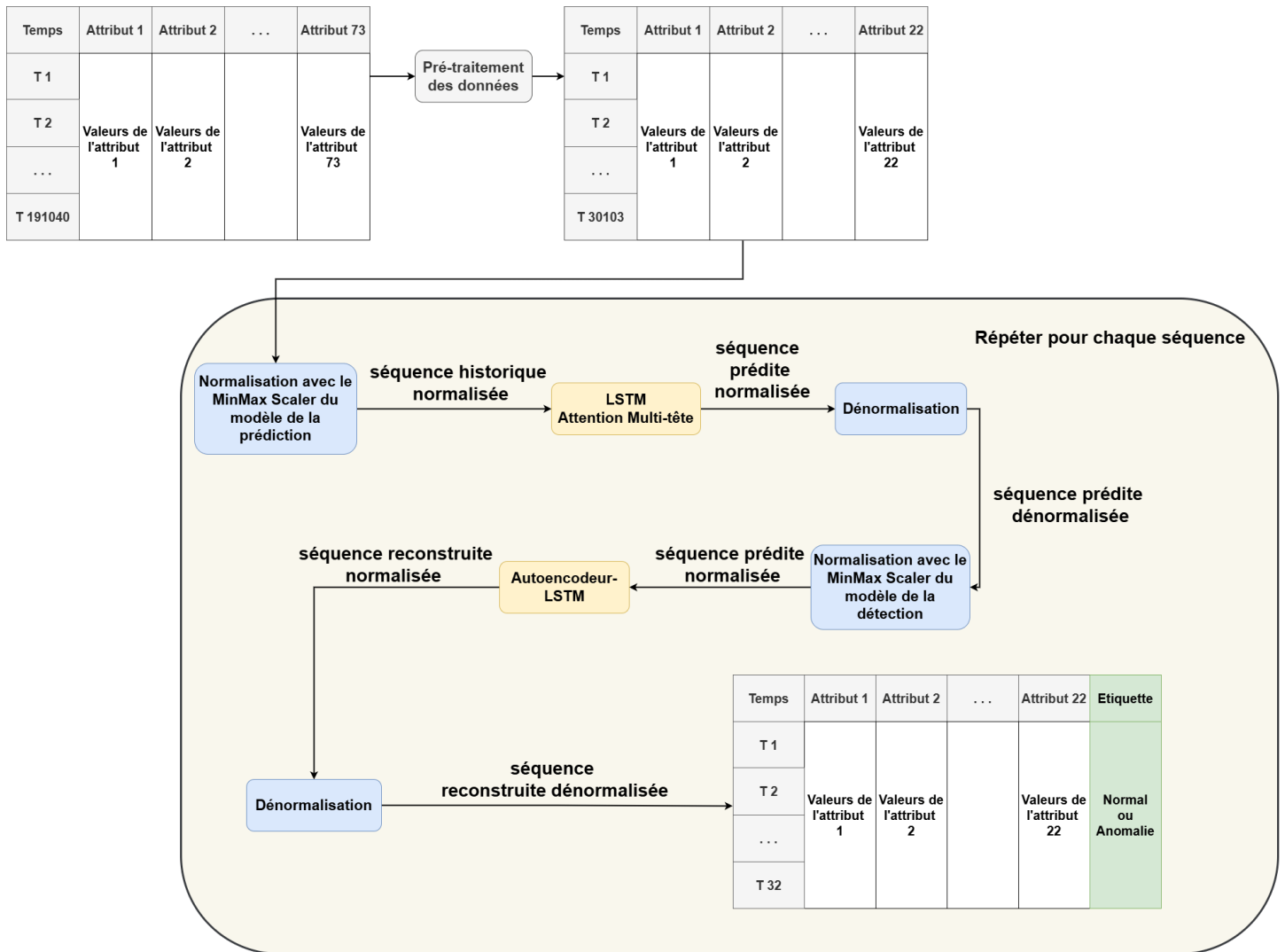


FIGURE 5.18 – Architecture finale de notre solution

5.7.1 Chargement des modèles et des données

Les modèles préalablement entraînés sont chargés, ainsi que les scalers correspondants nécessaires à la normalisation des données. Le fichier d'entrée contient les observations temporelles accompagnées des étiquettes d'anomalies (labels) données par les ingénieurs de procédés de Sonatrach. Ces labels ont été fournis dans le cadre d'un processus de labellisation particulièrement complexe, en raison de la nature industrielle des données.

En effet, plusieurs facteurs ont rendu ce travail difficile : la structure du dataset est très dense et peu intuitive, les seuils de fonctionnement normal ne sont ni constants ni clairement définis et le temps alloué aux ingénieurs pour effectuer cette tâche était limité. De plus, l'historique de pannes ou de dysfonctionnements n'est pas numérisé, mais existe uniquement sous forme de comptes rendus hebdomadaires manuscrits ou imprimés, ce qui a compliqué leur exploitation.

C'est dans ce contexte que les ingénieurs ont pu annoter environ 1408 lignes, en se basant principalement sur ces rapports d'exploitation, où des problèmes ont été signalés. Bien que partielle, cette annotation experte représente une base précieuse pour l'évaluation du modèle de détection.

5.7.2 Prédiction avec le modèle LSTM Attention multi-tête

Le modèle reçoit des séquences de longueur 32 (paramètre `INPUT_STEPS`) et prédit les 32 prochaines valeurs (`FORECAST_STEPS`).

6.2.2.1 Évaluation quantitative de la prédiction

Les sorties sont comparées aux véritables valeurs cibles. Les résultats suivants ont été obtenus avant dénormalisation :

- **MSE (Mean Squared Error)** : 0.00542
- **MAE (Mean Absolute Error)** : 0.05451

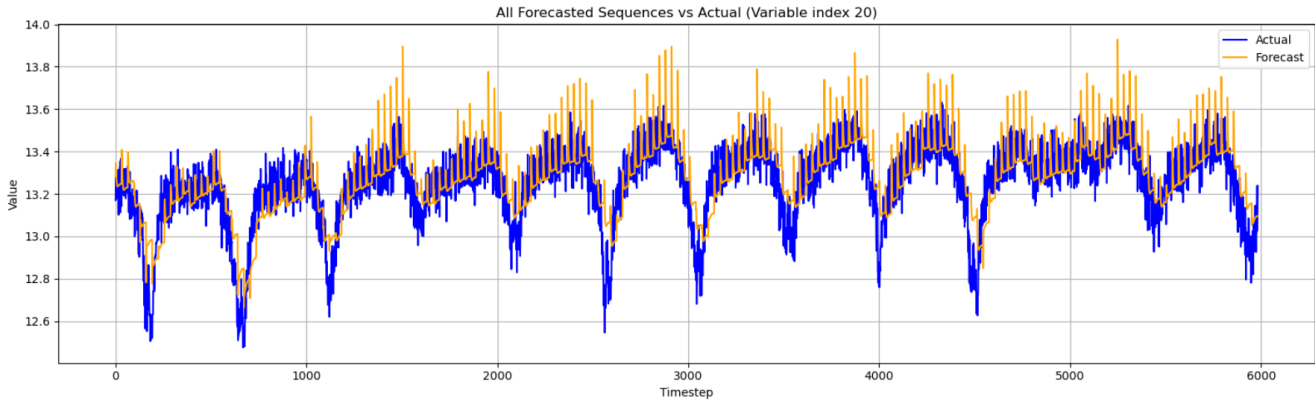


FIGURE 5.19 – Prédiction vs valeurs réelles sur l'attribut 20

Bien que les métriques MSE et MAE soient très satisfaisantes, la figure 5.19 révèle un léger décalage ponctuel entre les prédictions et les valeurs réelles, principalement au tout début des séquences. Ce comportement, typique des modèles séquentiels à encodeur, s'explique par le manque de contexte initial. En revanche, notre modèle parvient à capturer fidèlement la forme générale et la distribution des données, ce qui témoigne de sa capacité à apprendre les dynamiques sous-jacentes du procédé. Pour renforcer encore la stabilité des sorties en production, un lissage est appliqué sur les cinq premières valeurs de chaque séquence, en les remplaçant par la sixième prédite. Cette stratégie permet d'éliminer les fluctuations initiales sans compromettre la cohérence globale du signal.

5.7.3 Détection d'anomalies avec l'Autoencodeur BiLSTM

Les prédictions du modèle de forecast sont dénormalisées puis injectées dans le modèle d'auto-encodeur. Celui-ci reconstruit les séquences d'entrée et calcule une erreur de reconstruction par point, reflétant l'écart entre les données d'entrée et leur version reconstruite.

6.2.3.1 Définition du seuil d'anomalie

Un seuil fixe $T = 0.178$ a été choisi à partir d'une analyse de la courbe ROC et d'une comparaison manuelle de plusieurs seuils afin d'obtenir une bonne précision, en utilisant le seuil suggéré par la courbe ROC comme point de départ (0.156006). Ce seuil permet de distinguer les pas de temps normaux de ceux suspectés comme anormaux.

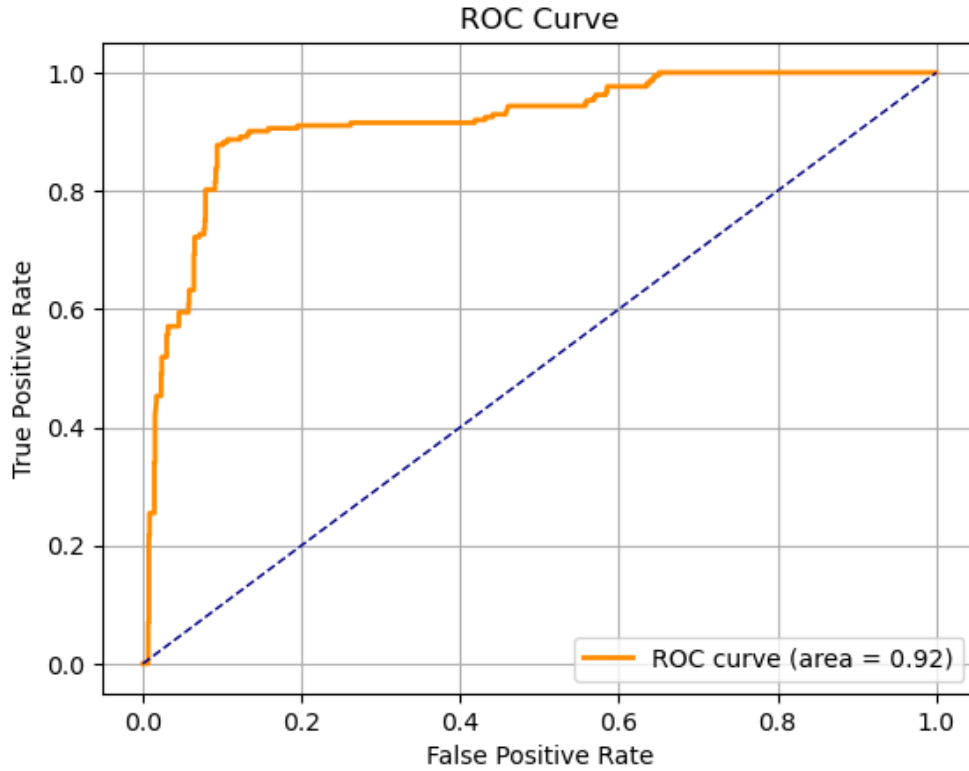


FIGURE 5.20 – Courbe ROC du modèle de détection (AUC = 0.92)

6.2.3.2 Visualisation des erreurs de reconstruction

Choix de la mesure d'erreur Pour calculer l'erreur de reconstruction dans notre modèle autoencodeur, nous avons utilisé la formule suivante :

$$\text{Erreur}(t) = \max_j |x_j(t) - y_j(t)|$$

où $x_j(t)$ est la valeur réelle de l'attribut j à l'instant t et $y_j(t)$ sa reconstruction par le modèle.

Contrairement à la MSE (erreur quadratique moyenne), cette métrique repose sur l'écart absolu maximal entre les attributs à chaque pas de temps. Elle est particulièrement adaptée dans notre contexte industriel, car une seule valeur aberrante dans un seul capteur suffit à indiquer une déviation anormale du procédé. Cette mesure met ainsi davantage en évidence les anomalies localisées et critiques que la MSE, qui a tendance à les diluer dans une moyenne globale.

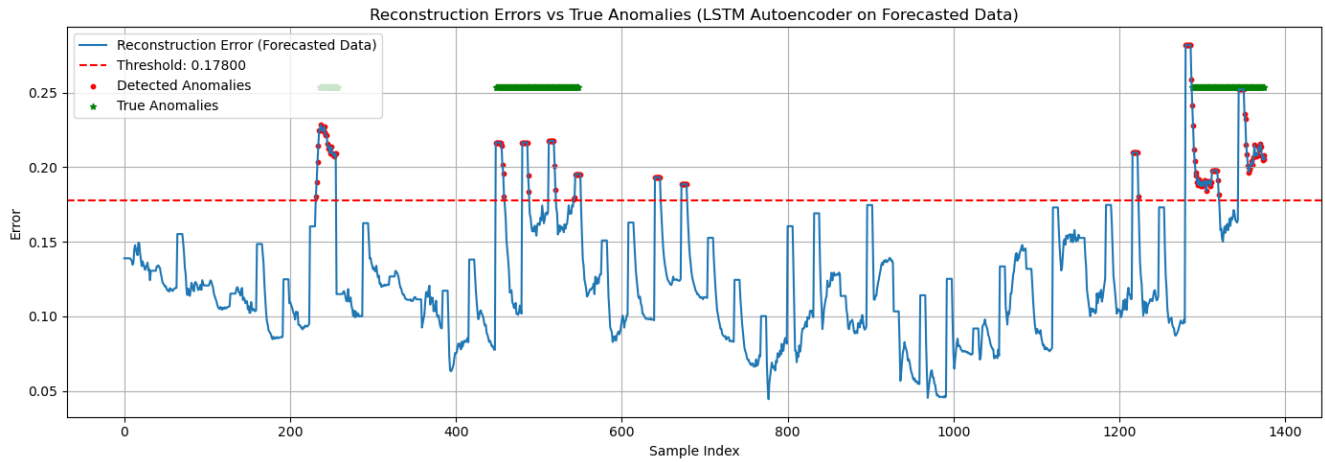


FIGURE 5.21 – Erreur de reconstruction vs anomalies réelles

La figure 5.21 confirme que les pics d'erreur dépassant le seuil coïncident bien avec les anomalies réelles identifiées. Le modèle ne souffre pas de sous-détection et démontre une capacité remarquable à anticiper les fenêtres anormales, en détectant certaines séquences légèrement avant les intervalles annotés par les ingénieurs. Plus impressionnant encore, le modèle signale plusieurs anomalies non identifiées dans les étiquettes d'origine. Cette performance s'explique par la qualité parfois incomplète du processus de labellisation, réalisé manuellement.

6.2.3.3 Évaluation de la détection

- **Précision** : 0.7273
- **Rappel** : 0.8889
- **F1-score** : 0.8000

Ces résultats montrent un bon compromis entre fausses alertes et détections manquées. Le rappel élevé est particulièrement important dans un contexte industriel : il vaut mieux alerter trop souvent que rater une anomalie critique.

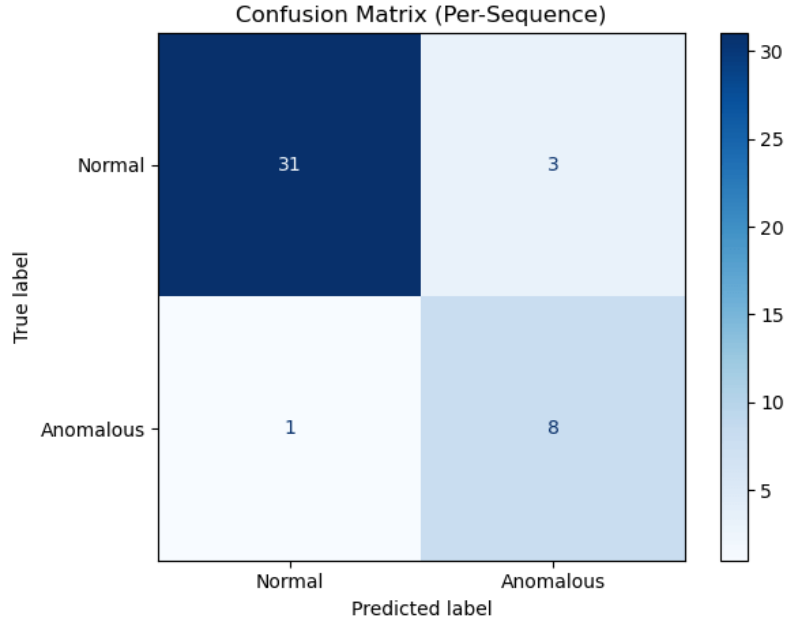


FIGURE 5.22 – Matrice de confusion (flagging par séquence)

La matrice de confusion confirme ces observations : seuls 3 faux positifs et 1 faux négatif ont été comptabilisés.

5.7.4 Analyse neutre par la déviation absolue moyenne

Étant donné que notre modèle de détection est non supervisé et que les étiquettes fournies par les ingénieurs procédés sont manuellement établies, nous avons mis en place une méthode neutre et objective pour évaluer la cohérence des anomalies détectées : la déviation absolue moyenne lissée.

Cette méthode repose sur le calcul de la déviation moyenne absolue (Mean Absolute Deviation) à chaque instant t , entre les valeurs des attributs et leur moyenne à ce même instant. La formule utilisée est la suivante :

$$\text{MAD}(t) = \frac{1}{n} \sum_{i=1}^n |x_i(t) - \mu(t)| \quad \text{où} \quad \mu(t) = \frac{1}{n} \sum_{i=1}^n x_i(t)$$

où $x_i(t)$ représente la valeur de l'attribut i au temps t et $\mu(t)$ est la moyenne de tous les attributs à cet instant. Cette série est ensuite lissée par une moyenne glissante de taille 32 (correspondant à la taille des séquences utilisées dans notre modèle).

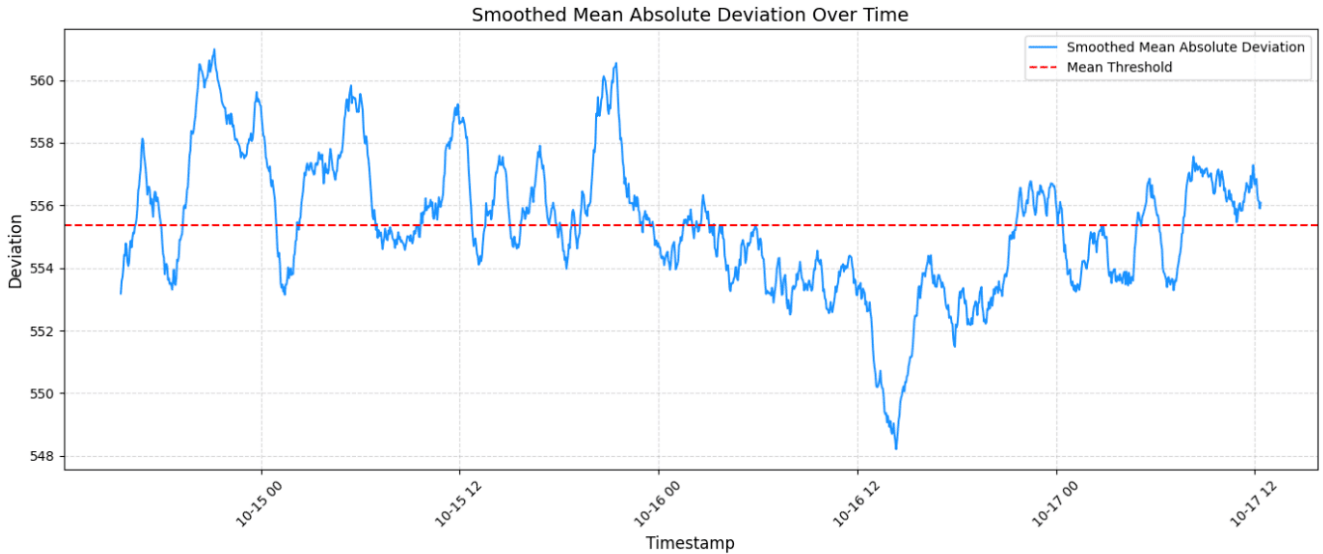


FIGURE 5.23 – Évolution de la déviation absolue moyenne lissée (fenêtre = 32)

La courbe de la figure 5.23 montre des pics significatifs de déviation qui coïncident plus ou moins avec les anomalies détectées par notre modèle. Bien que cela renforce la crédibilité de notre approche, il est important de souligner que la courbe de déviation des données n'est pas non plus une méthode d'évaluation parfaitement fiable à 100%, car elle repose uniquement sur des critères statistiques et ne tient pas compte du contexte métier ni de la complexité réelle du procédé.

5.8 Limites des approches causales et analyse des causes racines

Dans une tentative d'aller au-delà de la simple détection et prédiction, nous avons investi un temps conséquent en début de projet dans l'exploration de méthodes d'analyse causale et de *Root Cause Analysis* (RCA), bien que cela ne fasse pas partie des objectifs demandés. Cette démarche relevait d'une initiative personnelle, motivée par le désir de mieux comprendre les mécanismes internes du procédé et d'identifier les attributs à l'origine des anomalies. Toutefois, ces expérimentations se sont révélées peu fructueuses dans notre contexte spécifique et ont soulevé plusieurs limitations structurelles.

Pour l'analyse des causes racines, nous avons testé différentes approches centrées sur l'identification des attributs ayant le plus contribué aux anomalies détectées, en nous basant notamment sur les fluctuations brutales, les variations locales anormales, ainsi que sur l'analyse de l'erreur de reconstruction générée par le modèle autoencodeur. L'idée était de repérer les variables dont la reconstruction échouait systématiquement en présence d'anomalies, ce qui pourrait indiquer leur responsabilité dans la dégradation du comportement global.

Si ces méthodes permettaient effectivement de mettre en évidence des attributs fortement impliqués dans les séquences anormales, les résultats étaient toutefois systématiquement biaisés vers des variables représentant des vannes ou actions de contrôle directement manipulées par les opérateurs. Ces attributs, bien qu'ils présentent des signaux spectaculaires (pics, oscillations, ruptures), sont en réalité des conséquences visibles des anomalies ; souvent des réponses humaines ou automatiques du système et non leurs causes profondes. Cela limite fortement la pertinence des diagnostics proposés et souligne la difficulté d'isoler une « cause première » sans interprétation métier.

Du côté de l'extraction de relations causales, nous avons exploré un large éventail de techniques allant de méthodes statistiques et probabilistes (comme Granger causality, PCMCI) à des approches plus avancées de type graph neural networks et modèles causaux basés sur le deep learning. Cependant, ces méthodes ont toutes échoué à produire des graphes informatifs : en raison de la forte corrélation entre la majorité des attributs, les graphes générés présentaient un très grand nombre de liens peu discriminants, souvent avec des poids similaires. Cela rendait l'interprétation des résultats difficile, voire inutile dans la pratique.

Face à ces constats, nous avons jugé que l'analyse causale et l'identification automatisée des causes racines ne pouvaient pas être traitées de manière fiable sans une connaissance approfondie du fonctionnement chimique et opérationnel du procédé RFCC. Il apparaît clairement que cette tâche devrait faire l'objet d'un projet à part entière, mené par une équipe pluridisciplinaire incluant des experts en procédés pétrochimiques, instrumentation industrielle et en intelligence artificielle spécialisée dans la modélisation explicative.

Malgré le temps important investi dans cette direction, nous considérons cette tentative comme une expérience formatrice, qui nous a permis de mieux cerner les limites des méthodes d'IA sur des systèmes industriels complexes. Elle a également renforcé la pertinence de notre choix de recentrer les efforts sur la détection et la prédiction, où les résultats sont concrets, mesurables et exploitables par les ingénieurs de terrain.

5.9 Conclusion

Ce chapitre a présenté la concrétisation de notre approche sous la forme d'une application complète, alliant un backend performant et un frontend interactif, conçus pour répondre aux contraintes du secteur industriel. L'ensemble du système a été réalisé avec des technologies modernes, en veillant à garantir robustesse, modularité et facilité d'utilisation. Au-delà de la mise en œuvre, nous avons procédé à une évaluation comparative approfondie de plusieurs architectures de prédiction, mettant en évidence la supériorité du modèle LSTM Attention Multi-tête pour la prévision et de l'Autoencodeur BiLSTM pour la détection d'anomalies. Ces modèles ont été intégrés dans un pipeline final, testé avec succès sur des données annotées réelles. Les résultats obtenus, tant en termes de métriques que de cohérence visuelle, confirment la pertinence et la fiabilité de notre solution pour la surveillance intelligente des procédés industriels.

Conclusion générale

Ce mémoire a présenté une solution hybride et intelligente pour la prédiction et la détection d'anomalies dans les séries temporelles industrielles issues d'une unité RFCC (Régénération du Catalyseur en Flux Catalytique), avec un cas d'étude concret basé sur la raffinerie d'Alger. L'objectif principal était de construire un pipeline robuste et interprétable, capable de détecter des comportements anormaux critiques tout en s'adaptant à la complexité des données réelles et aux contraintes industrielles.

Le pipeline proposé repose sur une combinaison efficace entre deux modèles : un *LSTM Attention multi-tête* pour la prédiction des valeurs futures et un *Autoencodeur BiLSTM* pour la détection d'anomalies à partir de l'erreur de reconstruction. Ce système a été renforcé par une règle heuristique métier (seuil sur le capteur TI.0037), validée par les ingénieurs de procédés de Sonatrach.

Les résultats obtenus sont satisfaisants aussi bien du point de vue des métriques classiques (MSE, MAE, précision, recall, F1-score), que d'un point de vue visuel et fonctionnel. Notre modèle a su détecter les fenêtres anormales signalées par les ingénieurs, mais également anticiper certaines anomalies avant leur pic et même révéler des événements suspects qui n'avaient pas été initialement identifiés. Une méthode de validation neutre par calcul de la déviation moyenne absolue a permis de démontrer objectivement la pertinence de notre système.

Ce travail a également mis en lumière plusieurs difficultés, notamment la qualité imparfaite des labels, la forte corrélation entre attributs, ou encore les défis liés à l'explicabilité et à l'analyse des causes racines dans les systèmes complexes. Néanmoins, ces limites ont été contournées grâce à des choix méthodologiques judicieux, une adaptation des modèles à la nature des données et l'intégration d'une expertise métier.

Perspectives

Ce projet ouvre la voie à plusieurs perspectives d'amélioration et de valorisation :

- **Industrialisation** : une prochaine étape logique serait le déploiement de notre solution dans un environnement de production réel, via une architecture microservices avec streaming temps réel (Kafka), comme envisagé par l'équipe technique de Sonatrach.
- **Apprentissage actif** : pour pallier le manque de labels, une stratégie d'apprentissage actif pourrait être mise en place, où le modèle propose des fenêtres suspectes aux ingénieurs pour validation, permettant ainsi une amélioration continue du système.
- **Exploration des causes racines** : bien que cette tâche se soit révélée complexe dans notre cadre, une collaboration étroite avec des experts en procédés chimiques pourrait permettre d'exploiter les résultats de détection pour remonter aux origines des anomalies, en combinant intelligence artificielle et analyse dirigée par la connaissance du domaine.
- **Approches auto-supervisées** : l'utilisation de techniques récentes comme les *transformateur auto-supervisés*, ou la modélisation de séries multivariées avec graphes causaux dynamiques, pourrait permettre de renforcer encore davantage la détection précoce.

- **Généralisation** : le pipeline conçu peut être adapté à d'autres installations ou procédés industriels similaires, avec une personnalisation minimale, rendant cette solution réutilisable dans d'autres unités ou même dans d'autres secteurs.

En somme, ce mémoire illustre comment une approche centrée sur la donnée, intégrant les avancées récentes en deep learning tout en respectant les contraintes métier, peut déboucher sur des solutions concrètes et impactantes pour l'industrie 4.0.

Références

bibliographie

- [3] D.E. ADANENCHE, A. ALIYU, A.Y. ATTA et B.J. EL-YAKUBU. “Residue fluid catalytic cracking : A review on the mitigation strategies of metal poisoning of RFCC catalyst using metal passivators/traps”. In : *Fuel* 343 (2023). ISSN : 0016-2361. DOI : 10.1016/j.fuel.2023.127894. URL : <https://www.sciencedirect.com/science/article/pii/S0016236123005070>.
- [4] CPECC ET SONATRACH ACTIVITÉ LRP, PÔLE RAFFINAGE. “Manuel Opérateur – Unité de Craquage Catalytique du Fluide Résiduel”. In : (2020). Document technique interne, révision 4, 324 pages.
- [6] Varun CHANDOLA, Arindam BANERJEE et Vipin KUMAR. “Anomaly Detection : A Survey”. In : *ACM Comput. Surv.* 41 (juill. 2009). DOI : 10.1145/1541880.1541882.
- [7] M. HEJAZI et Y. P. SINGH. “One-Class Support Vector Machines Approach to Anomaly Detection”. In : *Applied Artificial Intelligence* 27.5 (2013). DOI : 10.1080/08839514.2013.785791.
- [8] Jiawei HAN et Micheline KAMBER. “Data Mining : Concepts and Techniques”. In : The Morgan Kaufmann Series in Data Management Systems (mars 2006).
- [9] Fei Tony LIU, Kai Ming TING et Zhi-Hua ZHOU. “Isolation forest”. In : (2008).
- [10] Tolga ERGEN et Suleyman Serdar KOZAT. “Unsupervised Anomaly Detection With LSTM Neural Networks”. In : *IEEE Transactions on Neural Networks and Learning Systems* 31.8 (août 2020).
- [11] Krzysztof ZARZYCKI et Maciej ŁAWRYŃCZUK. “LSTM and GRU Neural Networks as Models of Dynamical Processes Used in Predictive Control : A Comparison of Models Developed for Two Chemical Reactors”. In : *Processes* 8.5 (2020). DOI : 10.3390/pr8050582. URL : <https://www.mdpi.com/1424-8220/21/16/5625>.
- [12] Tzu-Hsuan LIN et Jehn-Ruey JIANG. “Anomaly Detection with Autoencoder and Random Forest”. In : *2020 International Computer Symposium (ICS)*. 2020. DOI : 10.1109/ICS51289.2020.00028.
- [13] Kukjin CHOI, Jihun YI, Changhwa PARK et Sungroh YOON. “Deep Learning for Anomaly Detection in Time-Series Data : Review, Analysis, and Guidelines”. In : *IEEE Access* 9 (2021). DOI : 10.1109/ACCESS.2021.3107975.
- [14] AIML.COM. “Sequence Models Compared : RNNs, LSTMs, GRUs, and Transformers”. In : (2025).
- [15] Saidul ISLAM, Hanae ELMEEKI, Ahmed ELSEBAI, Jamal BENTAHAR, Nagat DRAWEL, Gaith RJOUB et Witold PEDRYCZ. “A comprehensive survey on applications of transformers for deep learning tasks”. In : *Expert Systems with Applications* 241 (2024). ISSN : 0957-4174. DOI : <https://doi.org/10.1016/j.eswa.2023.122666>.

- [16] Ashish VASWANI, Noam SHAZEER, Niki PARMAR, Jakob USZKOREIT, Llion JONES, Aidan GOMEZ, Lukasz KAISER et Illia POLOSUKHIN. “Attention Is All You Need”. In : (juin 2017). DOI : 10.48550/arXiv.1706.03762.
- [17] Chenwei TANG, Jialiang HUANG, Mao XU, Xu LIU, Fan YANG, Wentao FENG, Zhenan HE et Jiancheng LV. “Attention-based early warning framework for abnormal operating conditions in fluid catalytic cracking units”. In : *Applied Soft Computing* 153 (2024), p. 111275. ISSN : 1568-4946. DOI : <https://doi.org/10.1016/j.asoc.2024.111275>.
- [18] Ge HE, Lei LUO, Li ZHOU, Yiyang DAI, Xu JI, Chao GUO et Zhaopeng LU. “Deep learning prediction of yields of fluid catalytic cracking via differential evolutionary dual-stage attention-based LSTM”. In : *Fuel* 370 (2024), p. 131826. ISSN : 0016-2361. DOI : <https://doi.org/10.1016/j.fuel.2024.131826>.
- [19] Wende TIAN, Shaochen WANG, Suli SUN, Chuankun LI et Yang LIN. “Intelligent prediction and early warning of abnormal conditions for fluid catalytic cracking process”. In : *Chemical Engineering Research and Design* 181 (2022), p. 304-320. ISSN : 0263-8762. DOI : <https://doi.org/10.1016/j.cherd.2022.03.031>.
- [20] A. TRIVEDI. “Explainable Forecasting Models For Load Forecasting”. In : (juill. 2024).
- [21] Yuqi NIE, Nam H. NGUYEN, Phanwadee SINTHONG et Jayant KALAGNANAM. “A Time Series is Worth 64 Words : Long-term Forecasting with Transformers”. In : (2023). Published as a conference paper at ICLR 2023.
- [22] Zhongrun XIANG, Jun YAN et Ibrahim DEMIR. “A Rainfall-Runoff Model with LSTM-based Sequence-to-Sequence Learning”. In : *Water Resources Research* 56 (jan. 2020). DOI : 10.1029/2019WR025326.
- [31] H. ZHOU, S. ZHANG, J. PENG, S. ZHANG, J. LI, H. XIONG et W. ZHANG. “Informer : Beyond Efficient Transformer for Long Sequence Time-Series Forecasting”. In : *Proceedings of the AAAI Conference on Artificial Intelligence* 35.12 (2021). DOI : 10.1609/aaai.v35i12.17325.
- [32] David M W POWERS. “Evaluation : From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation”. In : *Journal of Machine Learning Technologies* 2.1 (2011).
- [39] Jérémy ROBERT. “Machine Learning : Définition, fonctionnement, utilisations”. In : (nov. 2020).
- [41] INTELLIGENCE ARTIFICIELLE SCHOOL. “Qu’est-ce que l’overfitting et comment s’en protéger ?” In : (Juin 2024).
- [42] INTELLIGENCE ARTIFICIELLE SCHOOL. “Underfitting : le sous-apprentissage en Machine Learning”. In : (Février 2024).
- [43] Alex GRAVES. “Sequence transduction with recurrent neural networks”. In : *arXiv preprint arXiv :1211.3711* (2012). URL : <https://arxiv.org/abs/1211.3711>.
- [45] PURE ILLUSION. *Définition de Framework : un socle pour construire une application*. Consulté en juillet 2025. Pure Illusion - Lexique Digital. Fév. 2024. URL : <https://www.pure-illusion.com/lexique/definition-de-framework>.

Webographie

- [1] MINISTÈRE DE L'ÉNERGIE , DES MINES ET DES ENERGIES RENOUVELABLES. *Produits pétroliers*. <https://www.energy.gov.dz/?rubrique=produits-petroliers>.
- [5] HUSSEIN JAWAD, MOHAMED KARIM ABID. *Détection d'anomalies dans les séries temporelles*. <https://www.quantmetry.com/blog/detection-danomalies-dans-les-series-temporelles/>.
- [23] PYTHON SOFTWARE FOUNDATION. *Python Programming Language*. <https://www.python.org/doc/essays/blurb/>.
- [24] GOOGLE BRAIN TEAM. *TensorFlow : An end-to-end open-source machine learning platform*. <https://www.tensorflow.org>.
- [25] SCIKIT-LEARN DEVELOPERS. *scikit-learn : Machine Learning in Python*. <https://scikit-learn.org>.
- [26] ANACONDA, INC.. *Conda : Package, dependency and environment management*. <https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html>.
- [27] SEBASTIÁN RAMÍREZ. *FastAPI - Modern, Fast (high-performance) web framework for building APIs with Python*. <https://fastapi.tiangolo.com>.
- [28] ENCODE OSS. *Uvicorn : The lightning-fast ASGI server implementation, using uvloop and httptools*. <https://www.uvicorn.org>.
- [29] META PLATFORMS, INC.. *React – A JavaScript library for building user interfaces*. <https://reactjs.org>.
- [30] GIT SCM. *Git - Distributed Version Control System*. <https://git-scm.com>.
- [33] CONNAISSANCE DES ÉNERGIES. *Le raffinage pétrolier*. <https://www.connaissancedesenergies.org/fiche-pedagogique/raffinage-petrolier>. Juin 2024.
- [34] CONNAISSANCE DES ÉNERGIES. *Qu'est-ce qu'un hydrocarbure et comment migre-t-il vers la surface ?* <https://www.connaissancedesenergies.org/questions-et-reponses-energies/pourquoi-et-comment-un-hydrocarbure-migre-t-il-vers-la-surface>.
- [35] OFFICE QUÉBÉCOIS DE LA LANGUE FRANÇAISE. *Hydrodésulfuration*. <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8883584/hydrodesulfuration>.
- [36] STUDYSMARTER. *Catalyseur*. <https://www.studysmarter.fr/resumes/physique-chimie/chimie/catalyseur/>.
- [37] ENCYCLOPÆDIA UNIVERSALIS. *Combustion*. <https://junior.universalis.fr/encyclopedie/combustion>.

- [38] APEC. *Fiche métier : Ingénieur procédés*. <https://www.apec.fr/tous-nos-metiers/services-techniques/ingenieur-procedes.html>.
- [40] IBM CORPORATION. *Que sont les réseaux neuronaux ?* <https://www.ibm.com/fr-fr/think/topics/neural-networks>.
- [44] Jun HOSOI. *Qu'est-ce que l'horodatage ?* <https://www.globalsign.com/fr/blog/horodatage-definition-et-fonctionnement>.
- [46] LINKWEB. *Front-end*. <https://linkweb.fr/creation-site-internet-toulouse/front-end/>.
- [47] LINKWEB. *Back-end*. <https://linkweb.fr/creation-site-internet-toulouse/back-end/>.
- [48] RED HAT. *Apache Kafka, qu'est-ce que c'est ?* URL : <https://www.redhat.com/fr/topics/integration/what-is-apache-kafka>.

Annexes

Chapitre A : Glossaire technique

A.1 Raffinage

Le raffinage du pétrole est un processus industriel essentiel qui consiste à transformer le pétrole brut (inutilisable tel quel) en produits finis exploitables comme l'essence, le fioul ou encore le naphtha. À l'état naturel, le pétrole brut est un mélange complexe et désordonné de différentes molécules d'hydrocarbures, composées principalement de carbone et d'hydrogène. Pour en tirer des produits utiles, il est nécessaire de séparer ces composants en fonction de leurs propriétés. C'est précisément ce que permet le raffinage [33].

A.2 Hydrocarbures

Les hydrocarbures sont des molécules formées uniquement d'atomes de carbone et d'hydrogène. On les retrouve principalement dans le pétrole et le gaz naturel ; ils composent la majeure partie des combustibles fossiles utilisés comme sources d'énergie [34].

A.3 Distillation atmosphérique

Cette opération vise à séparer les différents composants d'un liquide en fonction de leur température d'évaporation. Dans le cas du pétrole brut, il est chauffé à environ 400°C puis injecté dans une grande tour de distillation d'environ 60 mètres de haut.

Sous l'effet de la chaleur, les hydrocarbures qu'il contient se vaporisent progressivement : d'abord les plus légers, ensuite les intermédiaires, et enfin une partie des plus lourds. La température diminue au fur et à mesure qu'on monte dans la tour, ce qui permet à chaque type d'hydrocarbure de se condenser à un étage spécifique. Les plus légers se liquéfient en haut de la colonne, tandis que les plus lourds restent en bas [33].

A.4 Distillation sous vide

Cette étape reprend le même principe que la distillation atmosphérique, mais s'applique aux résidus les plus lourds du pétrole. Pour les traiter, on utilise une seconde colonne, plus petite, dans laquelle on crée du vide, d'où le nom de distillation sous vide. En abaissant la pression, on diminue aussi les températures d'ébullition, ce qui permet de séparer les produits lourds sans avoir à trop les chauffer, évitant ainsi de les dégrader.

Dans cette colonne, le gazole est récupéré dans la partie haute, tandis que le fioul lourd se dépose en bas. Enfin, les résidus restants (encore plus lourds) sont stockés pour être ensuite utilisés dans la fabrication de lubrifiants [33].

A.5 Hydrodésulfuration

C'est un procédé qui permet d'éliminer le soufre présent dans le pétrole brut. Pour cela, on chauffe les hydrocarbures en présence d'hydrogène, ce qui provoque une réaction chimique qui extrait le soufre sous forme de gaz. Ce traitement est essentiel pour réduire la pollution liée aux carburants et répondre aux normes environnementales [35].

A.6 Catalyseur

Un catalyseur est une substance qui facilite une réaction chimique en accélérant son déroulement, sans être consommée ni modifiée chimiquement. Il agit en abaissant l'énergie nécessaire pour que la réaction se produise, permettant ainsi aux molécules de réagir plus facilement, et réduisant les déchets [36].

A.7 Combustion

C'est une réaction chimique entre un combustible (comme du bois, du gaz ou de l'essence) et un comburant, le plus souvent l'oxygène de l'air. Par exemple, lorsque du bois brûle, l'oxygène de l'air se combine à son carbone. Cette réaction libère de la chaleur, de la lumière, et produit généralement des gaz comme le dioxyde de carbone (CO_2) et de la vapeur d'eau [37].

A.8 Ingénieur procédés

L'ingénieur procédés a un rôle très important dans le bon fonctionnement des unités de production. Il a pour tâche d'imaginer, concevoir, analyser et améliorer les procédés industriels, de sorte qu'ils soient à la fois performants, sûrs et adaptés aux besoins de l'entreprise.

Il intervient dans de nombreux domaines comme la chimie, la pharmacie, l'agroalimentaire, l'énergie ou encore la pétrochimie.

Son travail quotidien consiste à élaborer des dossiers techniques, rédiger des cahiers des charges, suivre l'installation de nouveaux équipements et s'assurer que tout fonctionne de manière fiable et sécurisée. Il garde toujours un œil critique sur les performances des procédés en place et propose des ajustements pour les rendre plus efficaces, tout en respectant les normes de qualité, de sécurité et d'environnement [38].

A.9 Apprentissage automatique (machine learning)

C'est une branche de l'intelligence artificielle qui permet à des algorithmes d'apprendre à partir de données plutôt que d'être explicitement programmés pour chaque tâche. L'idée est de fournir à l'algorithme de grandes quantités de données (nombres, textes, images, mesures...), et il apprend à en repérer les motifs récurrents, ce qu'on appelle des "patterns".

À force d'entraînement, le modèle devient capable de faire des prédictions ou de prendre des décisions sur de nouvelles données, en s'appuyant sur ce qu'il a appris. Plus il est exposé à des exemples pertinents, plus ses résultats sont précis. C'est ainsi qu'un système peut, par exemple, prédire une température, détecter une anomalie ou reconnaître une voix [39].

A.10 Réseau de neurones

Les réseaux neuronaux sont des modèles d'apprentissage automatique inspirés du fonctionnement du cerveau humain. Ils apprennent à reconnaître des motifs dans les données en imitant la manière dont nos neurones communiquent entre eux pour analyser des informations, prendre des décisions ou faire des prédictions.

Un réseau neuronal est constitué de plusieurs couches : une couche d'entrée qui reçoit les données, des couches cachées où se fait l'apprentissage, et une couche de sortie qui produit le résultat.

Chaque "neurone" artificiel est connecté à d'autres et possède un poids et un seuil. Si l'information reçue dépasse ce seuil, le neurone s'active et transmet l'information aux suivants, sinon, il reste silencieux. Grâce à un grand nombre d'exemples, le réseau ajuste ses poids pour apprendre à prédire ou classer avec précision.

Ces réseaux sont aujourd'hui capables de traiter des volumes massifs de données très rapidement, ce qui les rend essentiels dans des domaines comme la reconnaissance vocale, l'analyse d'images ou les moteurs de recherche. Google, par exemple, s'appuie sur ce type de technologies pour améliorer ses résultats de recherche.

Les réseaux neuronaux, aussi appelés RNA (réseaux neuronaux artificiels), font partie du deep learning, une branche avancée du machine learning [40].

A.11 Surapprentissage (Overfitting)

L'overfitting, ou surajustement, est un phénomène courant en apprentissage automatique (machine learning) et en modélisation. Il survient lorsque le modèle apprend trop bien les données d'entraînement, au point d'en mémoriser non seulement les tendances générales, mais aussi les bruits, anomalies ou détails spécifiques. Résultat, il fonctionne très bien sur les données qu'il connaît déjà, mais échoue à faire des prédictions fiables sur de nouvelles données. Autrement dit, il manque de capacité de généralisation.

Au lieu de capturer les structures sous-jacentes des données, le modèle devient trop spécialisé, ce qui le rend moins robuste face à des exemples inédits. On observe alors un écart important entre la performance sur l'entraînement et celle sur le test ou la validation [41].

A.12 Sous-apprentissage (Underfitting)

En machine learning, on parle d'underfitting (ou sous-ajustement) lorsqu'un modèle est incapable de bien apprendre à partir des données. En général, cela arrive quand le modèle est trop simple ou qu'il manque de capacité pour capturer les relations complexes présentes dans les données d'entraînement. Résultat, ses prédictions sont peu précises, aussi bien sur les données qu'il a vues pendant l'entraînement que sur de nouvelles données. Ce type de comportement est souvent lié à un biais élevé, c'est-à-dire que le modèle fait des hypothèses trop fortes ou trop simplistes sur les données [42].

A.13 Transduction

En machine learning, la transduction désigne un processus par lequel un modèle apprend à transformer une séquence d'entrée en une séquence de sortie. On parle souvent de transformation séquence-à-séquence (sequence-to-sequence ou seq2seq), et ce concept est au cœur de nombreuses

applications, comme la traduction automatique, la reconnaissance vocale, le résumé de texte, ou encore la génération de réponses dans les chatbots.

Contrairement à une approche purement inductive, où l'on cherche à apprendre une règle générale à partir des données pour ensuite l'appliquer à n'importe quelle situation, la transduction s'attache davantage à prédire les sorties pour des entrées précises, en exploitant directement les correspondances observées dans les données d'entraînement. L'idée n'est pas forcément de généraliser une loi abstraite, mais plutôt de réaliser la bonne transformation pour chaque paire entrée-sortie rencontrée.

Dans les architectures modernes comme les Transformers, ce mécanisme est géré via des modules d'encodeur-décodeur : l'encodeur traite la séquence d'entrée pour en extraire une représentation significative, que le décodeur utilise ensuite pour produire la séquence de sortie [43].

A.14 Horodatage

L'horodatage (ou timestamping en anglais) est une méthode qui consiste à associer une date et une heure précises à un événement ou une donnée. C'est un peu comme apposer un cachet temporel sur un fichier, un message ou une action, afin de marquer le moment exact où cela s'est produit. Ce mécanisme est omniprésent dans le monde numérique, que ce soit pour organiser des fichiers, assurer la sécurité des transactions ou garantir l'intégrité des documents [44].

A.15 Framework

Un framework web, c'est un peu comme une boîte à outils mise à disposition des développeurs. Le mot vient de l'anglais : *frame* signifie "cadre" et *work*, "travail". Concrètement, il s'agit d'un ensemble de composants prêts à l'emploi qui facilitent la création de sites web ou d'applications. Ces outils prennent en charge des tâches courantes, comme la gestion des données (CRUD), la structure du projet, la sécurité, ou encore le respect des bonnes pratiques, ce qui permet aux développeurs de gagner du temps et de se concentrer sur la logique métier plutôt que de tout reconstruire de zéro à chaque fois.

En somme, un framework pose un cadre structuré pour développer plus rapidement, plus proprement, et souvent de manière plus sécurisée [45].

A.16 Frontend

En développement web, le front-end désigne tout ce que l'utilisateur peut voir et interagir avec directement sur un site ou une application web. Cela comprend par exemple les menus, les boutons, les images, les textes ou encore les animations. C'est, en quelque sorte, la partie visible de l'iceberg [46].

A.17 Backend

À l'opposé du front-end, le back-end s'occupe de tout ce qui se passe en coulisses : traitement des données, logique métier, interactions avec les bases de données, etc [47].

A.18 Kafka

Apache Kafka est une plateforme distribuée conçue pour gérer des flux de données en temps réel. Elle permet de publier, stocker, traiter et consommer des flux d'événements (ou logs) de manière rapide et fiable.

Ce qui la rend unique, c'est sa capacité à faire transiter d'énormes volumes de données entre différentes applications ou systèmes, et ce, en simultané. Elle agit comme un hub central qui relie de nombreuses sources de données (capteurs, services, applications...) à des systèmes consommateurs, le tout en assurant une haute disponibilité et une faible latence.

En résumé, Kafka ne se contente pas de transférer les données d'un point A à un point B : elle offre la souplesse de les faire circuler entre autant de points que nécessaire, de façon efficace et en continu [48].

Chapitre B : Tableaux de données

B.1 Affinage des modèles de prédiction

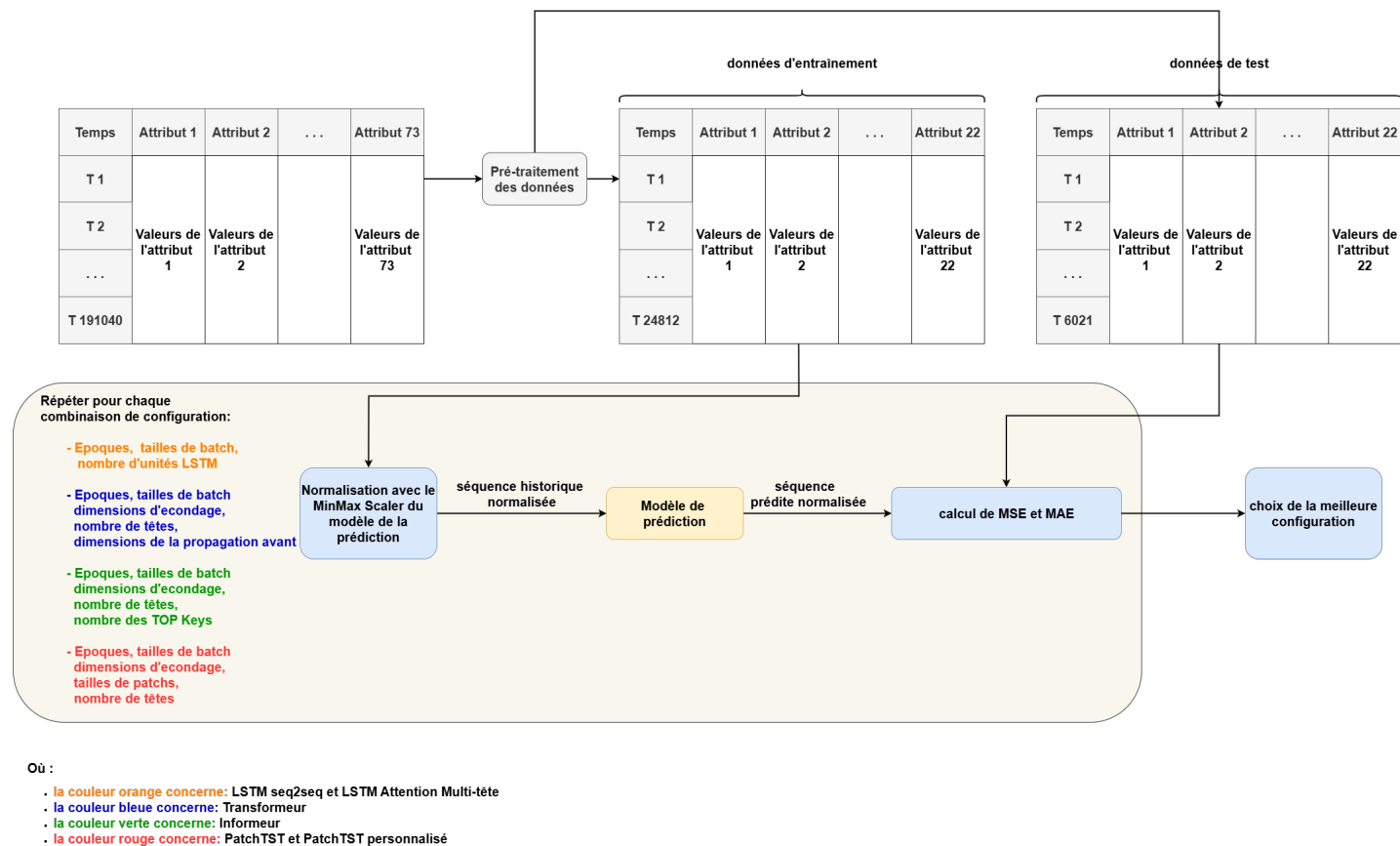


FIGURE B.1 – Pipeline d'affinage des modèles de prédiction

Ce schéma illustre le pipeline pour la prédiction. Il intègre différents types de modèles (LSTM, Transformer, PatchTST) et explore plusieurs configurations. L'évaluation se fait à l'aide de métriques telles que MSE et MAE sur des données de test, afin d'identifier la meilleure architecture pour la tâche prédictive.

B.1.1 Modèle LSTM seq2seq

Epochs	Batch Size	LSTM Units	Validation MSE	Validation MAE
50	128	32	0.005997	0.054246
50	128	64	0.004316	0.044812
50	128	128	0.003224	0.036543
50	256	32	0.007994	0.060472
50	256	64	0.004210	0.043538
50	256	128	0.003644	0.038909
50	512	32	0.011214	0.069197
50	512	64	0.007929	0.061060
50	512	128	0.003951	0.043494
50	1024	32	0.014451	0.078944
50	1024	64	0.012268	0.073205
50	1024	128	0.009507	0.064183

TABLE B.1 – Résultats d’affinage des hyperparamètres du modèle LSTM seq2seq

B.1.2 Modèle LSTM Attention multi-tête

Epochs	Batch Size	LSTM Units	Val MSE	Val MAE
50	128	32	0.005229	0.052088
50	128	64	0.004634	0.048936
50	128	128	0.004267	0.046870
50	256	32	0.007160	0.062592
50	256	64	0.006300	0.057995
50	256	128	0.004386	0.047417
50	512	32	0.008040	0.068366
50	512	64	0.008046	0.066489
50	512	128	0.005549	0.054649
50	1024	32	0.009999	0.075977
50	1024	64	0.009509	0.075292
50	1024	128	0.007346	0.064754

TABLE B.2 – Résultats d’affinage du modèle LSTM Attention multi-tête

B.1.3 Modèle Transformers

Epochs	Batch Size	Embed Dims	Heads	FF Dims	Val MSE	Val MAE
50	128	64	4	64	0.005926	0.056789
50	128	64	4	128	0.006988	0.063287
50	128	64	8	64	0.006869	0.060477
50	128	64	8	128	0.006230	0.055680
50	128	64	16	64	0.008064	0.062720

Suite page suivante

Epochs	Batch Size	Embed Dims	Heads	FF Dims	Val MSE	Val MAE
50	128	64	16	128	0.008132	0.063874
50	128	128	4	64	0.008560	0.066405
50	128	128	4	128	0.006056	0.057270
50	128	128	8	64	0.007906	0.068401
50	128	128	8	128	0.007060	0.061485
50	128	128	16	64	0.006439	0.056572
50	128	128	16	128	0.008873	0.071576
50	256	64	4	64	0.006935	0.060065
50	256	64	4	128	0.007436	0.060944
50	256	64	8	64	0.008264	0.064801
50	256	64	8	128	0.007423	0.063240
50	256	64	16	64	0.007065	0.060548
50	256	64	16	128	0.007112	0.060548
50	256	128	4	64	0.006300	0.057663
50	256	128	4	128	0.006947	0.058937
50	256	128	8	64	0.007933	0.062416
50	256	128	8	128	0.007735	0.063369
50	256	128	16	64	0.007505	0.062710
50	256	128	16	128	0.008240	0.063906
50	512	64	4	64	0.007632	0.063304
50	512	64	4	128	0.007905	0.063994
50	512	64	8	64	0.007676	0.062083
50	512	64	8	128	0.008072	0.065382
50	512	64	16	64	0.007370	0.060062
50	512	64	16	128	0.007335	0.060547
50	512	128	4	64	0.007264	0.059671
50	512	128	4	128	0.007694	0.063475
50	512	128	8	64	0.008014	0.063659
50	512	128	8	128	0.007642	0.061440
50	512	128	16	64	0.007672	0.061585
50	512	128	16	128	0.007950	0.062513
50	1024	64	4	64	0.009156	0.070008
50	1024	64	4	128	0.009934	0.072953
50	1024	64	8	64	0.009434	0.068627
50	1024	64	8	128	0.009784	0.069658
50	1024	64	16	64	0.007710	0.064009
50	1024	64	16	128	0.008552	0.066936
50	1024	128	4	64	0.008875	0.066876
50	1024	128	4	128	0.008524	0.065175
50	1024	128	8	64	0.008043	0.062020
50	1024	128	8	128	0.008534	0.065615
50	1024	128	16	64	0.007704	0.063836
50	1024	128	16	128	0.009210	0.068030

TABLE B.3 – Résultats d’affinage des hyperparamètres pour le modèle Transformers

B.1.4 Modèle PatchTST personnalisé

Epochs	Batch Size	Embed Dims	NB Heads	Patch Len	Val MSE	Val MAE
50	128	64	4	2	0.004572	0.046815
50	128	64	8	2	0.005710	0.057380
50	128	64	16	2	0.005645	0.053070
50	128	128	4	2	0.004062	0.045966
50	128	128	8	2	0.004417	0.047632
50	128	128	16	2	0.004784	0.049193
50	128	64	4	4	0.004591	0.049393
50	128	64	8	4	0.005717	0.055670
50	128	64	16	4	0.004181	0.045743
50	128	128	4	4	0.004048	0.043744
50	128	128	8	4	0.003991	0.045914
50	128	128	16	4	0.003849	0.042934
50	128	64	4	8	0.004667	0.047639
50	128	64	8	8	0.005014	0.050723
50	128	64	16	8	0.005640	0.055338
50	128	128	4	8	0.004288	0.046365
50	128	128	8	8	0.005089	0.051889
50	128	128	16	8	0.004107	0.044572
50	128	64	4	16	0.005740	0.053985
50	128	64	8	16	0.005063	0.052321
50	128	64	16	16	0.005930	0.053320
50	128	128	4	16	0.004166	0.045562
50	128	128	8	16	0.004491	0.047817
50	128	128	16	16	0.004091	0.044412
50	256	64	4	2	0.007412	0.061596
50	256	64	8	2	0.005732	0.053764
50	256	64	16	2	0.006219	0.056458
50	256	128	4	2	0.004718	0.047602
50	256	128	8	2	0.004907	0.050799
50	256	128	16	2	0.006413	0.058012
50	256	64	4	4	0.006821	0.059621
50	256	64	8	4	0.006136	0.055580
50	256	64	16	4	0.007815	0.061784
50	256	128	4	4	0.005898	0.055383
50	256	128	8	4	0.004866	0.049608
50	256	128	16	4	0.005335	0.052847
50	256	64	4	8	0.007130	0.059261
50	256	64	8	8	0.008550	0.065350
50	256	64	16	8	0.006487	0.057708
50	256	128	4	8	0.005169	0.050707
50	256	128	8	8	0.004891	0.049795
50	256	128	16	8	0.006661	0.056748

Suite page suivante

Epochs	Batch Size	Embed Dims	NB Heads	Patch Len	Val MSE	Val MAE
50	256	64	4	16	0.005815	0.054220
50	256	64	8	16	0.006675	0.058653
50	256	64	16	16	0.006756	0.059121
50	256	128	4	16	0.006505	0.056513
50	256	128	8	16	0.006401	0.056052
50	256	128	16	16	0.006438	0.058013
50	512	64	4	2	0.008760	0.067329
50	512	64	8	2	0.009023	0.066034
50	512	64	16	2	0.008816	0.065381
50	512	128	4	2	0.005835	0.053774
50	512	128	8	2	0.006330	0.059739
50	512	128	16	2	0.008062	0.063963

TABLE B.4 – Résultats d’affinage des hyperparamètres pour le modèle PatchTST personnalisé

B.1.5 Modèle PatchTST

Epochs	Batch Size	Embed Dims	NB Heads	Patch Len	Val MSE	Val MAE
50	128	64	4	4	0.052292	0.186301
50	128	64	8	4	0.052318	0.180447
50	128	64	4	8	0.051700	0.185856
50	128	64	8	8	0.051289	0.184532
50	256	64	4	4	0.075479	0.227473
50	256	64	8	4	0.076349	0.229757
50	256	64	4	8	0.060624	0.205952
50	256	64	8	8	0.057463	0.195235
50	512	64	4	4	0.076720	0.230387
50	512	64	8	4	0.077367	0.233348
50	512	64	4	8	0.070898	0.226341
50	512	64	8	8	0.068396	0.218516
50	1024	64	4	4	0.078930	0.228741
50	1024	64	8	4	0.078477	0.231129
50	1024	64	4	8	0.076609	0.232774
50	1024	64	8	8	0.074788	0.226565

TABLE B.5 – Résultats d’affinage des hyperparamètres du modèle PatchTST

B.1.6 Modèle Informers

Epochs	Batch Size	Embed Dims	NB Heads	TOP-K	Val MSE	Val MAE
50	128	64	4	2	0.005777	0.051240
50	128	64	4	4	0.004958	0.049834
50	128	64	4	8	0.005548	0.053524

Suite page suivante

Epochs	Batch Size	Embed Dims	NB Heads	TOP-K	Val MSE	Val MAE
50	128	64	4	16	0.004278	0.045901
50	128	64	8	2	0.004784	0.047207
50	128	64	8	4	0.004599	0.048533
50	128	64	8	8	0.005324	0.052192
50	128	64	8	16	0.004507	0.046693
50	128	64	16	2	0.004520	0.048066
50	128	64	16	4	0.004835	0.050274
50	128	64	16	8	0.005322	0.052695
50	128	64	16	16	0.006006	0.056856
50	128	128	4	2	0.004860	0.051468
50	128	128	4	4	0.003937	0.043095
50	128	128	4	8	0.003883	0.043374
50	128	128	4	16	0.004808	0.047303
50	128	128	8	2	0.004053	0.043693
50	128	128	8	4	0.004523	0.048474
50	128	128	8	8	0.004955	0.048839
50	128	128	8	16	0.004507	0.048151
50	128	128	16	2	0.004423	0.046931
50	128	128	16	4	0.004509	0.047946
50	128	128	16	8	0.003800	0.041697
50	128	128	16	16	0.003901	0.043715
50	256	64	4	2	0.004948	0.051149
50	256	64	4	4	0.006739	0.057694
50	256	64	4	8	0.006879	0.058218
50	256	64	4	16	0.006924	0.059443
50	256	64	8	2	0.005759	0.055248
50	256	64	8	4	0.004862	0.049653
50	256	64	8	8	0.006832	0.057646
50	256	64	8	16	0.006011	0.055861
50	256	64	16	2	0.005868	0.054324
50	256	64	16	4	0.006582	0.056378
50	256	64	16	8	0.006431	0.056175
50	256	64	16	16	0.006490	0.057704
50	256	128	4	2	0.004582	0.047125
50	256	128	4	4	0.004551	0.046481
50	256	128	4	8	0.004185	0.045418
50	256	128	4	16	0.004554	0.049928
50	256	128	8	2	0.005278	0.049666
50	256	128	8	4	0.005427	0.054112
50	256	128	8	8	0.004272	0.044903
50	256	128	8	16	0.005389	0.052953
50	256	128	16	2	0.005193	0.051392
50	256	128	16	4	0.004615	0.049955
50	256	128	16	8	0.005016	0.050729
50	256	128	16	16	0.003996	0.043198

Suite page suivante

Epochs	Batch Size	Embed Dims	NB Heads	TOP-K	Val MSE	Val MAE
50	512	64	4	2	0.008750	0.068172
50	512	64	4	4	0.007760	0.063138
50	512	64	4	8	0.008343	0.064216
50	512	64	4	16	0.008574	0.064941
50	512	64	8	2	0.008003	0.063921
50	512	64	8	4	0.005462	0.053656
50	512	64	8	8	0.008392	0.065625
50	512	64	8	16	0.006466	0.054236
50	512	64	16	2	0.008264	0.065233
50	512	64	16	4	0.007098	0.057937
50	512	64	16	8	0.007798	0.063587
50	512	64	16	16	0.008210	0.066267
50	512	128	4	2	0.004681	0.050289
50	512	128	4	4	0.006740	0.057173
50	512	128	4	8	0.005343	0.051870
50	512	128	4	16	0.005364	0.053690
50	512	128	8	2	0.005372	0.050015
50	512	128	8	4	0.006016	0.056329
50	512	128	8	8	0.004284	0.046015
50	512	128	8	16	0.006843	0.059557
50	512	128	16	2	0.003918	0.043692
50	512	128	16	4	0.005125	0.050467
50	512	128	16	8	0.006662	0.059284
50	512	128	16	16	0.005479	0.054048
50	1024	64	4	2	0.010051	0.069788
50	1024	64	4	4	0.011642	0.075615
50	1024	64	4	8	0.009926	0.070596
50	1024	64	4	16	0.009999	0.069041
50	1024	64	8	2	0.009965	0.070952
50	1024	64	8	4	0.009297	0.071173
50	1024	64	8	8	0.009481	0.069527
50	1024	64	8	16	0.009896	0.068786
50	1024	64	16	2	0.009999	0.072491
50	1024	64	16	4	0.010320	0.071783
50	1024	64	16	8	0.008901	0.067211
50	1024	64	16	16	0.009526	0.068801
50	1024	128	4	2	0.007720	0.062927
50	1024	128	4	4	0.008411	0.066207
50	1024	128	4	8	0.007204	0.060501
50	1024	128	4	16	0.008653	0.067370
50	1024	128	8	2	0.007682	0.062292
50	1024	128	8	4	0.007309	0.062369
50	1024	128	8	8	0.007017	0.061715
50	1024	128	8	16	0.006978	0.063092
50	1024	128	16	2	0.008204	0.064381

Suite page suivante

Epochs	Batch Size	Embed Dims	NB Heads	TOP-K	Val MSE	Val MAE
50	1024	128	16	4	0.006945	0.060834
50	1024	128	16	8	0.006829	0.057997
50	1024	128	16	16	0.007466	0.060394

TABLE B.6 – Résultats d’affinage des hyperparamètres du modèle Informer

B.2 Affinage des modèles de détection

B.2.1 Modèle Autoencodeur BiLSTM

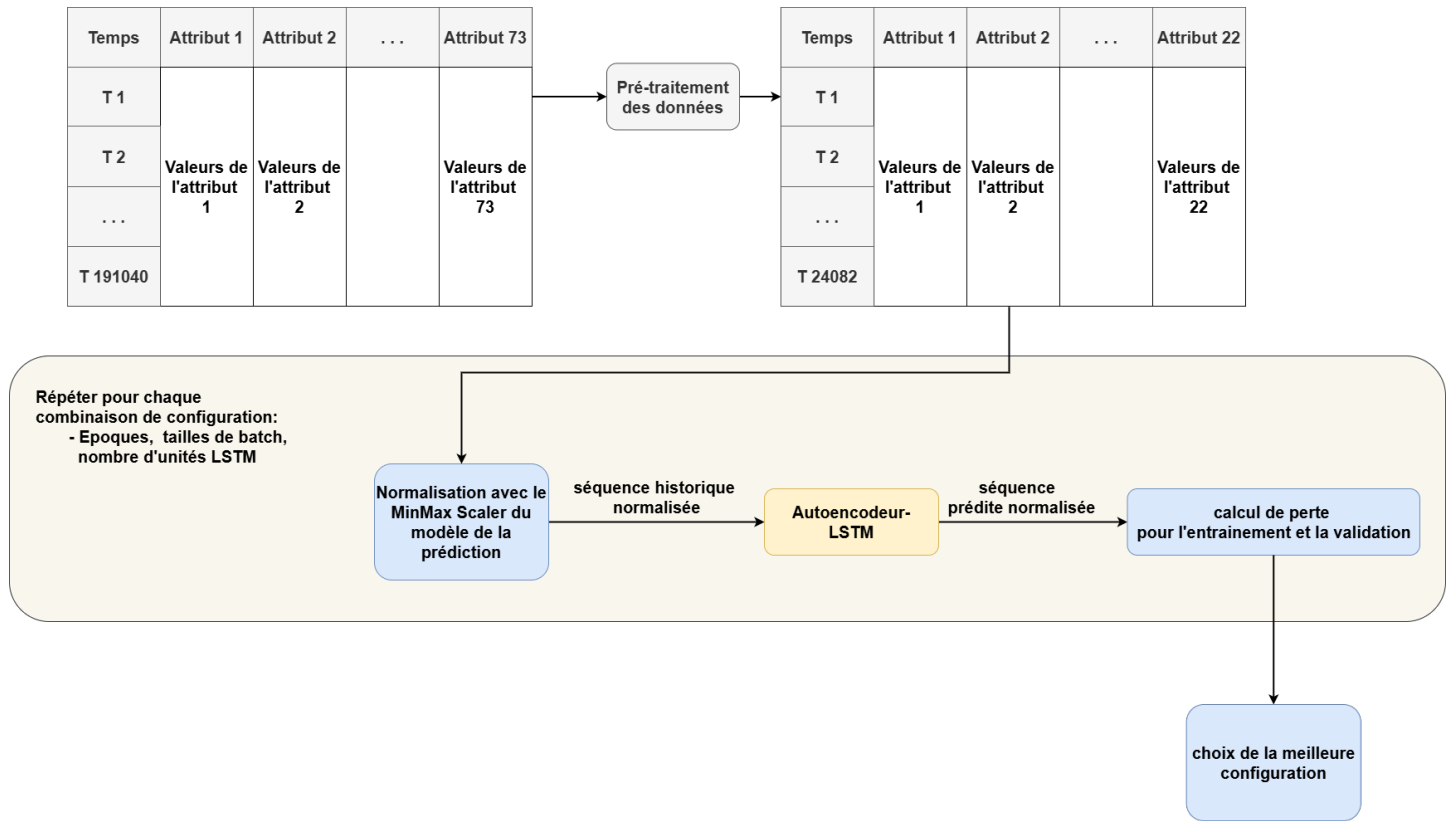


FIGURE B.2 – Pipeline d’affinage du modèle de détection Autoencodeur BiLSTM

Le schéma ci-dessus représente le processus d’affinage pour un modèle de détection d’anomalies. Il utilise un autoencodeur BiLSTM pour apprendre la reconstruction de séquences normalisées. Les performances sont évaluées via une fonction de perte sur les données d’entraînement et de validation, permettant de sélectionner la meilleure configuration.

Index	Epochs	Batch Size	LSTM Units	Train Loss	Val Loss
0	50	64	64	0.002576	0.002059
1	50	64	128	0.003624	0.001833
2	50	64	256	0.002261	0.001736
3	50	128	64	0.003050	0.003058
4	50	128	128	0.001814	0.001532
5	50	128	256	0.001989	0.001695
6	50	256	64	0.003080	0.003139
7	50	256	128	0.002118	0.001992
8	50	256	256	0.001727	0.001686
9	50	512	64	0.004868	0.004047
10	50	512	128	0.004095	0.002788
11	50	512	256	0.002569	0.002320
12	50	1024	64	0.009330	0.010072
13	50	1024	128	0.004693	0.005289
14	50	1024	256	0.005327	0.005554

TABLE B.7 – Résultats d’affinage des hyperparamètres du modèle Autoencodeur BiLSTM